

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
CAPSTONE PROJECT
SEMESTER 253, ACADEMIC YEAR 2025-2026**

**DEVELOPING A PERSON SEARCH SYSTEM FROM
MULTIMODAL DESCRIPTIONS**

MAJOR: COMPUTER SCIENCE

COUNCIL: 1CC

SUPERVISOR: Dr. Le Thanh Sach

REVIEWER: Dr. Tran Tuan Anh

—o0o—

STUDENT 1: Nguyen Cong Huy - 2113499

STUDENT 2: Nguyen Huu Cuong - 2252098

HO CHI MINH CITY, OCTOBER 2026

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
CAPSTONE PROJECT
SEMESTER 253, ACADEMIC YEAR 2025-2026**

**DEVELOPING A PERSON SEARCH SYSTEM FROM
MULTIMODAL DESCRIPTIONS**

MAJOR: COMPUTER SCIENCE

COUNCIL: 1CC

SUPERVISOR: Dr. Le Thanh Sach

REVIEWER: Dr. Tran Tuan Anh

—o0o—

STUDENT 1: Nguyen Cong Huy - 2113499

STUDENT 2: Nguyen Huu Cuong - 2252098

HO CHI MINH CITY, OCTOBER 2026

Declaration of Authenticity

We declare that the capstone project “Developing a Person Search System from Multi-modal Descriptions” is our own work, carried out under the supervision of Dr. Le Thanh Sach, Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology - Vietnam National University Ho Chi Minh City; the contribution of each of us is stated on the Work Contribution page.

The system architecture, the application source code, the research pipeline, and the experimental data and results presented in this report are genuine and were produced by us. The third-party components used in the project, including the YOLO11 object detector, the ByteTrack tracker, the RaSa model with its published checkpoint, the CLIP, Qwen2.5-VL and Gemini models, the WILDTRACK and Market-1501 datasets and the open-source libraries, are credited to their sources and used under their licence terms, for academic and non-commercial purposes only. Every document, algorithm and idea of other authors that we consulted is fully cited.

With the approval of our supervisor, we used artificial intelligence tools, including Claude (Anthropic), to assist with writing source code, testing and drafting this report. Everything produced with this assistance was checked by us before being included.

We take full responsibility before the Council and the University for any fraud, plagiarism or copyright infringement that may be found. Ho Chi Minh City University of Technology - Vietnam National University Ho Chi Minh City bears no responsibility for any copyright infringement that may occur in this project.

Ho Chi Minh City, October 2026

Student 1

Student 2

(Signature and full name) *(Signature and full name)*

Nguyen Cong Huy

Nguyen Huu Cuong

Acknowledgements

We would like to express our sincere gratitude to Dr. Le Thanh Sach, who guided and supported us throughout this project. His advice on the direction of the topic, his comments on the system architecture and his encouragement helped us reach the goals we set out.

We are grateful to the lecturers of the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology - Vietnam National University Ho Chi Minh City, for giving us access to the resources we needed and for the foundation of knowledge they taught us during our years at the university, on which this project is built. We also thank the authors of the WILDTRACK and Market-1501 datasets, of the RaSa, CLIP and Qwen models and of the ZARA framework, and the open-source community, whose published work this project builds on.

Above all, we thank our families and friends, who gave us time, financial support and encouragement throughout the project.

Given our limited expertise and practical experience, this report surely still has shortcomings. We look forward to the assessment and comments of our reviewer, Dr. Tran Tuan Anh, and of the members of the Council, so that the project can be improved further.

Ho Chi Minh City, October 2026

The authors

Nguyen Cong Huy Nguyen Huu Cuong

Work Contribution

The project has two parts that were carried out by different members. **Part I** (Chapters 2 to 8) builds and evaluates the person search application. **Part II** (Chapters 9 to 11 and Appendix A) studies an LLM-based reasoning layer that re-ranks the results of text-based person search. The table below lists the work of each member.

Member	Work	Report
Nguyen Cong Huy 2113499	Survey of related systems; requirements, use cases and system architecture; design and implementation of the application (application server, background AI worker, web application, three data stores); deployment on the target machine, preparation of the demonstration data; software testing and evaluation of the application.	Part I: Chapters 2 to 8
Nguyen Huu Cuong 2252098	Research on improving the search block: attribute index built with a vision-language model, multi-agent LLM reasoning funnel and its prompts, evaluation protocol and tooling, experiments on Market-1501, error analysis and ablations.	Part II: Chapters 9 to 11, Appendix A
Both members	Topic definition and scope; Introduction and Conclusion; front matter.	Chapters 1 and 12

Abstract

When a person must be found in the data of a surveillance camera system, operators usually review the video by hand, which takes a long time and easily misses things as the number of cameras and the recording time grow. This project builds a web application that helps search for people in camera data, letting the user describe the person in one of three forms: a reference image, an English description, or a set of predefined appearance attributes. The system only proposes and ranks results; the decision whether a result is the person being sought remains with the user.

The application has two flows. The indexing flow runs in a background worker: video from an RTSP stream or an uploaded file is sampled, people are detected with YOLO11n and tracked with ByteTrack to group the frames of one person into an appearance, and the representative frame is encoded into a vector with the RaSa model. The data is written to three stores: PostgreSQL for the business data, Milvus for the vectors and MinIO for the frames. The search flow runs in a Flask application server: the query is encoded into the same vector space and matched only within the cameras the user is allowed to see. A React web application serves three roles, the Administrator, the Operator and the Viewer, with functions for managing cameras and AI models, searching, viewing results in their original frame and saving results into case files.

The application was deployed on a CPU-only laptop and tested with the seven cameras of the WILDTRACK dataset, of which 1,488 tracks from the first 120 seconds of each video were indexed without errors. It passes 713 unit tests together with integration and end-to-end tests, and the demonstration script for all three roles ran automatically twice in a row with 15/15 steps passed. Search by image performs well, while search by text and by attributes is limited by the gap between the model's training data and person images cropped from cameras. The analysis is about seven times slower than real time, so the system processes the cameras in turn.

To address the weakness of text search, the second part of the project studies a semantic reasoning layer on top of CLIP retrieval. Each person crop is described offline by a vision-language model (Qwen2.5-VL-3B) into eighteen human-readable attributes with per-region image-quality statistics; at query time, CLIP returns a candidate pool and a multi-agent funnel driven by a small language model (Gemini Flash-Lite) re-ranks it from the attribute evidence, over a coarse and a fine round. On 100 identities of a 496-identity subset of Market-1501, the four-stage funnel raises $R@10$ from 33.0% for CLIP to 56.0%, rescuing 28 of the 67 identities CLIP missed while losing 5 of the 33 it had found. Together, the two parts show that a complete multimodal person search system can be built from existing models on commodity hardware, and that reasoning over readable attribute evidence is a promising way to improve its weakest form of search.

Chapter Summary

The report has a shared introduction, two parts carried out by different members (see the Work Contribution page), and a shared conclusion.

Chapter 1: Introduction

Presents the difficulty of finding a person in a large amount of surveillance video by manual review, and sets the objectives of building an application that searches for people by image, description or appearance features, and of studying how to improve search from text descriptions. It also defines the scope and the limitations of the project.

Part I: The Person Search Application

Chapter 2: Related Systems

Surveys three commercial systems that search for people in video, BriefCam, Avigilon and Verkada, compares them on criteria relevant to the project, and derives the directions of the system.

Chapter 3: Theoretical Background

Presents person search and cross-modal retrieval and the techniques the application uses: person detection, multi-object tracking, frame sampling, joint image-text representation learning with the RaSa model, vector search with the HNSW index, and the RTSP protocol used to simulate cameras.

Chapter 4: System Analysis

Defines the three actors, the Administrator, the Operator and the Viewer, lists the functional and non-functional requirements, presents the use case diagram and the specifications of the main use cases, and ends with the permission matrix.

Chapter 5: System Design

Presents the architecture that separates the AI processing from the application server, the two main flows of indexing and search, the data design over the PostgreSQL, Milvus and MinIO stores, the application programming interface, the security mechanisms and the user interface design.

Chapter 6: System Implementation

Explains the choice of technologies and models, describes the implementation of the background AI worker and the web application, presents the main screens by role, and the operation monitoring tools for the Administrator.

Chapter 7: System Deployment

Describes the deployment environment on a CPU-only laptop, the start-up, backup and restore procedures, and the preparation of the demonstration data from the seven cameras of the WILDTRACK dataset.

Chapter 8: Testing and Evaluation

Presents the results of testing at five levels, from unit tests to the demonstration script, the evaluation of the three forms of search, and the measurements of the sampling interval, RTSP handling, search latency, storage and memory. The measurement that was not carried out is stated together with its method.

Part II: Semantic Reasoning for Text-Based Person Search

Chapter 9: Motivation and Overview

States the problem and the contributions of the research, reads the pipeline as a template of replaceable blocks, argues for improving the search block with a reasoning layer inspired by ZARA, and fills each block with a concrete model.

Chapter 10: Attribute Index and Multi-Agent Reasoning

Describes the attribute index built with a vision-language model and the four-stage multi-agent funnel that re-ranks the CLIP candidate pool from that evidence.

Chapter 11: Experiments and Results

Presents the experimental setup on Market-1501, the results against the CLIP base-

line decomposed into rescued and lost identities, a targeted re-run on a larger model, the ablations of the prompt mechanisms and the evaluation tooling.

Chapter 12: Conclusion and Future Work

Summarises the results of both parts against the objectives, the limitations confirmed by the experiments and the directions for future work, including how the two parts could be brought together.

Contents

Declaration of Authenticity	i
Acknowledgements	ii
Work Contribution	iii
Abstract	iv
Chapter Summary	vi
List of Figures	xvi
List of Tables	xviii
List of Abbreviations	xxi
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	2
1.3 Scope	3
1.4 Limitations	5
I The Person Search Application	7
2 Related Systems	8
2.1 Existing Systems on the Market	8
2.1.1 BriefCam	8
2.1.2 Avigilon Appearance Search	9
2.1.3 Verkada AI-Powered Search	10

2.2	Comparison and Discussion	11
2.3	Development Directions	13
3	Theoretical Background	15
3.1	Person Search and Cross-Modal Retrieval	15
3.1.1	Problem Statement	15
3.1.2	Three Forms of Query	16
3.1.3	Approach of the Project	17
3.2	Person Detection	18
3.2.1	Object Detection	18
3.2.2	Two-Stage and One-Stage Detectors	18
3.2.3	The YOLO Family	18
3.2.4	Post-Processing: Confidence Threshold and Duplicate Removal	19
3.2.5	Role of Detection in the System	19
3.3	Multi-Object Tracking	19
3.3.1	Problem and Basic Components	19
3.3.2	ByteTrack	20
3.3.3	BoT-SORT	20
3.3.4	Track Life Cycle in the System	20
3.4	Frame Sampling	21
3.5	Joint Image-Text Representation Learning	22
3.5.1	Joint Embedding Space and Contrastive Learning	22
3.5.2	The RaSa Model	23
3.5.3	Use of RaSa in the System	23
3.6	Vector Search	24
3.6.1	Nearest Neighbour Search and the HNSW Index	24
3.6.2	Vector Search with Filters	25
3.6.3	Vector Search in the System	25
3.7	RTSP and Camera Stream Simulation	26
3.7.1	H.264 Compressed Video	26
3.7.2	RTSP and RTP	26
3.7.3	Simulating Cameras with a Streaming Server	26

3.7.4	Receiving RTSP Streams in the System	27
4	System Analysis	28
4.1	Actors	28
4.1.1	Administrator	29
4.1.2	Operator	29
4.1.3	Viewer	30
4.1.4	Principles of Role Separation	30
4.2	Functional Requirements	31
4.2.1	Common Functions	31
4.2.2	Administrator Functions	31
4.2.3	Operator Functions	32
4.2.4	Viewer Functions	34
4.2.5	Automatic System Functions	34
4.3	Non-Functional Requirements	34
4.3.1	Security and Access Control	35
4.3.2	Data Integrity and Recoverability	35
4.3.3	Performance and Resources	36
4.3.4	Traceability, Usability and Maintainability	36
4.4	Use Case Diagram	37
4.5	Use Case Specifications	40
4.6	Permission Matrix	47
5	System Design	49
5.1	System Architecture	49
5.1.1	Components	49
5.1.2	Design Principles	51
5.2	Main Processing Flows	52
5.2.1	Camera Data Processing Flow	52
5.2.2	Representative Frame Selection	55
5.2.3	Search Flow	55
5.2.4	Case Management Flow	57

5.3	Database Design	59
5.3.1	Relational Database	59
5.3.2	Feature Vector Storage	61
5.3.3	Frame Storage	63
5.3.4	Consistency Across the Stores	63
5.4	Application Programming Interface Design	65
5.5	Security and Access Control Design	66
5.5.1	Authentication and Sessions	66
5.5.2	Cross-Site Request Forgery Protection	67
5.5.3	Access Control	67
5.5.4	Data Protection and Abuse Limitation	68
5.6	User Interface Design	68
5.6.1	Screen Structure by Role	68
5.6.2	Design Principles	70
6	System Implementation	71
6.1	Evaluation and Choice of Technologies and Models	71
6.1.1	Front-End Technologies	72
6.1.2	Server-Side Technologies	73
6.1.3	Database and Storage	73
6.1.4	AI Models	74
6.2	Background AI Worker Implementation	75
6.2.1	Model Management and Packaging	75
6.2.2	Resource Limits and Error Handling	75
6.3	User Interface Implementation	76
6.3.1	Navigation and Communication with the Server	76
6.3.2	Rendering Person Images in the Results	77
6.3.3	Main Screens	77
6.4	Operation Monitoring	81
7	System Deployment	84
7.1	Environment and Deployment Procedure	84

7.2	Test Data and Simulated Cameras	85
8	Testing and Evaluation	86
8.1	Software Testing	86
8.2	Search Quality Evaluation	88
8.3	Performance and Camera Stream Handling	89
II	Semantic Reasoning for Text-Based Person Search	93
9	Semantic Reasoning: Motivation and Overview	94
9.1	Motivation and Contributions	94
9.1.1	Problem	94
9.1.2	Why a reasoning layer	94
9.1.3	Contributions	95
9.2	The Pipeline, and Where to Improve It	96
9.2.1	The pipeline in the abstract	96
9.2.2	Possible directions for improvement	97
9.2.3	ZARA: related work and inspiration	98
9.3	System Overview	99
9.3.1	Filling the slots	99
9.3.2	Offline indexing	100
9.3.3	Online search	101
10	Attribute Index and Multi-Agent Reasoning	103
10.1	The Attribute Index	103
10.1.1	Schema	103
10.1.2	Visual context	104
10.1.3	Three different counts	105
10.1.4	The image-quality problem, and why voting does not work . . .	106
10.2	Multi-Agent Semantic Reasoning	106
10.2.1	The funnel	106
10.2.2	Stage 0: query analyst	108

10.2.3	Stage 1: attribute-value selector	109
10.2.4	Stage 2A: evidence reader	111
10.2.5	Stage 2B: the ranker	113
10.2.6	What the CLIP score is worth inside the pool	115
10.2.7	Stages 3 and 4	116
10.2.8	Engineering	116
11	Experiments and Results	117
11.1	Experimental Setup	117
11.1.1	Dataset	117
11.1.2	Models	118
11.1.3	Funnel sizes and token budget	119
11.1.4	Baseline	119
11.1.5	Metrics	120
11.2	Results	120
11.2.1	Main comparison	120
11.2.2	What the funnel changed: rescued and lost	121
11.2.3	Where the losses happen	122
11.2.4	Is the ranker the bottleneck? A targeted re-run	123
11.2.5	Case studies	124
11.2.6	Error analysis	126
11.3	Ablations and Design Decisions	127
11.4	Evaluation Tooling and Reproducibility	129
12	Conclusion and Future Work	131
12.1	Achievements	131
12.1.1	Part I: The Application	131
12.1.2	Part II: The Reasoning Layer	132
12.2	Limitations	133
12.2.1	Part I: The Application	133
12.2.2	Part II: The Reasoning Layer	133
12.3	Future Work	135

12.3.1	Part I: The Application	135
12.3.2	Bringing the Two Parts Together	136
References		137
A	System Prompts of Part II	143
A.1	Stage 0: query analyst	143
A.2	Stage 1: attribute-value selector	146
A.3	Stage 3: the refine selector's insertion	149
A.4	Stage 2A / 4A: evidence reader	149
A.5	Stage 2B / 4B: the ranker	159
A.6	The coverage blocks	172

List of Figures

2.1	The search and filtering interface of BriefCam [5]	9
2.2	Avigilon Control Center with a marked subject [8]	10
2.3	Results of a natural-language query on the Verkada Command platform [9]	11
4.1	Use case diagram of the system	38
5.1	Overall architecture of the system	50
5.2	State diagram of a processing job	53
5.3	Flowchart of one processing job	54
5.4	Sequence diagram of the search flow	56
5.5	Sequence diagram of saving a result into a case	58
5.6	Entity-relationship diagram of the relational database	60
5.7	State diagram of an appearance	64
5.8	Screen flow by role	69
6.1	Person search by image	78
6.2	Person search by attributes	79
6.3	The result detail dialog and the case creation dialog	79
6.4	The Operator’s case management screen	80
6.5	The Viewer’s overview screen	80
6.6	Camera management screen	81
6.7	AI model configuration screen	81
6.8	Uploaded video processing screen	82
6.9	System status screen	82
6.10	AI check screen	83

9.1	The pipeline as a template. Offline: video → object detection → tracker → image encoder → database. Online: text query → text encoder → similarity search against the same database. Every box is a slot for any detector, tracker or encoder.	96
9.2	ZARA’s retrieval of discriminative evidence for candidate classes, the pattern this project adapts.	99
9.3	Extended offline indexing. The Qwen2.5-VL-3B branch turns each crop into 18 readable attributes alongside its embedding, which the reasoning stages need.	101
9.4	Online search with the reasoning layer. CLIP produces the candidates and the funnel orders them. The user’s text reaches the reasoning block directly and is not summarised by the retrieval stage.	101
10.1	The semantic reasoning block. The knowledge base supplies per-attribute statistics to both selector calls; the text query reaches the pruning and de- cision agents directly.	107
10.2	The selector in the abstract. It decides <i>where to look</i> , never what is true: the output is a set of pairs with reasons, not a verdict.	110
10.3	The same stage with its real inputs and output: the two prior tables as input, an attribute-value pair with its reason as output.	110
10.4	The evidence and decision stages in the abstract: stage A reads the ev- idence per candidate and per attribute, and stage B ranks from what it wrote.	111
10.5	The same two stages at work: one rendered evidence block, the note the reader writes from it, and the ranking the decision agent produces. Every figure in the note appears in the block; nothing is recomputed or invented.	112
11.1	A lost case. Left: three gallery crops of the correct person. Right: the CLIP ordering and the two funnel rounds, ranks 1 to 5 and 6 to 10. . . .	124
11.2	The same query after re-ranking with the larger model at stages 2B and 4B only. Stages 0, 1 and 2A were replayed unchanged.	125
11.3	A rescued case: outside CLIP’s top 10, first after reasoning.	126

List of Tables

2.1	Comparison of systems for searching people in video	11
3.1	The three forms of describing the person and how the project handles them	17
4.1	List of use cases	40
4.2	Use case specification UC-01: Log in	41
4.3	Use case specification UC-03: Manage cameras	42
4.4	Use case specification UC-04: Manage AI processing of cameras	43
4.5	Use case specification UC-05: Configure AI models	44
4.6	Use case specification UC-09: Search for people with AI	44
4.7	Use case specification UC-10: View and assess search results	45
4.8	Use case specification UC-11: Manage case files	46
4.9	Use case specification UC-13: View case files	46
4.10	Permission matrix by role	48
5.1	Tables of the relational database	59
5.2	Schema of the Milvus collection of feature vectors	62
6.1	Summary of technology and model choices	72
6.2	Comparison of YOLO11 sizes on the COCO validation set [22]	74
7.1	Number of tracks in the demonstration data per camera (first 120 seconds of each video)	85
8.1	Test results by level	87
8.2	Search quality results (6 queries)	88
8.3	Comparison of the two sampling intervals on a 10-second video (confi- dence threshold 0.1)	90

8.4	Search latency through the API (30 calls per form, $k = 8$)	91
9.1	The three places the pipeline can be improved, what each has to do, and what that costs.	97
9.2	One model per slot. None of them is fine-tuned on this dataset.	100
10.1	The attribute schema, and the visual band each attribute is judged from. The region column is a manual routing table: a wrong entry is invisible at run time, because the evidence line still reads correctly but quotes the sharpness of the wrong part of the body.	104
10.2	VLM noise on one track. A majority vote would report <code>hair_color=black</code> on a 40/30/30 split, and would break the 40/40 tie on <code>upper_inner_color</code> by whichever value the sort visited first.	106
10.3	What the CLIP score separates <i>inside</i> the 75-candidate pool. The AUC is the probability that a ground-truth crop outranks a non-ground-truth crop there; 1.0 would mean the score alone sorts the pool perfectly, and 0.5 would be chance. It is computed with ties taken into account, because many crops share the same similarity to three decimals and counting a tie as a win would overstate the score exactly where it is weakest. . . .	115
11.1	Per-call token budget. Output excludes thinking tokens, which are billed as output and counted against the same rate window, so the real output load is higher than this column. The reliable signal for a truncated reply is the API's <code>finish_reason</code> , not the reported output count.	119
11.2	100 identities out of the 496-identity subset (Section 11.1.1), pool of 75 candidates, zero-shot, one descriptive phrasing each. At $n = 100$ a percentage point is one identity.	120
11.3	Lost: inside CLIP's top 10, not in the funnel's. Rescued: outside CLIP's top 10, inside the funnel's. No pool: the ground truth was not among the 75 retrieved crops at all, so no reranking could have returned it.	121
11.4	The five identities CLIP had in its top 10 and the funnel did not return. 'Stage 2' is the coarse round's rank out of the candidates it ranked; 'n/a' in stage 4 means the refine round never saw that candidate.	122

11.5	The five lost identities re-ranked by a larger model at stages 2B and 4B only. ‘Stage 2’ and ‘Stage 4’ read <i>before</i> $\rightarrow$ <i>after</i>	123
11.6	What each mechanism changed, and how strong the evidence is.	128

List of Abbreviations

Abbreviation	Meaning
ALBEF	ALign BEfore Fuse (an image-text representation learning model)
ANN	Approximate Nearest Neighbor
API	Application Programming Interface
AUC	Area Under the Curve
CLIP	Contrastive Language-Image Pre-training
COCO	Common Objects in Context (an object detection dataset)
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSRF	Cross-Site Request Forgery
E2E	End-to-End (testing)
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IoU	Intersection over Union
IP	Internet Protocol
JPEG	Joint Photographic Experts Group (an image compression format)
JSON	JavaScript Object Notation
LLM	Large Language Model
mAP	mean Average Precision
MOT	Multi-Object Tracking
MRR	Mean Reciprocal Rank
NMS	Non-Maximum Suppression
RAM	Random Access Memory
RaSa	Relation and Sensitivity Aware representation learning
ReID	Person Re-identification
REST	Representational State Transfer
RTP	Real-time Transport Protocol
RTSP	Real Time Streaming Protocol

Abbreviation	Meaning
SHA	Secure Hash Algorithm
SORT	Simple Online and Realtime Tracking
TBPS	Text-Based Person Search
TCP	Transmission Control Protocol
UC	Use case
UDP	User Datagram Protocol
ViT	Vision Transformer
VLM	Vision-Language Model
YOLO	You Only Look Once

Chapter 1

Introduction

This chapter presents the context of the project and the reasons for choosing it, the objectives it pursues, its scope and its limitations.

1.1 Motivation

Surveillance cameras are being deployed more and more widely in Vietnam. According to figures from the General Department of Customs cited by the Ministry of Information and Communications, more than 16 million surveillance cameras were imported into Vietnam over the last five years, about 3.2 million a year on average, and the country is estimated to have more than 20 million surveillance cameras in use by 2025 [1]. The large number of cameras and their continuous recording produce a very large volume of video.

The value of a camera system does not lie only in deterrence. A review of forty years of research found that surveillance cameras reduce crime by a modest amount [2], but their more prominent role is to provide evidence for tracing events after they happen. A study of 251,195 incidents recorded by the British Transport Police found that camera footage was useful to the investigation in about 29% of cases, and that when useful footage was available, the share of cases solved rose from about 20% to about 50% [3]. The practical value of camera data therefore depends heavily on whether the right footage can be found.

In practice, many organisations run cameras that record continuously and need to find again the images, times and places at which a specific person appeared. Reviewing every video of every camera by hand is slow and error-prone. When the cameras span several

areas, searching is usually assigned by area, and the results must be recorded in case files for managers to follow, which adds requirements for access control over the data and for managing the results.

Moreover, the initial information about the person being sought is often incomplete and comes in different forms: a reference image, a verbal description, or a few appearance features such as gender, the type and colour of clothing, or carried items. A useful search system should therefore accept these multimodal descriptions and bring them into a single matching mechanism against the data analysed from the cameras, rather than support only one form of query.

These practical needs motivate the project “Developing a Person Search System from Multimodal Descriptions”.

1.2 Objectives

The overall objective of the project is to build an application that helps users search for people in the data collected by a surveillance camera system, letting them describe the person in several ways instead of reviewing video by hand. The application plays a supporting role: the system proposes and ranks matching results, while the final decision on whether a result is the person being sought remains with the user.

To reach this overall objective, the project sets the following specific objectives:

1. **Turn camera data into searchable data.** The application automatically analyses the images from the cameras, detects and follows the people appearing in the video, and stores each appearance of a person as a unit that can be searched.
2. **Search for people with multimodal descriptions.** Users can describe the person with a reference image, a sentence or a set of predefined appearance features; all three forms are matched against the analysed data and the results are ranked by relevance.
3. **Support the assessment and management of search results.** Users can review each result in its original context to judge it themselves, and save the confirmed results into case files for later follow-up and reference.

4. **Operate safely with many users.** The application grants permissions according to the responsibility of each group of users, limits the data each person may search to the surveillance areas assigned to them, and provides the functions needed to administer and monitor the system.
5. **Assess the applicability of existing artificial intelligence models.** The project does not train new models; it selects published models suited to the problem and to commodity hardware, then evaluates the search quality and processing cost when they are integrated into the application.
6. **Study how to improve search from text descriptions.** Separately from the application, the project investigates a reasoning layer that reads human-readable attribute evidence about each candidate and re-ranks the results of text-based search, and measures what it gains and what it costs against a conventional retrieval baseline.

The project is considered to have met its objectives when the application runs the whole workflow, from receiving camera data, searching and assessing results to saving case files, with measurements of search quality and performance presented in Chapter 8, and when the reasoning layer of objective 6 has been evaluated as reported in Chapter 11.

1.3 Scope

The scope of the project is defined as follows.

Search target. The system only searches for people. Search is based on overall appearance, such as clothing, colours and carried items, not on faces or other biometric features.

Camera data. The system receives camera images as video streams over the Real Time Streaming Protocol (RTSP), the common protocol of surveillance cameras. Within the project, the camera system is simulated by streaming the videos of a public dataset, made of seven static cameras observing the same place, as RTSP streams; each stream acts as one camera and is assigned to a surveillance area to demonstrate area-based access

control. Because the same protocol is used, the simulated streams can in principle be replaced by real cameras. The system processes the streams one at a time, not concurrently, and stores the analysis results for later retrieval. Since the target machine only has a central processing unit, it cannot analyse high-resolution video in real time; the data used for the demonstration is therefore indexed in advance from the video files of the dataset themselves, through the same processing pipeline as the RTSP streams.

Forms of description. Users describe the person with one of three forms:

- a pre-cropped reference image containing one person;
- a text description in English;
- a set of predefined appearance attributes: gender, type and colour of the upper garment, type and colour of the lower garment, and carried items such as a backpack or a handbag.

The system does not support queries in Vietnamese and does not translate descriptions automatically.

Search results. Each result corresponds to one appearance of a person on one camera, ranked by how well it matches the description. Users choose how many results to see and can restrict the search to certain cameras and a time range. The relevance score is only used to rank and to support assessment within the current search; the system draws no conclusion about identity and does not keep a history of searches. Only results that the user deliberately saves are written to a case file.

Users. The application serves several groups of users with different responsibilities and permissions; the concrete roles are defined in the system analysis of Chapter 4. Creating and managing the list of surveillance areas is outside the scope; the list is declared at deployment.

Deployment environment. The system is deployed as a web application on a personal computer without a discrete graphics card, for demonstration within the capstone project and not for commercial use.

Research on the search block. The research of Part II is a separate study of the search step for text descriptions and is not integrated into the application. It works on a public person re-identification dataset whose images are already cropped and grouped by identity, uses its own choice of models, and evaluates a re-ranking layer offline on a sample of identities. Its results show what such a layer could contribute to the search step; they are not a measurement of the deployed application.

Out of scope. The project does not cover:

- face recognition or determining the identity of the person in an image;
- searching for vehicles or objects other than people;
- training or fine-tuning artificial intelligence models;
- linking the appearances of the same person across different cameras in the application;
- processing many camera streams concurrently and raising real-time alerts.

1.4 Limitations

Besides the defined scope, the results of the project have the following limitations; their extent is quantified in Chapters 8 and 11 and summarised in Chapter 12.

- **Search quality depends on existing models.** The models were trained on data from contexts different from the project's camera data and are not fine-tuned. Search by text and by attributes is affected most, so search by image is the most reliable form.
- **Narrow test data.** The system was only tested on seven static cameras observing one outdoor place in daylight; harder conditions such as low light or low resolution have not been verified.
- **Not all cameras are analysed continuously.** Running on a machine without a discrete graphics card, the system analyses video more slowly than it is recorded and processes the cameras in turn, so the searchable data of each camera has gaps.

- **Appearances may be missed or split.** The system analyses only part of the frames and represents each appearance by one frame, so a person who appears very briefly may be missed, and an occluded person may be split into several results or ranked low.
- **Limited descriptions.** Text descriptions are only supported in English; the attribute set only covers gender, clothing and carried items, with no negative options.
- **Relative scores and small deployment.** The relevance score only ranks results within one search; the system is deployed on a single computer at demonstration scale.
- **The reasoning layer is evaluated offline and on a small sample.** The research of Part II was measured on 100 identities of a public dataset with a hosted language model, and depends on that model's quotas and latency; it has not been run inside the application.

Part I

The Person Search Application

Chapter 2

Related Systems

This chapter surveys several commercial systems for searching people in video, compares them on criteria relevant to the project and derives the development directions of the system.

2.1 Existing Systems on the Market

This section surveys three commercial products that represent three approaches to searching for people in camera data: comprehensive video analytics (BriefCam), appearance search (Avigilon) and natural-language search on a cloud platform (Verkada). Academic work is presented in Chapter 3.

2.1.1 BriefCam

BriefCam is video content analytics software based on deep learning, acquired by Canon in 2018 [4] and now distributed as a Milestone Systems product [5]. Its functions are split into three modules: *Review* supports investigation on recorded video, filtering people and vehicles by attributes such as colour, size, speed and direction, finding objects that look similar to a sample object, and condensing hours of video into a short clip (Video Synopsis); *Respond* raises real-time rule-based alerts; and *Research* provides statistics, heat maps and people counting [5]. Figure 2.1 shows the search interface of the product. BriefCam can be installed on premises or in the cloud, but the processing server must have an NVIDIA graphics card [6].

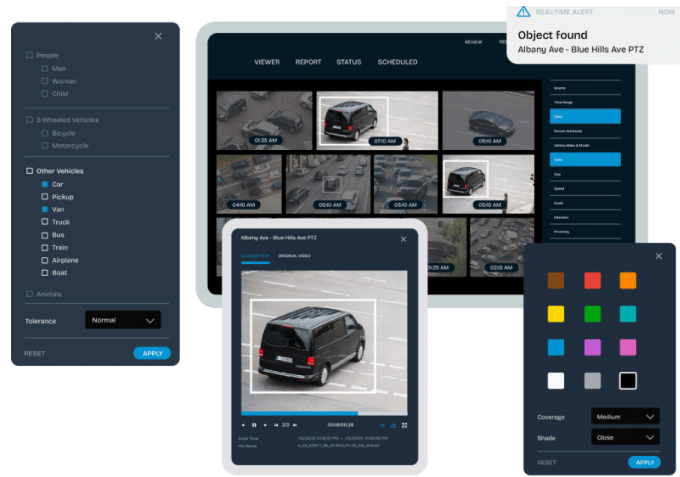


Figure 2.1: The search and filtering interface of BriefCam [5]

Relation to the project. BriefCam supports search by attributes and by an appearance sample, where the sample is an object selected in recorded video rather than an image uploaded from outside as in this project; the published documentation does not mention search by a natural-language description.

2.1.2 Avigilon Appearance Search

Avigilon Appearance Search is the deep-learning video search technology of Avigilon, part of Motorola Solutions, used to find a person or vehicle across hours of video from many cameras [7]. A search can start by choosing appearance features from a predefined list such as clothing colour, gender and age group; by uploading an image; or by selecting an object in the video as a sample [7]. The results help trace the route of the subject and can be bookmarked and exported as an evidence package describing the event. Figure 2.2 shows the interface with a person marked. The technology is built into the Avigilon Control Center video management software and runs on the vendor’s cameras and recorders, or on the Avigilon AI Appliance for third-party ONVIF cameras.



Figure 2.2: Avigilon Control Center with a marked subject [8]

Relation to the project. Avigilon supports two of the project’s forms of description, attribute selection and a reference image, and lets results be compiled into evidence, similar to the project’s case files. The documentation does not mention free-text search, and the system is tied to the vendor’s software and devices.

2.1.3 Verkada AI-Powered Search

Verkada AI-Powered Search, introduced in 2024 on the Command cloud security platform, lets users search for people and vehicles in everyday language, for example “person wearing helmet”, instead of choosing from a list of attributes [9], and search with an uploaded image of a person, vehicle or object [10]. According to the vendor’s technical article [11], cameras produce crops of objects; in the cloud, the crops are encoded into vectors with the CLIP model, which maps images and text into one representation space, and stored in a vector database; the query sentence is encoded the same way to find the nearest crops. Figure 2.3 shows the results of a natural-language query. The system works best with Verkada cameras; third-party cameras connect through a Command Connector device with higher latency [12].

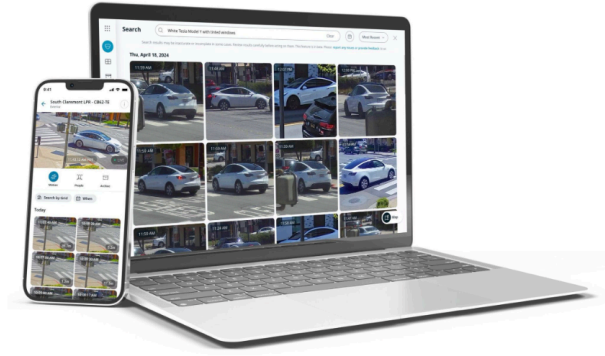


Figure 2.3: Results of a natural-language query on the Verkada Command platform [9]

Relation to the project. Verkada is technically closest to the project: both support search by a description and by an image, based on a shared representation space and a vector database. The difference is the deployment model: Verkada processes on the vendor’s cloud infrastructure, whereas the project is deployed on premises on a commodity computer.

2.2 Comparison and Discussion

Table 2.1 compares the three surveyed systems with the project’s system, based on the vendors’ published documentation (Section 2.1); what the documentation does not state is marked “not mentioned”.

Table 2.1: Comparison of systems for searching people in video

Criterion	BriefCam	Avigilon Appearance Search	Verkada AI-Powered Search	The project’s system
Search by attributes	Yes: object type, colour, size, speed, direction	Yes: clothing colour, gender, age group; vehicle type and colour	Yes, predefined attribute filters, extended by natural-language search	Yes: gender, type and colour of upper and lower garments, carried items; converted into a description sentence

(continued from Table 2.1)

Criterion	BriefCam	Avigilon Appearance Search	Verkada AI-Powered Search	The project's system
Search by image or appearance sample	Yes: select a sample object in the video to find similar appearances	Yes: upload an image or select a sample in the video	Yes: upload an image of a person, vehicle or object	Yes: upload a cropped image containing one person
Free-text search	Not mentioned	Not mentioned	Yes	Yes (English only)
Published matching mechanism	Metadata search and appearance similarity	Deep learning on appearance features, combined with facial features	Crops and query sentences encoded into one representation space (CLIP), searched in a vector database	Images, sentences and attributes encoded into one representation space, searched in a vector database
Other functions	Video Synopsis, face recognition, licence plates, real-time alerts, statistics and heat maps	Bookmarking and exporting results as evidence, multi-site search	Alerts created from a query, centralised cloud management	Case files, access control by role and surveillance area, overview dashboard of case files
Deployment model	On premises or on a cloud server	On premises, integrated with the vendor's video management software	The vendor's cloud platform	On premises on one personal computer
Hardware and device requirements	Processing server must have an NVIDIA graphics card	The vendor's cameras and recorders, or an AI Appliance for third-party ONVIF cameras	The vendor's cameras, or a Command Connector for third-party cameras	No discrete graphics card; receives streams from any RTSP camera
Distribution	Commercial software	Commercial software and devices	Commercial service platform	Self-developed application using published models

The comparison leads to the following observations:

- **Supporting several forms of description has become the norm.** All three systems support search by attributes and by an image or appearance sample, but within

the surveyed documentation only Verkada supports free-text search, a function that appeared in 2024. Verkada's approach, encoding images and query sentences into one representation space and searching a vector database, lets one mechanism serve both forms.

- **Search usually comes with tools for compiling results**, such as Avigilon's bookmarking and evidence export: users also need to store and share results during an investigation.
- **Commercial systems are tied to the vendor's infrastructure**: BriefCam needs a server with a dedicated graphics card, Avigilon needs the vendor's software and devices, and Verkada runs on the vendor's cloud. Cost and vendor lock-in are barriers for small organisations or those that must keep data on premises. These products also bundle many functions beyond search, such as alerts, face recognition, licence plates and statistics.

2.3 Development Directions

Based on the observations in Section 2.2, the project sets the following development directions:

1. **Support all three forms of description in one search mechanism**: a reference image, a free-text description in English, and a set of predefined attributes. As in Verkada's approach, all three are mapped into one representation space; the attribute set is turned into a structured description sentence, so no separate model is needed for each attribute.
2. **Organise results by appearance**, one appearance of a person on one camera, ranked by relevance and always shown with the original frame so that users can judge them; the system draws no conclusion about identity (Section 1.2).
3. **Manage results as case files**, similar to Avigilon's evidence compilation, so that the responsible people can review them without searching again.

4. **Control access by role and surveillance area**, separating the responsibilities of technical administration, searching and following results; searchers only access the data of their assigned area (Section 1.1).
5. **Deploy on premises, independent of hardware vendors**: a web application that receives images over the common RTSP protocol and processes and stores all data on a computer without a discrete graphics card.

Chapter 3

Theoretical Background

This chapter presents the background of the application: person search and cross-modal retrieval, techniques for detecting and tracking people, frame sampling, joint image-text representation learning, vector search and the protocol used to stream camera video.

3.1 Person Search and Cross-Modal Retrieval

3.1.1 Problem Statement

Given a set of person images extracted from the video of many cameras, called the *gallery* $\mathcal{G} = \{g_1, g_2, \dots, g_n\}$, and a *query* q describing the person being sought, the system must rank the elements of $\mathcal{G}$ by how well they match q and return the k best matches. The common approach is to learn a representation function that maps the query and every element into a vector space, then measure relevance with a similarity function $s(q, g_i)$; search then becomes finding the nearest vectors.

In surveillance data, the people to be indexed are not available as tightly cropped images as in benchmark datasets; they appear in full frames together with many other people and objects, so the complete problem also includes a person detection step. Xiao et al. learn detection and identification jointly in one model [13]; another approach uses a separate detector and a separate identification model [14]. The project follows the second approach so that each component can be replaced independently.

3.1.2 Three Forms of Query

Search by image. When the query is an image containing the person, the problem corresponds to *person re-identification* (ReID): finding the images of the same person in data collected from several cameras or at several times [15]. The main challenges are changes in viewpoint, lighting and pose, occlusion and low resolution: the same person can look very different between two cameras, while two people dressed alike can look very similar.

Search by text. When the user only has a verbal description, for example “a man wearing a black jacket and blue jeans, carrying a backpack”, the problem is text-based person search (TBPS), proposed by Li et al. together with the CUHK-PEDES dataset of person images with English descriptions [16]. It is a cross-modal problem: the model must bring text and images into one representation space, while descriptions are often vague and the differences between people often lie in small details.

Search by attributes. The user chooses appearance attributes from a predefined list such as gender, garment type and colour, and carried items. There are two ways to implement it: use a pedestrian attribute recognition model [17] and filter by label, which needs labelled data for every attribute; or combine the attributes into a description sentence from a fixed template and handle it as a text query. The project uses the second way so that it shares the model of text search; the trade-off is that attributes only affect the ranking and are not hard filters. Negative options such as “no backpack” are not supported, because CLIP-style vision-language models have considerable difficulty with negation, in many cases performing at chance level [18].

Appearance search differs from face recognition: it does not rely on biometric features, which need a clearly visible face, but on clothing, colours and carried items, which are easy to observe from cameras mounted high and far away but can change and be shared by many people. The results are therefore suggestions for the user to assess, not conclusions about identity.

3.1.3 Approach of the Project

Table 3.1 summarises the three forms of query and how the project handles them.

Table 3.1: The three forms of describing the person and how the project handles them

Form	Input	Strengths and challenges	Handling in the project
Reference image (ReID)	A cropped image containing one person	Rich visual information; sensitive to viewpoint, lighting and occlusion	Encoded with the image encoder
Text description (TBPS)	An English sentence	No image needed; descriptions may be vague or incomplete	Encoded with the text encoder
Attributes	Gender, type and colour of the upper and lower garments, carried items	Easy to enter and unambiguous; only expresses the available attributes	Turned into a structured English sentence, then handled as a text description

- **One shared representation space.** The RaSa model [19] encodes both images and text into one vector space, so person images from video, query images, description sentences and sentences generated from attributes can all be compared directly (Section 3.5).
- **The result unit is an appearance.** People are detected and tracked across frames (Sections 3.2 and 3.3); each continuous appearance of a person on one camera (a track) is represented by one representative frame and one vector, which reduces the number of elements to match and avoids duplicate results.
- **Two separate stages.** The indexing stage runs in the background, processes camera streams and stores features; the query stage only encodes the user’s description and finds the nearest vectors (Section 3.6), without processing images again.
- **A human makes the final decision.** The system only ranks; users assess each result on the original image before saving it to a case file.

3.2 Person Detection

3.2.1 Object Detection

Object detection locates the objects in an image and determines their class. For each object, the model returns a rectangular *bounding box*, usually given by the coordinates of two corners (x_1, y_1, x_2, y_2) , the *class* of the object and a *confidence score*. Two boxes are compared with the Intersection over Union (IoU):

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (3.1)$$

which ranges from 0 when the boxes do not overlap to 1 when they coincide. IoU is used to remove duplicate predictions and to link detections across frames in tracking (Section 3.3).

3.2.2 Two-Stage and One-Stage Detectors

A *two-stage* model such as Faster R-CNN [20] first uses a region proposal network to generate regions likely to contain objects, then classifies each region and refines its box; this is accurate but costly because many proposals must be processed. A *one-stage* model such as YOLO (You Only Look Once) [21] treats detection as a single regression problem, predicting the boxes and classes of all objects in one pass over the image, which is considerably faster and suits processing many video frames.

3.2.3 The YOLO Family

Modern YOLO models have three parts: a feature extractor (backbone) at several levels of detail, a feature aggregator (neck) at several scales so that both large objects close to the camera and small distant ones can be detected, and a prediction head for boxes, classes and confidence. YOLO11 was released by Ultralytics in 2024 in five sizes (n, s, m, l, x) that trade speed for accuracy [22]. The models are pre-trained on COCO [23], which has 80 common object classes including *person*, so they can detect people directly without further training.

3.2.4 Post-Processing: Confidence Threshold and Duplicate Removal

The raw output of the model contains many low-confidence boxes and many overlapping boxes describing the same object. Two post-processing steps are applied: boxes below a confidence threshold are discarded, and Non-Maximum Suppression (NMS) repeatedly keeps the most confident box and removes every box of the same class whose IoU with it exceeds a threshold. A lower confidence threshold misses fewer people but produces more false detections; a lower IoU threshold removes overlapping boxes more aggressively but may remove people standing close together.

3.2.5 Role of Detection in the System

The detector is applied to the sampled frames (Section 3.4) and keeps only the *person* class, with a confidence threshold of 0.1, an NMS IoU threshold of 0.7 and at most 300 detections per frame. The low confidence threshold is deliberate: the ByteTrack tracker in the next step needs low-confidence detections, often partially occluded people, to keep existing tracks alive (Section 3.3). These detections are not used to start new tracks, and a track is only confirmed when the object is detected in at least two processed frames, so most transient false detections are removed by the tracking step. The reasons for choosing YOLO11n are given in Section 6.1.4.

3.3 Multi-Object Tracking

3.3.1 Problem and Basic Components

A detector processes each frame independently, so it cannot tell whether a person in this frame is the one who appeared in the previous frame. Multi-Object Tracking (MOT) assigns each object an identifier and keeps it across frames; the sequence of bounding boxes of the same object over time is called a *track*. In the project, tracking turns separate detections into *appearances*, the unit of search results (Section 3.1.3).

The common approach is *tracking-by-detection*, founded on the SORT algorithm [24], which has three components. First, each track has a Kalman filter [25] that predicts the position and size of its box in the current frame under a near-constant-velocity assump-

tion, then corrects the prediction with the actual detection. Second, the match between a track and a detection is measured by the IoU (Equation 3.1) between the predicted box and the detected box. Third, assigning each detection to at most one track at minimum total cost is solved with the Hungarian algorithm [26]; an unmatched detection may start a new track, and an unmatched track is considered lost.

3.3.2 ByteTrack

Earlier methods usually discarded low-confidence detections immediately, while a partially occluded person is often detected only with low confidence, so tracks broke easily. ByteTrack [27] uses every detection through two association steps: high-confidence detections are first matched with all tracks; the remaining tracks are then matched with the low-confidence detections, so that occluded people keep their tracks, and the low-confidence detections left unmatched are discarded as mostly background or errors. Only high-confidence detections start new tracks, and a lost track is kept for a short time so that it can be resumed.

3.3.3 BoT-SORT

BoT-SORT [28] extends ByteTrack by adjusting the Kalman filter state, compensating camera motion and adding appearance re-identification (ReID) features to the association step. In the project, BoT-SORT is an alternative tracker with the same thresholds as ByteTrack; camera motion compensation and ReID are disabled because the cameras are static and to keep the cost low on a CPU-only machine.

3.3.4 Track Life Cycle in the System

The tracker receives the detections of the sampled frames, separately for each camera, with the following rules:

- **Confidence thresholds:** detections of 0.25 or more are high and may start a new track; detections from 0.1 to below 0.25 are only used in the second association step.

- **Confirmation:** a track is confirmed only after at least two observations; an unconfirmed track produces no result.
- **Termination:** a track ends when it has been lost for more than three consecutive processed frames or more than two seconds since its last observation, and when the stream ends. For video files, the timestamp is the one recorded in the file, so the rule does not depend on the speed of the machine; for RTSP streams, the timestamp is the time the system receives the frame, so when processing is slower than the stream, the two-second condition may end tracks early.
- **Identifier scope:** one camera and one processing session only; the system does not link the tracks of the same person across cameras.

3.4 Frame Sampling

Surveillance video usually has a few dozen frames per second; the videos of the project’s dataset have about 60 frames per second, so two consecutive frames are only about 17 milliseconds apart and almost identical. Running the models on every frame is therefore very costly and brings little new information. The project samples frames uniformly by index: with frame index i and sampling interval N , a frame is selected if and only if

$$i \bmod N = 0, \tag{3.2}$$

and it keeps its original index and timestamp, so that the time at which a person appears follows the camera’s own timeline. With a source of f frames per second, two processed frames are N/f seconds apart: about 0.17 seconds with $N = 10$ and 0.33 seconds with $N = 20$.

The sampling interval is a trade-off. Sampling comes before the detector, so the number of AI model runs drops by about N times; decoding, however, must still be done on every frame, because most H.264 frames can only be decoded after the frames before them (Section 3.7.1). Conversely, the larger N is, the more easily a person who appears only briefly is missed, since a track needs at least two observations, and the further people move between two observations, which makes IoU association harder and tracks break more easily. The system provides two presets: *baseline* with $N = 10$, which favours

track quality, and *throughput* with $N = 20$, which favours speed and was chosen as the default based on measurements on the target machine (Section 8.3). Each job records the value of N it used so that results can be compared and reproduced.

3.5 Joint Image-Text Representation Learning

3.5.1 Joint Embedding Space and Contrastive Learning

To compare a description with a person image, the system uses two encoders: an image encoder turns an image into a feature vector, and a text encoder turns a sentence into a vector of the same dimension. The two encoders are trained so that the vector of an image and the vector of the sentence describing that image are close, while non-matching pairs are far apart; a space with this property is called a *joint embedding space*. Closeness is measured with the cosine similarity

$$s(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}, \quad (3.3)$$

which becomes the inner product $\mathbf{u}^\top \mathbf{v}$ when the vectors are normalised to unit length (L2 normalisation). An important advantage is that the gallery images only need to be encoded once, at indexing time; at query time, the system only encodes the query and compares it with the stored vectors (Section 3.1.3).

The common training method is *contrastive learning*: in each batch of B image-text pairs, the correct sentence of an image is the positive sample, the other $B - 1$ sentences are negatives, and the loss

$$\mathcal{L}_i = -\log \frac{\exp(s(\mathbf{v}_i, \mathbf{t}_i)/\tau)}{\sum_{j=1}^B \exp(s(\mathbf{v}_i, \mathbf{t}_j)/\tau)} \quad (3.4)$$

pushes the similarity to the positive well above the negatives, with τ a temperature parameter; the text-to-image direction is computed the same way. CLIP [29] applies this method to about 400 million image-text pairs from the Internet. ALBEF [30] first aligns the representations of the two modalities with contrastive learning, then fuses them in a multimodal encoder using cross-attention to match individual words with image regions in detail.

3.5.2 The RaSa Model

General models such as CLIP and ALBEF struggle to separate small differences between people who look alike, for example the same jacket with trousers of a different colour. RaSa (Relation and Sensitivity Aware representation learning) [19] is built on ALBEF and designed specifically for text-based person search, with a 12-layer transformer image encoder, a 6-layer text encoder and a 6-layer multimodal encoder. Its three key points are:

- **Intra- and inter-modal contrastive learning.** Besides pairing images with sentences, images of the same person are also pulled together. This makes the vectors of two images of one person close, which is the basis for using the same vector space for image queries in the project.
- **Relation-aware learning.** Each description is written for *one* specific image and may not be fully true of the other images of that person; RaSa separates these two kinds of positive pairs so as not to learn details that are not true.
- **Sensitivity-aware learning.** The model is trained to notice when an important word is replaced, such as “red shirt” becoming “blue shirt”, which changes the meaning of the sentence.

At query time, RaSa can re-rank the top 128 results with the multimodal encoder to improve accuracy, but this is far more costly than comparing vectors alone. On the CUHK-PEDES test set, RaSa reaches a Rank-1 of 76.51% and an mAP of 69.38% [19].

3.5.3 Use of RaSa in the System

The system uses RaSa weights pre-trained on CUHK-PEDES, without retraining. The image encoder encodes the person image cropped from the representative frame at indexing time and the query image when searching by image, with input images resized to 384×384 pixels; the text encoder encodes description sentences and sentences generated from attributes. Vectors have 256 dimensions and are L2-normalised, so all three kinds of query are compared with the same set of vectors by inner product, and each track needs only one vector. Re-ranking is not enabled, because it only applies to text queries,

is costly on a CPU-only machine, and no measurement yet shows that it improves results on the project’s data.

Two points should be noted. First, RaSa was designed and evaluated for text-based search; using its image encoder for image-to-image queries is the project’s choice based on its intra-modal contrastive learning, not a published result. Second, CUHK-PEDES was built from the images of five existing person re-identification datasets [16], each image being a tightly cropped person with a sentence written for that image; the person images in the project are cropped automatically from full frames of a crowded area, with a different context and viewpoint, and are often partially occluded by the people around them. This domain gap directly affects the quality of text and attribute queries (Section 8.2).

3.6 Vector Search

3.6.1 Nearest Neighbour Search and the HNSW Index

When each appearance is represented by one vector (Section 3.5) and the query is also encoded as a vector $\mathbf{q}$, search becomes a *k-nearest-neighbour* problem: among n stored vectors, find the k with the largest inner product with $\mathbf{q}$, equivalent to cosine similarity for normalised vectors. Exhaustive search is always exact but its cost grows linearly with n , so the response time grows as data accumulates. *Approximate Nearest Neighbor* (ANN) search builds an index in advance so that a query only examines a small part of the data, accepting that a small fraction of the true neighbours may be missed in exchange for speed.

HNSW (Hierarchical Navigable Small World) [31] is a graph-based ANN method: each vector is a node linked to some vectors near it, and search repeatedly moves to a neighbouring node closer to the query. The nodes are organised in layers; the sparse upper layers move quickly to the region near the query, and the bottom layer, which contains every vector, searches that region precisely, so the search complexity grows logarithmically with the number of elements [31]. The index has three main parameters [32]: the maximum number of edges per node M and the number of candidates during construction *efConstruction*, which give a better graph when larger but cost more

memory and construction time; and the number of candidates during search ef , which is more accurate but slower when larger and should not be smaller than the number of results requested.

3.6.2 Vector Search with Filters

Queries in a surveillance system usually come with conditions, such as only within the user's area or within a time range. With **post-filtering**, the system takes the k nearest vectors over all data and only then removes the results that fail the condition, so it may return fewer than k results, or none, when the user's area is a small part of the data. With **pre-filtering**, the system determines the set of vectors that satisfy the condition and searches only within it, so every result is within the permitted scope and there are fewer than k results only when the scope really holds little data. Milvus [33], an open-source vector database, stores scalar fields next to the vectors and performs search with pre-filtering [34].

3.6.3 Vector Search in the System

The system uses Milvus with an HNSW index, the inner-product metric, $M = 16$, $ef\text{-Construction} = 128$ and $ef = 64$. A search sends Milvus the query vector together with a filter made of the user's area, the permitted cameras, the time range if any and the ready status, and receives at most k results with $k \in \{4, 8, 12, 16\}$. The similarity is shown as a relevance score and is computed only at query time. Each result is checked against PostgreSQL to fetch its details and re-check the permission scope; results that are no longer valid because the two stores are temporarily out of sync are removed, and if fewer than k results remain, the system searches again with twice the number, up to three rounds, so the number requested never exceeds 64, the value of ef . With the project's demonstration data (1,523 tracks, Section 7.2), exhaustive search would still be fast enough; the HNSW index was chosen to keep the response time as data accumulates from many cameras over a long period.

3.7 RTSP and Camera Stream Simulation

3.7.1 H.264 Compressed Video

IP cameras compress video before transmitting it, because an uncompressed 1920×1080 frame takes about 6 MB, or about 370 MB per second at 60 frames per second. H.264 [35] is the common compression standard of surveillance cameras and also the video format of the project. An *I-frame* only uses prediction within the frame itself, so it can be decoded independently; *P-frames* and *B-frames* only store motion and differences relative to reference frames, so they can only be decoded once their references are available. Decoding must therefore be done on every frame even though the system only analyses some of them (Section 3.4), and a decoder that joins a stream midway must wait for the next I-frame.

3.7.2 RTSP and RTP

RTSP (Real Time Streaming Protocol) [36] is a *control* protocol: the receiver identifies a stream by an address of the form `rtsp://<host>:<port>/<path>` and exchanges the DESCRIBE, SETUP, PLAY and TEARDOWN commands to obtain the description, set up, start and end the session. The data is carried by RTP (Real-time Transport Protocol) [37]: compressed video is split into packets with sequence numbers and timestamps; for H.264 the timestamps use a 90 kHz clock [38]. RTP packets can be sent over UDP, with low latency but possible packet loss, or interleaved on the TCP connection of the RTSP session, without loss but possibly late under congestion. RTSP 2.0 [39] removed the commands used to push a stream to a server, so the project's simulation tools use version 1.0.

3.7.3 Simulating Cameras with a Streaming Server

So that the system is tested with the actual protocol of IP cameras, each video file is re-played as an RTSP stream that acts as one camera. The MediaMTX streaming server [40] accepts streams pushed in and serves streams out over RTSP, each at its own path such as `/cam1`. The FFmpeg tool [41] reads the file and pushes it to MediaMTX with three options: `-re` to send at the original frame rate, `-stream_loop -1` to loop forever like a

camera that never stops, and `-c` copy to copy the compressed data without re-encoding, so the stream carries exactly the H.264 data of the original file and costs almost no resources. A simulated stream resembles a real camera in using the same protocol and format and in *not waiting* for the receiver: if the receiver is slow, new data keeps being sent. The difference is that the content repeats periodically and there is almost no packet loss or disconnection as on a real camera network.

3.7.4 Receiving RTSP Streams in the System

The background worker reads streams with the PyAV library, which uses FFmpeg's decoding libraries, with RTP interleaved over TCP to avoid packet loss corrupting images. Connecting and waiting for data are both limited to 8 seconds; when a stream is interrupted, the system tries to reconnect up to three times with a waiting time growing from 0.25 to 2 seconds, then records the processing run as failed. Each frame is timestamped with the time the system receives it rather than with the stream's RTP timestamp (Section 3.3.4). Since processing is much slower than the stream, the system cannot analyse every camera continuously; how the processing time is shared among cameras is described in Section 5.2.1.

Chapter 4

System Analysis

This chapter analyses the requirements of the system: actors, functional and non-functional requirements, the use case model and the permission matrix.

4.1 Actors

The system has three actors, corresponding to three kinds of user account divided by responsibility: the **Administrator** (Admin), the **Operator** and the **Viewer**. All three share one login function; after authentication, the system determines the role of the account and only offers the functions and data suited to that role.

Before describing each actor, two business concepts used throughout the following chapters need to be defined:

- **Surveillance area:** a geographic scope such as a building or a public space. Each camera belongs to exactly one area, set when the camera is added and not changed afterwards. The list of areas is declared at deployment; the current version has no interface for adding, editing or deleting areas.
- **Case:** a record created by an Operator to keep the search results related to an incident, with a title, notes and a status of *Open* or *Closed*. A case belongs to the Operator in charge of it but is not tied directly to a surveillance area: the area only limits the scope of search, while the rights over a case follow its owner. Since an Operator's area can change, a case may contain results from several areas.

4.1.1 Administrator

The Administrator is responsible for the *technical administration* of the system, keeping the camera system and the AI processing running so that the other actors can use them. The Administrator's responsibilities are:

- **Managing accounts** of Operators and Viewers, including assigning a surveillance area to each Operator.
- **Managing cameras** and the connection to the RTSP stream of each camera.
- **Managing AI processing:** deciding which cameras are analysed, and choosing the person detector (Detector) and the tracking algorithm (Tracker) used by the whole system.
- **Monitoring operation:** following the status of cameras and AI processing, checking the AI components, and consulting the audit log.

Technical administration rights do *not* imply business rights: by default, the Administrator may not search for people and does not see cases. The Administrator account is created at deployment; through the interface, the Administrator can only create and assign the Operator and Viewer roles.

4.1.2 Operator

The Operator uses the main business function of the system directly: searching for people in the analysed camera data and saving the matching results into cases. The Operator's responsibilities are:

- **Searching for people** by image, English description or appearance attributes.
- **Assessing results:** viewing each result in the context of its frame and deciding which results really show the person being sought.
- **Managing cases:** saving matching results into cases and following each case until it is closed.

The Operator's scope is limited by two principles. First, the *search scope* is set by the surveillance area attached to the account: the system only searches the operational cameras of that area, and rejects any request containing a camera or area outside the Operator's rights. Second, the *case scope* is set by ownership: each case has exactly one Operator in charge, the one who created it; an Operator only acts on the cases he or she owns and cannot transfer ownership to someone else. The two scopes are independent: when moved to another area, an Operator still sees and manages his or her old cases, while new searches only run in the new area.

4.1.3 Viewer

The Viewer is at the management level of the organisation, such as a head of department or a director, who needs to follow the results of the work but does not search or configure the system. The Viewer's responsibilities are:

- **Viewing an overview** of the cases across the whole system.
- **Viewing cases:** every case and the results saved in it.

The Viewer only has read access and does not search for people. Unlike the Operator, the Viewer is not limited by area, because the Viewer's scope is the cases, which are not tied to areas.

4.1.4 Principles of Role Separation

The division above follows two principles: **separating technical rights from business rights**, so that the people who configure the system do not automatically access search data and the people who use the data cannot interfere with the configuration; and **least privilege**, so that each actor only has the rights needed for its work. The rights of each role for each group of actions are summarised in the permission matrix of Section 4.6.

The camera streams and the background AI processing are not actors: a camera is a data source configured by the Administrator, and the analysis of camera data is performed automatically by the system when AI processing is enabled, not triggered directly by a user.

4.2 Functional Requirements

The functional requirements are grouped by the actors of Section 4.1, together with a group of common functions and a group of functions the system performs by itself. Each requirement has a code for cross-reference with the use case specifications (Section 4.5) and the test results (Chapter 8).

4.2.1 Common Functions

- FR-C01** Users log in with a username and password and are taken to the interface of their role; locked or deactivated accounts are rejected.
- FR-C02** Users can log out. A session expires 12 hours after it is issued or after 30 minutes of inactivity, and becomes invalid immediately when the account is locked or deactivated. While the user is active, the session is refreshed with a new session.

4.2.2 Administrator Functions

Account management.

- FR-A01** Create, update, lock, unlock, delete or deactivate accounts, assigning the Operator or Viewer role; usernames are unique.
- FR-A02** Each Operator is assigned exactly one surveillance area, chosen from the predefined list.

Camera management.

- FR-A03** Add and update cameras with a code, name, area, RTSP address and credentials; the camera code is unique, and the area cannot be changed after creation.
- FR-A04** Check the RTSP connection when a camera is added, when its address changes and on request; if the connection fails, the camera is still saved with a warning so that it can be checked again later.

FR-A05 Retire a camera from operation or bring it back. The data of a retired camera is kept but cannot be searched or added to cases for the time being; results already saved in cases remain visible. A camera brought back into operation starts with AI processing disabled.

AI processing management.

FR-A06 Enable or disable AI processing for each camera; enabling is refused when the camera has lost its RTSP connection, has been retired or the models are not ready.

FR-A07 Upload a video file for a camera to be processed through the same pipeline as an RTSP stream; the function is used to prepare the demonstration data (Section 1.3) and for cameras without RTSP.

FR-A08 Choose the Detector and Tracker from a registered list, applied to the whole system; the configuration is only applied when the models are ready and the pair is compatible, otherwise the previous configuration is kept. The image and text encoders are fixed.

Operation monitoring.

FR-A09 Follow the RTSP connection and AI processing status of each camera, filter abnormal cameras and refresh the data.

FR-A10 Check the camera processing group, on a real frame from one camera, and the search group separately; results are shown per component, and a frame without people is reported as not yet verifiable rather than as an error.

FR-A11 View and filter the audit log by actor, event type and time. The log records logins, logouts, failed logins, administrative actions, case actions and important technical errors; it does not record searches.

4.2.3 Operator Functions

Searching for people.

- FR-O01** Search by image, provided by file selection, drag and drop, pasting, or dragging a result into the search box to search again for the same person.
- FR-O02** Search by an English description; Vietnamese is not supported and nothing is translated automatically.
- FR-O03** Search by attributes, namely gender, the type and colour of the upper and lower garments, and carried items, with no negative options; the attributes are combined into an English description and searched as in FR-O02.
- FR-O04** The search scope is automatically limited to the account's area and can be narrowed further by operational camera and time range; requests containing cameras or areas outside the Operator's rights are rejected.
- FR-O05** Choose 4, 8, 12 or 16 results; the system returns at most that many in decreasing order of relevance, without a score threshold that discards results.

Viewing and assessing results.

- FR-O06** Each result is one appearance of a person on one camera, shown as the person image with surrounding context, rank, camera, area, time and relevance score.
- FR-O07** Select a result to see the full frame with the person's bounding box and the details.
- FR-O08** Sort results by relevance or time, and filter them again by camera or time.

Case management.

- FR-O09** Create a new case from a result with a title and notes; the owner is always the logged-in account.
- FR-O10** Add a result to an open case the Operator owns; each save creates a new entry, even for the same appearance.
- FR-O11** Optionally close the case while saving; if that step fails, the result is still saved and the user is informed.

- FR-O12** Edit the title and notes and remove results from an open case.
- FR-O13** Close or reopen a case; the content of a closed case cannot be edited and results cannot be added or removed.
- FR-O14** View the cases the Operator owns, including after a change of area; results are shown as in FR-O06 and FR-O07 but without the relevance score.

4.2.4 Viewer Functions

- FR-V01** View a system-wide overview: the number of cases by status, the number of saved results and the list of recent cases; select a case to see its details.
- FR-V02** View every case, filtered by status, creation time or the Operator in charge.
- FR-V03** View the details of a case and its saved results in read-only mode.
- FR-V04** View the details of a saved result and the full frame with its bounding box; the relevance score is not shown.

4.2.5 Automatic System Functions

- FR-S01** Automatically analyse, in turn, the cameras that are operational, have AI processing enabled and have RTSP, so that every camera is analysed.
- FR-S02** For each RTSP stream or video file: sample frames, detect people, track them into appearances, choose one representative frame and create a feature vector.
- FR-S03** Store the representative frame, bounding box, vector and camera, area and time information of each appearance; an appearance can only be found once it is fully stored.

4.3 Non-Functional Requirements

The quality requirements below stem from three characteristics of the problem: camera data is sensitive, the data of each appearance consists of several parts that must stay

consistent, and the system runs on a personal computer without a discrete graphics card.

4.3.1 Security and Access Control

- NFR-01** **Server-side permission checks** for every request, including image requests, based on the login session rather than on data sent by the browser; hiding a function in the interface is not considered access control.
- NFR-02** **Authentication:** passwords are only stored as one-way hashes, sessions follow FR-C02, and requests that change data are protected against cross-site request forgery (CSRF).
- NFR-03** **Image protection:** frames are never publicly accessible; they only reach the browser through the server after a permission check.
- NFR-04** **Camera connection protection:** RTSP credentials are encrypted and never shown again; the system only connects to the camera network ranges declared at deployment.
- NFR-05** **Abuse limitation:** the number of calls to sensitive functions is limited, uploaded files are size-limited and format-checked, and error messages do not reveal internal information.

4.3.2 Data Integrity and Recoverability

- NFR-06** **Data consistency:** an appearance can only be found once its description, vector and frame are all stored; incomplete data must be completable or cleanable.
- NFR-07** **No destruction of historical data:** retiring a camera, disabling AI processing, locking or deleting an account and removing a result from a case never delete the original data; the cases of a locked or deleted account keep their owner and remain visible to Viewers.
- NFR-08** **Case integrity:** results in a case keep a snapshot of the camera name, area and time; concurrent updates never silently overwrite each other.

- NFR-09** **No mixing of vector spaces** from different encoder versions.
- NFR-10** **Fault isolation:** a failing component only fails the related function, is reported as that component, and does not stop the processing of other cameras.
- NFR-11** **Self-recovery:** the system reconnects automatically when an RTSP stream is interrupted.
- NFR-12** **No misleading results:** when the search components are unavailable, the system reports a failure instead of returning empty results; when an image is missing, the remaining information is still shown.
- NFR-13** **Safe retries:** multi-step operations leave a valid state when interrupted and can be retried without creating duplicate data.

4.3.3 Performance and Resources

- NFR-14** **Runs on the target machine:** the whole system runs on an Intel Core i5-11300H laptop with 16 GB of memory and no discrete graphics card.
- NFR-15** **Search independent of indexing:** analysis runs in the background, and search only matches against indexed data, so it remains usable while the system is analysing cameras.
- NFR-16** **No real-time requirement:** analysing every stream in time or processing several streams concurrently is not required; the sampling interval is chosen from predefined presets.

4.3.4 Traceability, Usability and Maintainability

- NFR-17** **The audit log** (FR-A11) cannot be modified, contains no passwords or credentials, and keeps the actor's information when the account is deleted.
- NFR-18** **Reproducible provenance:** each appearance records the versions of the Detector, Tracker and encoder and the sampling interval used.
- NFR-19** **Language:** the interface is in Vietnamese; only the query content is in English.

- NFR-20 Support for visual assessment:** person images are placed in context, the relevance score is for reference only, the system draws no conclusion about identity and does not show the models' internal confidence.
- NFR-21 Several screen sizes:** usable on a computer and on a phone about 390 pixels wide.
- NFR-22 Replaceable models:** adding a new Detector or Tracker does not require changing other components; RTSP streams and video files share the pipeline after frame reading.
- NFR-23 Replaceable camera source:** communication uses standard RTSP, so the simulated streams can be replaced by real cameras without changing the processing pipeline.

4.4 Use Case Diagram

The functional requirements of Section 4.2 are organised into 15 use cases. Figure 4.1 shows all the use cases of the system with their relations and actors, and Table 4.1 lists each use case with its actor and purpose.

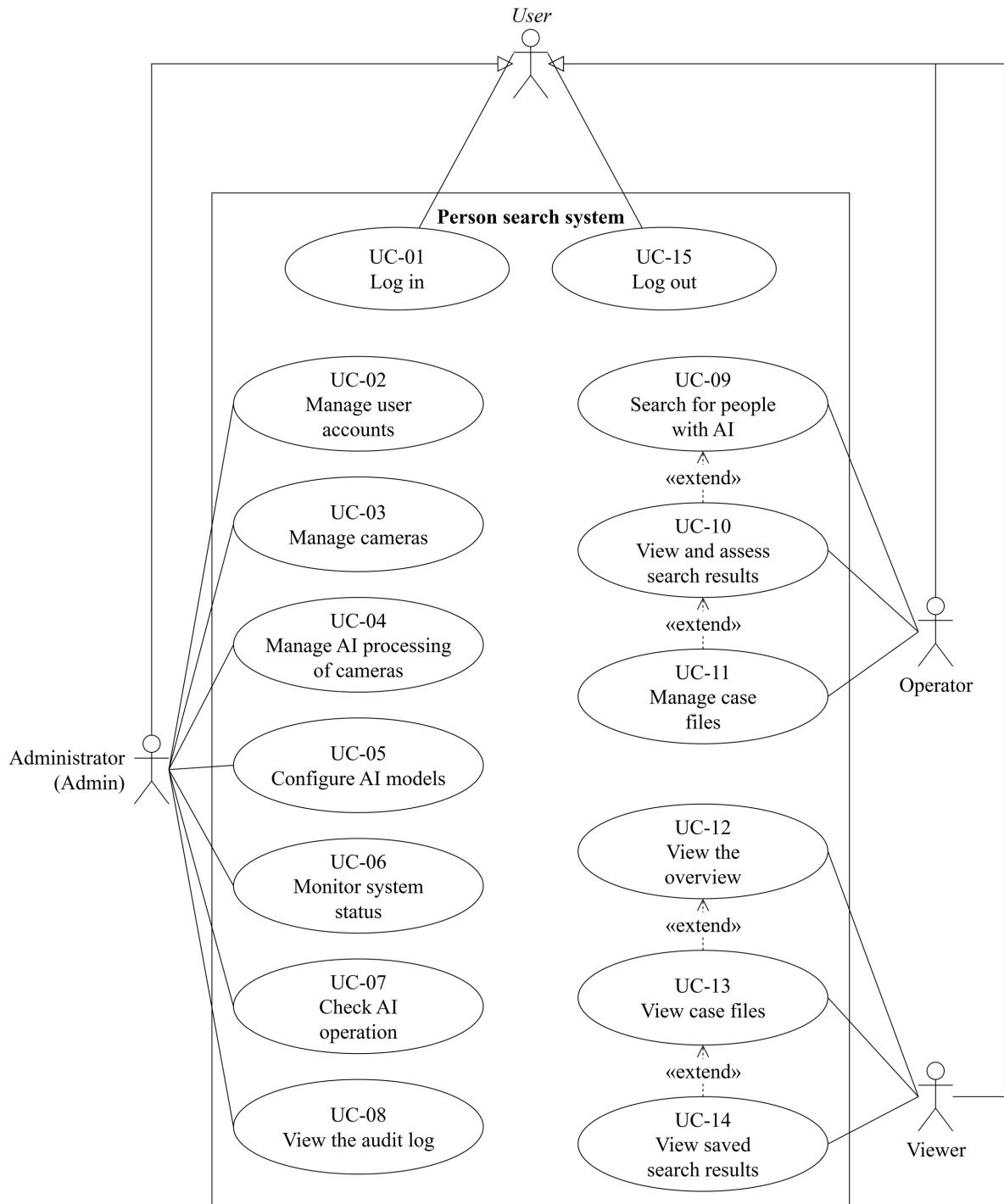


Figure 4.1: Use case diagram of the system

The diagram is organised as follows:

- **An abstract *User* actor.** The three concrete actors share login (UC-01) and logout (UC-15), so the diagram links these two use cases to an abstract *User* actor, which the three concrete actors generalise to.
- **Division by role.** The Administrator is linked to UC-02 to UC-08, the Operator to

UC-09 to UC-11 and the Viewer to UC-12 to UC-14; no use case is shared among these actors.

- **Extend relations.** UC-10 extends UC-09 and UC-11 extends UC-10: after searching, the Operator can view the details of a result and then save it into a case. Likewise, UC-13 extends UC-12 and UC-14 extends UC-13. The extending use cases are still linked directly to their actor, because the actor can also start them independently, for example opening the list of cases without searching first.
- **Login is a precondition** of every other use case, not a step inside them, so the diagram draws no «include» relation to UC-01.

The automatic system functions (Section 4.2.5) have no use case of their own because no actor triggers them; they take place when the Administrator enables AI processing in UC-04.

Table 4.1: List of use cases

Code	Use case	Actor	Purpose
UC-01	Log in	All three actors	Authenticate and access the functions of the role
UC-02	Manage user accounts	Administrator	Create, update, lock and delete accounts; assign roles and areas
UC-03	Manage cameras	Administrator	Add and update cameras, check the RTSP connection, retire and restore them
UC-04	Manage AI processing of cameras	Administrator	Enable or disable AI processing per camera; process fallback video files
UC-05	Configure AI models	Administrator	Choose the Detector and Tracker used by the whole system
UC-06	Monitor system status	Administrator	Follow the status of cameras, RTSP streams and AI processing
UC-07	Check AI operation	Administrator	Check the camera processing group and the search group
UC-08	View the audit log	Administrator	Consult the log of actions and technical events
UC-09	Search for people with AI	Operator	Search by image, description or attributes within the assigned area
UC-10	View and assess search results	Operator	View the details of each result, compare and assess it
UC-11	Manage case files	Operator	Create cases, save results, update, close and reopen cases
UC-12	View the overview	Viewer	See the system-wide figures and recent cases
UC-13	View case files	Viewer	View every case in read-only mode
UC-14	View saved search results	Viewer	View the details of a result saved in a case
UC-15	Log out	All three actors	End the session

4.5 Use Case Specifications

This section specifies the eight use cases that make up the main workflow of the system: a user logs in (UC-01); the Administrator adds a camera (UC-03), puts it under anal-

ysis (UC-04) and chooses the AI models (UC-05); the Operator searches for a person (UC-09), assesses the results (UC-10) and saves a result into a case (UC-11); the Viewer looks at the case files (UC-13). Each specification gives the actor, purpose, precondition, postcondition, main flow, exceptions (marked E) and alternative flows (marked A). The behaviour of the remaining use cases is stated in the functional requirements of Section 4.2; the data scope of each role is summarised in the permission matrix of Section 4.6.

Table 4.2: Use case specification UC-01: Log in

Actor	Administrator, Operator, Viewer
Purpose	Authenticate the account to access the functions and data suited to the user's role.
Precondition	The user has an active account.
Postcondition	The system creates a login session and takes the user to the home page of the role.
Main flow	1. The user opens the login page and enters a username and password. 2. The system checks the credentials and the status of the account. 3. The system creates a login session and determines the role of the account. 4. The system takes the user to the home page of the role; the user only sees the functions of that role.
Exceptions	E1 - Wrong credentials: The system reports that the login failed, with the same message for a wrong username and a wrong password; the failure is recorded in the audit log.
	E2 - Locked or deactivated account: The system refuses the login.
	E3 - Too many attempts: If there are too many login attempts in a short time, the system refuses temporarily and states how long to wait before trying again.
Alternative flows	None.

Table 4.3: Use case specification UC-03: Manage cameras

Actor	Administrator
Purpose	Manage the cameras of the system: add and update them, check the RTSP connection, retire a camera from operation and bring it back.
Precondition	The Administrator is logged in; the list of surveillance areas has been declared.
Postcondition	The information and operational status of the camera are updated; the result of the RTSP connection check is recorded.
Main flow	1. The Administrator opens camera management; the system shows the list of cameras with their area, operational status, RTSP connection status and AI processing status. 2. The Administrator chooses to add a camera and enters its code, name, area and RTSP address, optionally with the camera's username and password; the RTSP address is optional for a camera processed only through video files. 3. The system validates the data, separates the credentials from the address and stores them encrypted, then saves the camera with the connection status <i>Unverified</i>. 4. The system tries to connect to the RTSP stream and updates the connection status of the camera.
Exceptions	E1 - Invalid data: If required information is missing, the address is not a valid RTSP address, or the camera code already exists, the system refuses to save.
	E2 - Connection fails: The camera is still saved with the status <i>Offline</i> and a warning; the Administrator can fix the configuration and check again later.
	E3 - Address outside the permitted network: If the address is not an IP address in the camera network ranges declared at deployment, the system refuses to check the connection and the camera stays <i>Unverified</i> .
Alternative flows	A1 - Update a camera: The Administrator edits the name or the RTSP address; the area of the camera cannot be changed. When the address changes, the connection status returns to <i>Unverified</i> and is checked again.
	A2 - Check the connection again: The Administrator requests a connection check for a camera; the system tries to connect and updates the status.
	A3 - Retire from operation: The camera is retired, AI processing is disabled and its running job is cancelled; the data already created is kept but temporarily cannot be searched.
	A4 - Bring back into operation: The camera returns to operation with AI processing disabled; its old data can be searched again as normal.

Table 4.4: Use case specification UC-04: Manage AI processing of cameras

Actor	Administrator
Purpose	Decide which cameras are analysed by enabling or disabling AI processing for each camera, and process a recorded video file for a camera when needed.
Precondition	The Administrator is logged in; the camera has been added to the system and is operational.
Postcondition	The AI processing status of the camera is updated; the system starts or stops including the camera in the analysis.
Main flow	1. The Administrator opens AI processing management; the system shows the list of cameras with their AI processing status and the model configuration in use. 2. The Administrator enables or disables AI processing for a camera and confirms. 3. The system checks the status of the camera and the current model configuration. 4. The system saves the new status. When enabled, the camera is analysed in turn with the other cameras that have AI processing enabled. When disabled, the camera's running job is cancelled and the system stops creating new data; existing data is kept. 5. The system reports the result to the Administrator.
Exceptions	E1 - Camera offline: If a camera with RTSP has lost its connection, the system refuses to enable AI processing and reports it.
	E2 - AI models not ready: If the current model configuration is incomplete or unavailable, the system refuses to enable AI processing.
	E3 - Camera no longer operational: If the camera has been retired, the system does not allow its AI processing status to change.
	E4 - Invalid video file: If the uploaded file has the wrong format or exceeds the permitted size, the system refuses it and creates no job.
Alternative flows	A1 - Process a video file: The Administrator chooses a camera, uploads a recorded video file, and chooses the recording start time and the sampling mode. The system checks the file, creates a processing job that goes through the same analysis pipeline as an RTSP stream and shows its progress. This is the only way to process a camera without RTSP.

Table 4.5: Use case specification UC-05: Configure AI models

Actor	Administrator
Purpose	Choose the person detector (Detector) and tracking algorithm (Tracker) used by the whole system. The image and text encoders are fixed components that cannot be changed from the interface.
Precondition	The Administrator is logged in; the system has a registered list of Detectors and Trackers.
Postcondition	The chosen Detector-Tracker pair becomes the current configuration; processing jobs created afterwards use it.
Main flow	1. The Administrator opens the AI model configuration; the system shows the Detector-Tracker pair in use and the list of registered models, in which unavailable models are marked and cannot be selected. 2. The Administrator chooses a Detector, a Tracker or both, then confirms. 3. The system checks that the models are ready and that the Detector-Tracker pair is compatible, then trial-loads the new configuration. 4. The system saves the new configuration as the current one and reports the result.
Exceptions	E1 - Model unavailable: If a chosen model is missing or cannot be loaded, the system refuses and keeps the previous valid configuration.
	E2 - Incompatible Detector and Tracker: The system refuses and asks the Administrator to choose another pair.
Alternative flows	None.

Table 4.6: Use case specification UC-09: Search for people with AI

Actor	Operator
Purpose	Find a person in the analysed camera data by image, English description or appearance attributes, within the assigned surveillance area.
Precondition	The Operator is logged in; the account has an assigned surveillance area with analysed data.
Postcondition	The system shows at most k best-matching results within the valid scope, in decreasing order of relevance.
Main flow	1. The Operator chooses the form of search and provides an image of the person, an English description or the attributes. 2. The Operator may also choose cameras of the area and a time range, choose the number of results k among 4, 8, 12 and 16, then sends the request. 3. The system validates the query and limits the search scope to the operational cameras of the account's area. 4. The system encodes the query, matches it against the indexed data and takes the k results with the highest relevance.

(continued from Table 4.6)

	5. The system shows the person image, camera, area, time of appearance and relevance score of each result.
Exceptions	E1 - Invalid query: If the image has a wrong format, the description is not in English, the attributes are invalid or k is not an allowed value, the system reports the error and asks for new input.
	E2 - No data in scope: The system reports that there is no matching data and suggests widening the time range or choosing more cameras.
	E3 - Search components unavailable: If the encoder or the vector store is not working, the system reports that the search failed instead of returning an empty list.
	E4 - Request out of scope: If a request sent directly contains a camera outside the account's area, the server refuses it.
Alternative flows	A1 - Search again from a result: The Operator drags the image of a result into the image search box to search again for the same person.

Table 4.7: Use case specification UC-10: View and assess search results

Actor	Operator
Purpose	View the details of each result and assess it to decide which results really show the person being sought.
Precondition	The Operator has performed UC-09 and the system has returned at least one result.
Postcondition	The Operator has assessed the results; the original data is unchanged.
Main flow	1. The Operator views the list of results with the person image, camera, area, time of appearance and relevance score. 2. The Operator selects a result; the system shows the full frame with the bounding box of the person found, together with the details. 3. The Operator compares it with the person being sought and may go on to other results. 4. For a result to keep, the Operator chooses Create new case or Add to existing case; the system continues with UC-11.
Exceptions	E1 - Result no longer available: If the data of the result has been removed, the system reports that the result is no longer available.
	E2 - Image cannot be shown: The system still shows the remaining information and reports that the person image cannot be shown.
Alternative flows	A1 - Sort results: The Operator sorts the list by relevance or by time of appearance.
	A2 - Filter results: The Operator filters the current list by camera or time range without sending a new query.

Table 4.8: Use case specification UC-11: Manage case files

Actor	Operator
Purpose	Save the search results related to an incident into a case the Operator owns, and follow the case until it is closed.
Precondition	The Operator is logged in; the result to save is within the Operator's search scope.
Postcondition	The case is created or updated, with exactly one Operator in charge, a title, notes, a status and the saved results.
Main flow	1. From a search result, the Operator chooses Create new case or Add to existing case. 2. For a new case, the Operator enters a title and notes; the system sets the logged-in account as the owner. For an existing case, the system only lists the open cases owned by the Operator. 3. The system saves the result as a new entry in the case, even if the appearance is already in it. 4. The Operator may edit the title and notes and remove results from the case; the original data of the appearance is not deleted. 5. When the work is done, the Operator marks the case Closed; the system records the closing time and locks the case.
Exceptions	E1 - Result or case no longer available: The system reports it, saves nothing and reloads the list of cases.
	E2 - Result outside the Operator's rights: If a request sent directly tries to save a result outside the Operator's scope, the system refuses it.
	E3 - Case already closed: The system refuses to add or remove results or to edit the title and notes, and asks for the case to be reopened first.
	E4 - Data changed elsewhere: The system refuses to overwrite and asks the Operator to reload the case.
Alternative flows	A1 - Close while saving: If the Operator ticks Mark the case as closed when saving, the case is closed right after the save; if this step fails, the result is still saved and the system asks the Operator to close the case again.
	A2 - Reopen a case: The Operator reopens a closed case; it returns to Open and can be edited as normal.

Table 4.9: Use case specification UC-13: View case files

Actor	Viewer
Purpose	View every case created by the Operators with its saved results, in read-only mode.
Precondition	The Viewer is logged in.
Postcondition	The Viewer has seen the content of the case; no data has changed.
Main flow	1. The Viewer opens the case files; the system shows the list of all cases in the system.

(continued from Table 4.9)

	2. The Viewer may filter by status, creation time or the Operator in charge, then selects a case. 3. The system shows the title, notes, status, Operator in charge, creation time and saved results, without relevance scores. 4. The Viewer may select a result to see its details; the system continues with UC-14 (View saved search results).
Exceptions	E1 - No cases yet: The system shows an empty state.
	E2 - Case cannot be retrieved: The system reports that the case cannot be shown and allows retrying.
	E3 - Some results no longer available: The system still shows the case and the remaining data, and marks the results that could not be retrieved.
Alternative flows	A1 - Open from the overview: The Viewer selects a case in the recent-cases list of UC-12; the system opens its details directly.

4.6 Permission Matrix

The use case diagram (Figure 4.1) shows which functions each actor uses. For the same function, however, the roles also differ in the *scope of data* they may access. Table 4.10 summarises the rights of each role for each group of actions, with the conditions on the scope of data.

Table 4.10: Permission matrix by role

Group of actions	Administrator	Operator	Viewer
Log in, log out	Yes	Yes	Yes
Manage accounts, assign areas to Operators	Yes	No	No
Manage cameras, enable/disable AI processing, upload fallback videos, choose the Detector/Tracker	Yes	No	No
Monitor status, check AI operation, view the audit log	Yes	No	No
Search for people, view the images and frames of search results	No	Only on operational cameras of the current area	No
Create cases, add or remove results, edit the title and notes, close or reopen	No	Only for the cases the Operator owns; newly added results must belong to the current area	No
View cases and the images of saved results	No	The cases the Operator owns, including after a change of area	Every case, read-only
View the overview dashboard	No	No	System-wide

The images and frame of an appearance are granted on one of two grounds: the search right, meaning the appearance belongs to an operational camera in the Operator's current area; or the right over a case, meaning the appearance has been saved in a case the user may see. Thanks to the second ground, an Operator who has moved to another area and a Viewer can still see the images in a case, even without the right to search on that camera. Every right in the matrix is checked on the server (NFR-01).

How the tests verify that the system enforces every cell of this matrix is presented in Section 8.1.

Chapter 5

System Design

This chapter presents the design of the system: the overall architecture, the main processing flows, the database, the application programming interface, the security mechanisms and the user interface.

5.1 System Architecture

Figure 5.1 shows the overall architecture of the system, organised around the two stages of Section 3.1.3: the *indexing* flow in the left column turns camera data into searchable appearances, and the *search* flow in the right column serves the user. The two flows meet at the storage layer at the bottom. In the figure, black lines show the indexing flow, blue lines the search and data access flow, dashed lines the creation and claiming of jobs, and the thick line the continuous RTSP stream from the cameras.

5.1.1 Components

- **Camera data source.** The video files of the test dataset are streamed continuously by FFmpeg to a MediaMTX server as RTSP streams; each stream is a logical camera (Section 3.7.3) and, for the system, no different from a real IP camera.
- **Web application.** The interface is built with React and shared by all three roles; the web application only talks to the application server and never accesses a data store directly.
- **Application server.** A Flask process provides the API for every function and keeps the RaSa encoder loaded to encode search queries.

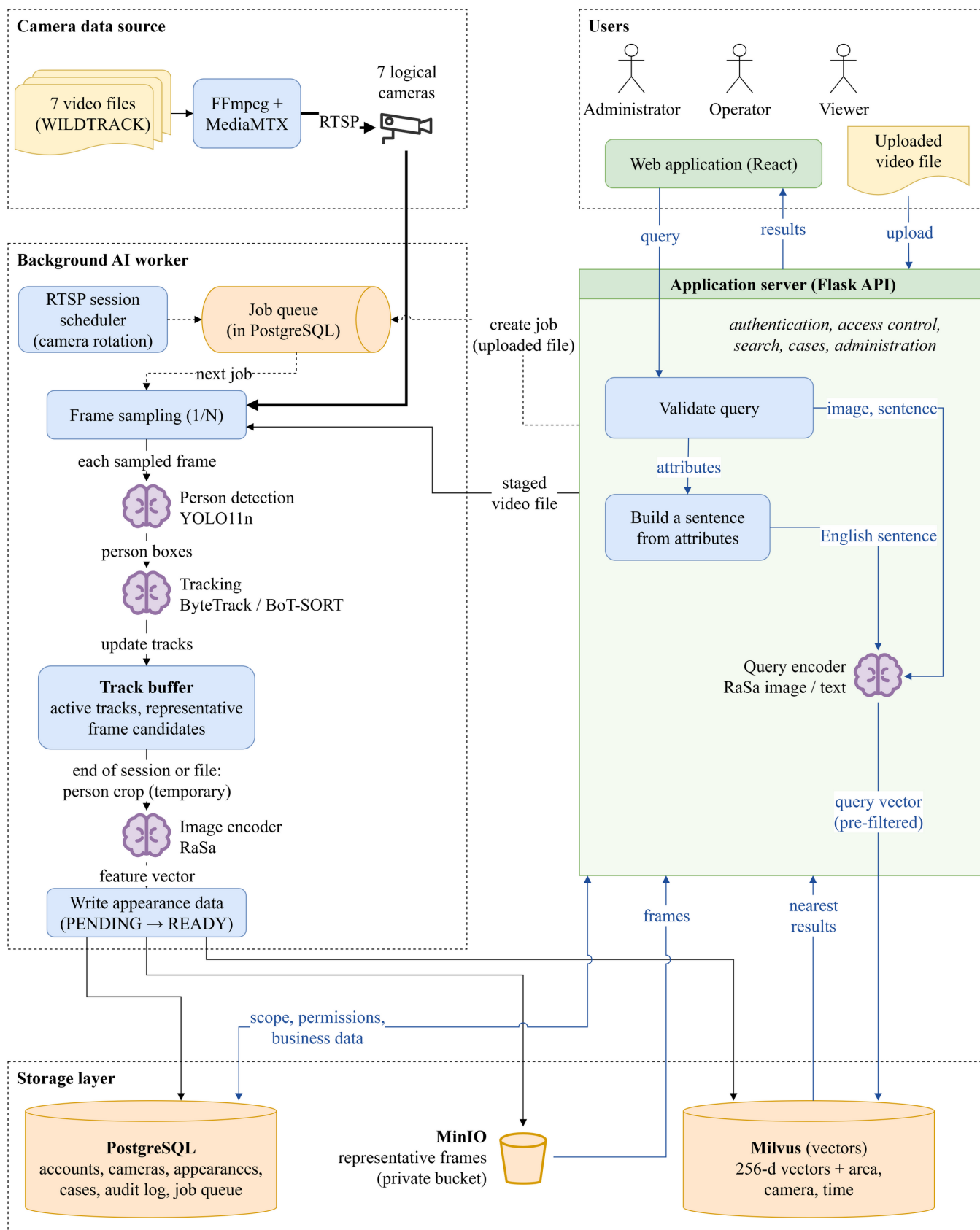


Figure 5.1: Overall architecture of the system

- **Background worker.** A separate Python process performs all the AI processing: it creates RTSP processing sessions in rotation among the cameras, takes the jobs of the queue one by one and runs the analysis pipeline.
- **Storage layer.** PostgreSQL stores the business data and the job queue; Milvus stores the feature vectors with the fields used for filtering; MinIO stores the representative full frame of each appearance.

5.1.2 Design Principles

The most important principle is to separate the AI processing from the application server. On the target machine, video analysis is about seven times slower than real time, so the application server does not read RTSP streams or run the analysis pipeline; it only creates jobs. The background worker executes those jobs independently. Users can still search normally while the system is indexing, and errors during analysis do not stop the application server.

The job queue lives in PostgreSQL itself: each job is a row, instead of an additional message queue system. This reduces the number of components to deploy on a personal machine and keeps the job status next to the business data. The background worker executes one job at a time, and the RTSP sessions, which have a bounded length, rotate among the cameras, so that every camera is analysed in turn without overloading a CPU-only machine.

For storage, each store holds one kind of data and PostgreSQL is the source of truth. The three parts of the same appearance are linked by a shared identifier, and every result returned by Milvus is checked against PostgreSQL before it is shown. Every data access, images included, goes through the application server, so that the permission matrix of Section 4.6 is enforced at a single point.

Finally, the two flows share the encoder and the pipeline. Queries are encoded with exactly the RaSa weights used for indexing, so both sides lie in the same vector space. RTSP streams and video files only differ in the frame reading step.

5.2 Main Processing Flows

5.2.1 Camera Data Processing Flow

All analysis is organised into *processing jobs* stored in a queue, each tied to one camera and one data source. For **automatic RTSP sessions**, when the queue is empty, the scheduler in the background worker chooses a camera that is operational, has AI processing enabled and has an RTSP address, preferring the camera that has waited longest, and creates a session limited to 1,800 source frames, about 30 seconds of video at 60 frames per second; a camera whose last session failed is skipped for 5 minutes so that it does not take the turn of the other cameras. Thanks to the rotation, the cameras are analysed in turn, but between two sessions of the same camera there are periods that are not analysed (Section 1.4). For **uploaded video files**, the application server checks the file, stores it temporarily and creates the corresponding job.

Job life cycle. Figure 5.2 shows the states of a job. A new job is *Queued*. When the background worker claims it, the job becomes *Running* with a 60-second lease that is renewed periodically during processing; if the worker stops suddenly and the lease expires, the job can be claimed again. For a video file job, transient errors are retried after a delay, up to three times; an RTSP session that fails ends at once, and the camera is processed again in a later session. A job ends as *Succeeded*, as *Failed* after an unrecoverable error or when the retries are exhausted, or as *Cancelled* when the Administrator cancels it, or when the camera is retired or its AI processing is disabled while the job is queued or running.

Processing steps. Figure 5.3 shows the flowchart of one job, which has two phases. In the *per-frame loop*, each frame is read with its index and timestamp (Section 3.7.4), and only one in every N frames goes through the Detector and the Tracker. The tracking result updates the *track buffer*, which holds the appearances being tracked with their representative frame candidates, and the system moves on to the next frame. When an appearance ends, its representative frame is chosen and kept in memory. When the source runs out or the session reaches its frame count, the appearances still being tracked are

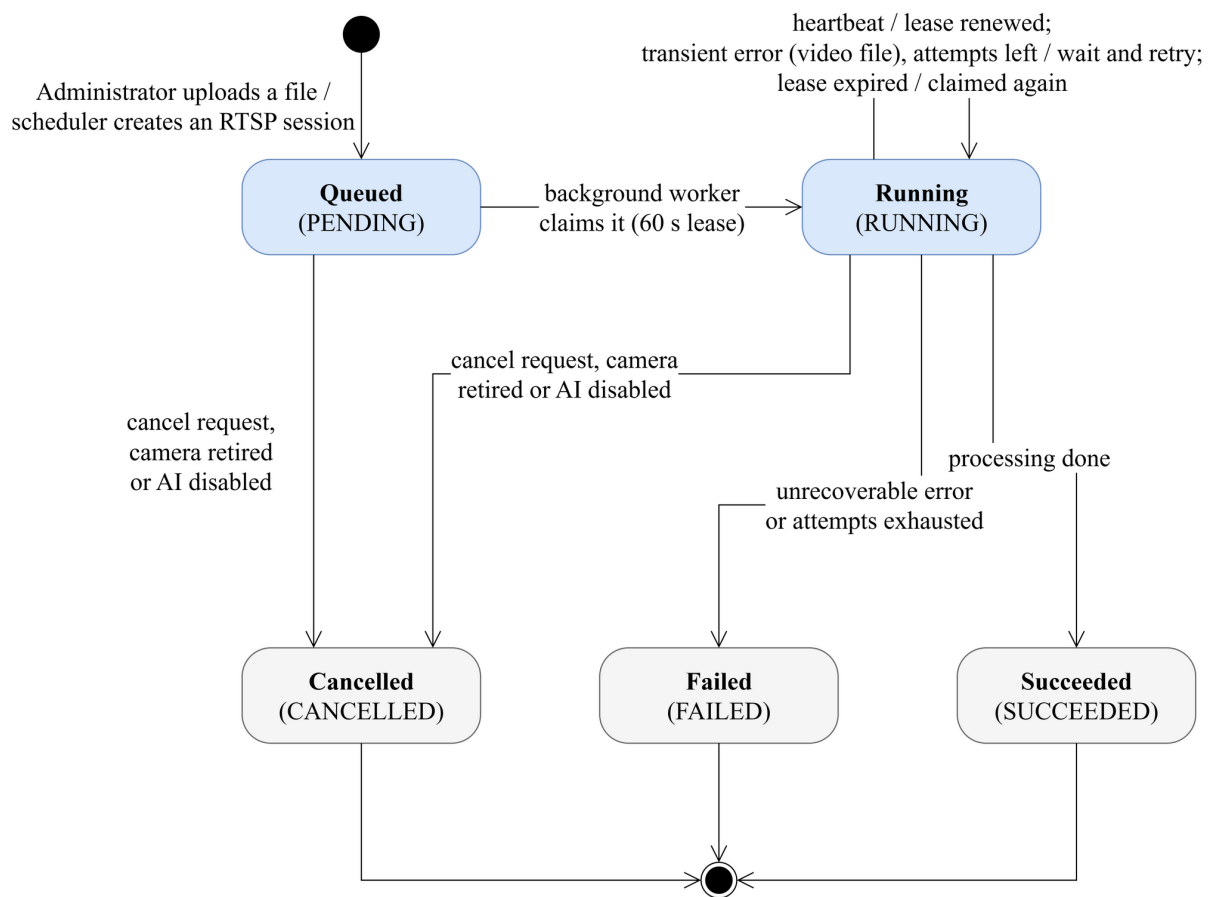


Figure 5.2: State diagram of a processing job

ended as well, and the job moves to the *encode and write* phase: for each appearance, the person region of the representative frame is cropped temporarily in memory, widened by 5% of the box size on each side, passed through the image encoder to produce the feature vector, and written to the three stores in the order of Section 5.3.4. The appearances of a job can therefore only be found after the job has finished its source. For RTSP streams, the source of a job is a session of 1,800 frames, so a camera’s data is written session by session: a person appearing in front of the camera can only be searched once the session containing that appearance has been processed. If the job is cancelled or its lease expires while it is reading and analysing the source, none of its appearances is written. Each appearance is written with the version of the Detector-Tracker configuration, the version of the encoder and the sampling interval N used.

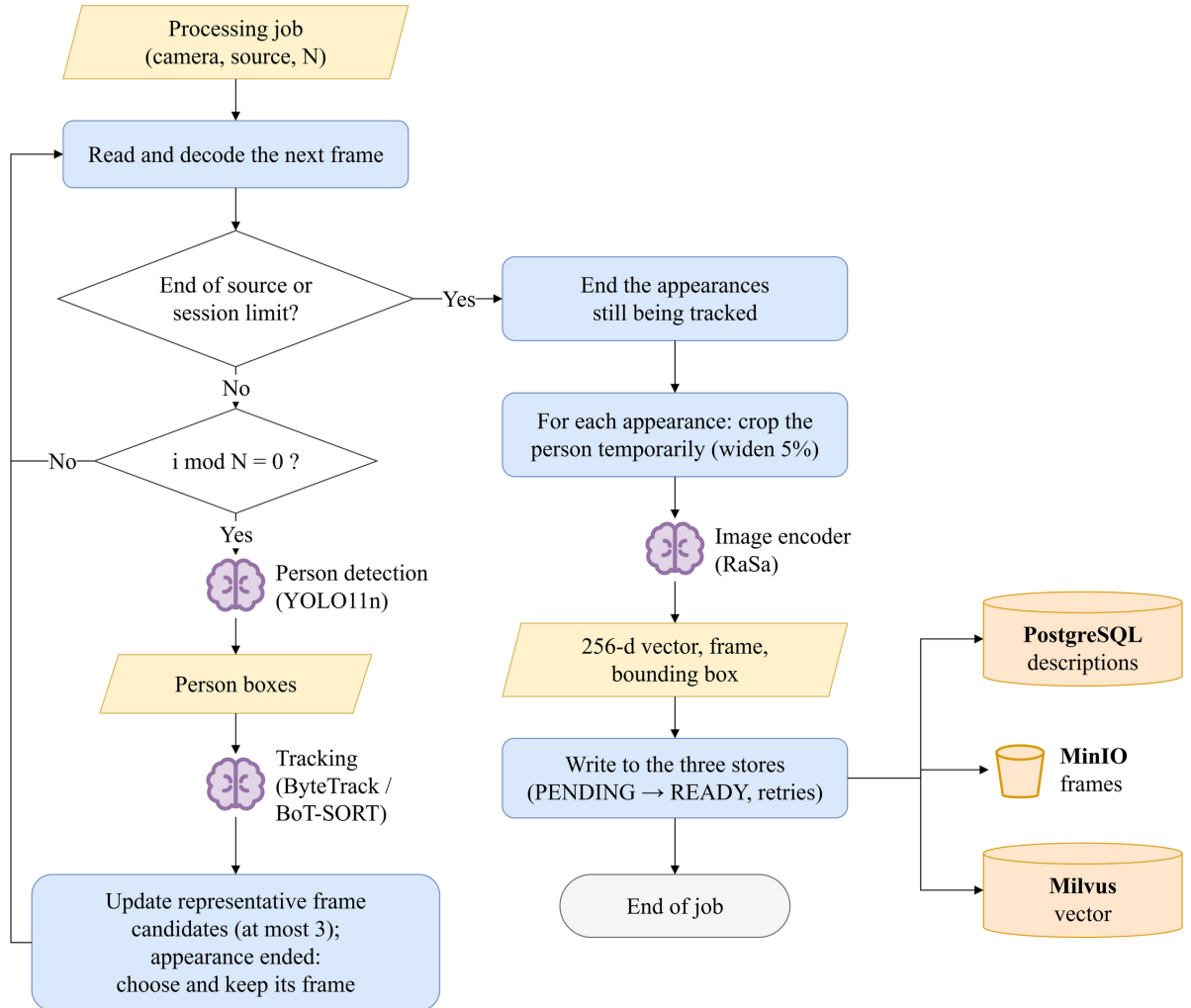


Figure 5.3: Flowchart of one processing job

5.2.2 Representative Frame Selection

Each appearance stores only *one* full frame, used both to create the vector and to display the result, so this frame should show the person as clearly as possible. The representative frame selector works as follows:

- **Keep at most three candidates** with the highest scores for each ongoing appearance; the total memory for the candidates of all appearances is limited to 256 MB.
- **Hard filters.** A candidate only passes when its box is at least 24×48 pixels, the person region is sharp enough, the box touches fewer than two edges of the frame and its shape does not change abruptly from the previous observation (area not changing more than 4 times, aspect ratio not changing more than 2.5 times), since an abrupt change usually signals a wrong person or occlusion.
- **The quality score** is a weighted sum of the person's size in the frame (0.30), sharpness (0.25), completeness of the body (0.20), stability relative to the previous observation (0.15) and closeness to the middle of the appearance (0.10), minus penalties when the box touches the frame edges or overlaps other people.
- **Choice at the end.** The highest-scoring candidate among those passing the hard filters is chosen; if none passes, the highest-scoring candidate is still chosen and the appearance is marked low quality instead of being discarded.

The criteria and weights were designed empirically, are stored in a configuration file and give deterministic results.

5.2.3 Search Flow

Figure 5.4 shows the search flow as a sequence diagram.

The web application sends the query with the camera filter, time range and number of results k . The application server then:

1. **validates the input:** a JPEG or PNG image of at most 10 MB; attributes from the allowed list forming a valid combination; k in $\{4, 8, 12, 16\}$;

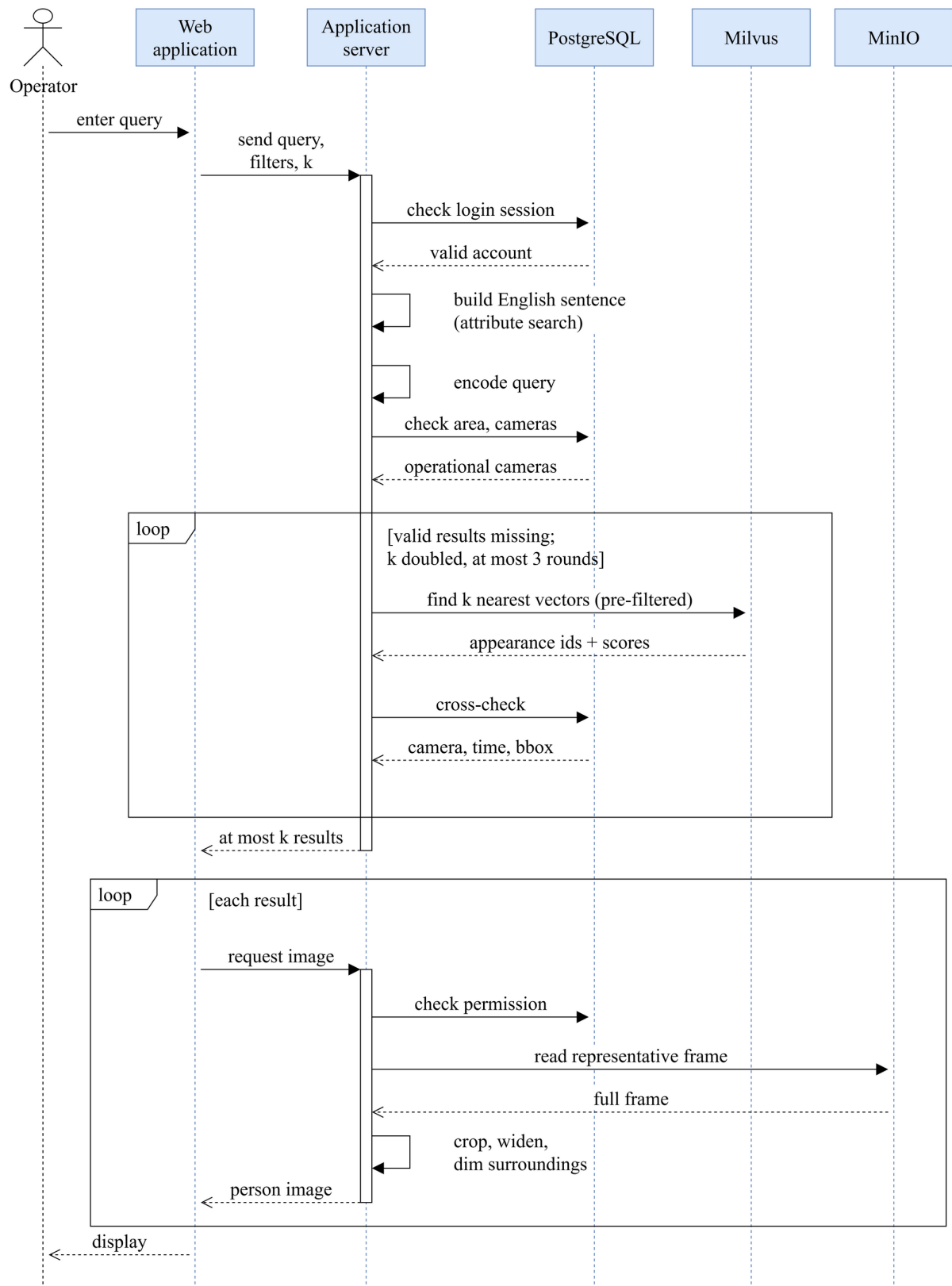


Figure 5.4: Sequence diagram of the search flow

2. **encodes the query:** an image through the image encoder, a description through the text encoder; attributes are combined into an English sentence from a fixed template, for example “*A woman wearing a red jacket and blue jeans, carrying a handbag.*”, and then handled as a description;
3. **determines the scope:** the area is taken from the Operator’s account; the cameras are those selected, or every operational camera of the area if none is selected; cameras outside the scope are rejected;
4. **runs a pre-filtered vector search** in Milvus, checks the results against PostgreSQL and searches again with twice the number when valid results are missing, up to three rounds (Section 3.6.3).

The returned list only contains the descriptive information and relevance score of each result. Images are fetched through a separate request for each result, in which the server checks the permission again and renders the person image from the representative frame (Section 6.3.2). This keeps the list fast to return and makes every image go through the permission check.

5.2.4 Case Management Flow

Figure 5.5 shows the flow for saving a search result into a new or existing case, with the option to close the case while saving.

When saving, the interface only sends the appearance identifier, not the relevance score or display information. The server checks that the appearance is ready and belongs to an operational camera in the Operator’s current area, and that the target case is owned by that Operator and open. If valid, the server creates a new result entry with a *snapshot* of the camera name, area name and time of appearance, so that the case still shows correctly if the camera is renamed later or its frame is temporarily unavailable. Each case carries a version number; update requests must include the version being displayed, so concurrent changes from elsewhere are never silently overwritten. The option to close the case is performed by the interface after saving: it reads the case again to get the latest version number, then sends the status change with that version number; if the status change fails, the result is kept and the user is told to close the case again. When a

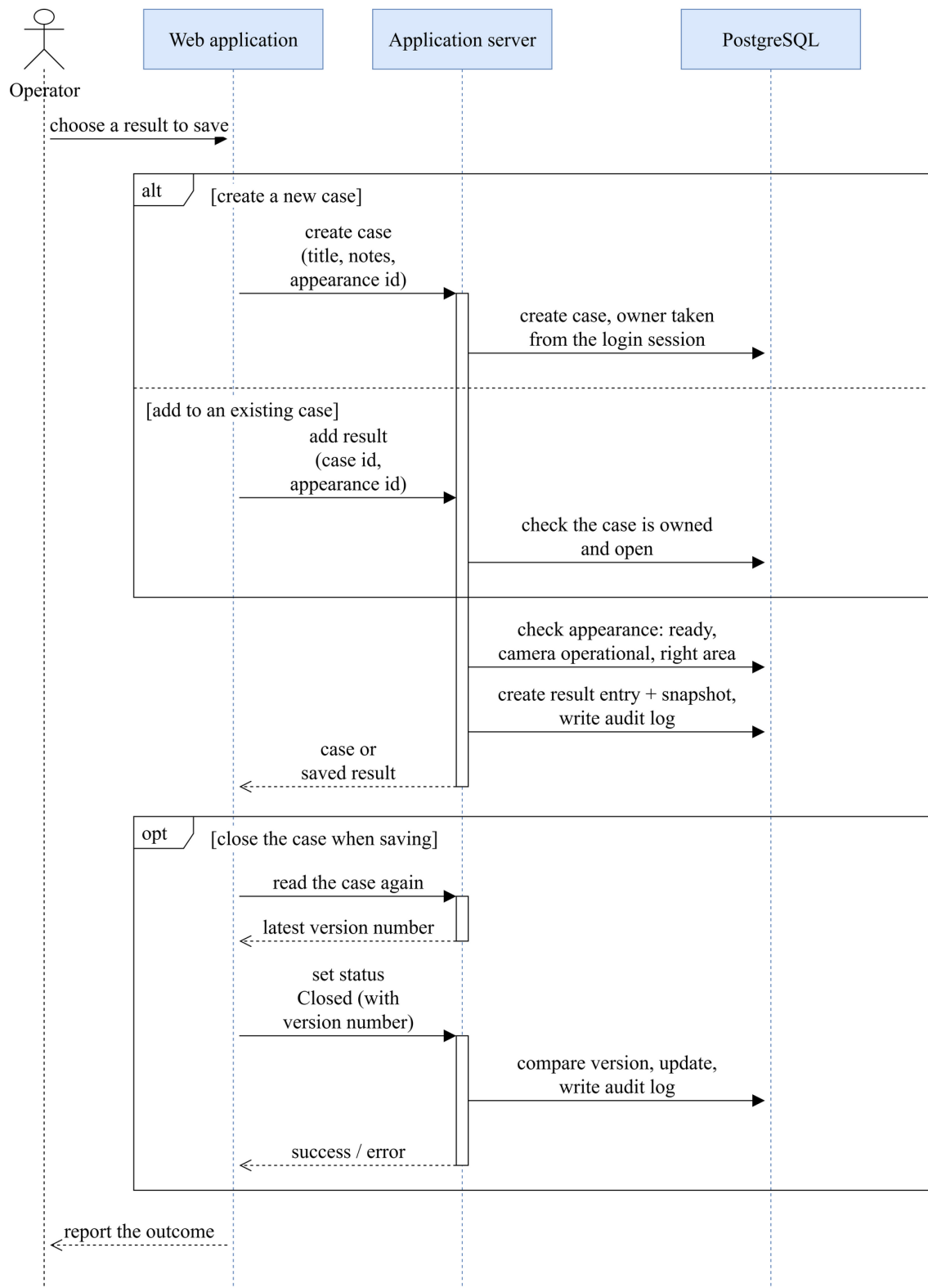


Figure 5.5: Sequence diagram of saving a result into a case

case is opened, images are granted by the right over the case rather than the search right (Section 4.6).

5.3 Database Design

5.3.1 Relational Database

PostgreSQL stores all the business data and is the source of truth for permissions and states. The database has 12 tables in five groups, as in Table 5.1.

Table 5.1: Tables of the relational database

Group	Tables	Role
Organisation and access	areas, users, auth_sessions	Surveillance areas, user accounts, login sessions
Cameras and AI processing	cameras, ai_config_versions, processing_jobs, worker_heartbeats	Logical cameras, versions of the Detector-Tracker-encoder configuration, job queue, background worker status
Search data	person_tracks, storage_outbox_events	Descriptions of the appearances; synchronisation events for the other two stores (Section 5.3.4)
Cases	cases, case_results	Cases and their saved results
Traceability	audit_logs	Audit log, append-only

Figure 5.6 shows the entity-relationship diagram of the 11 tables that have foreign keys, in crow’s foot notation; each table only shows its primary key, foreign keys and main attributes. The worker_heartbeats table has no foreign key and is not drawn.

Relationships. Most foreign keys are mandatory: each camera belongs to exactly one area, each case has exactly one owner, and each appearance belongs to exactly one camera and was created by exactly one job with one AI configuration version. Three foreign keys may be null: an account only has an area when it is an Operator, a job only has a requester when it was created from an uploaded file, and a log entry may not be tied to a user, for example when it records a technical error. cases and person_tracks have a many-to-many relationship through case_results. Each row of this table has its

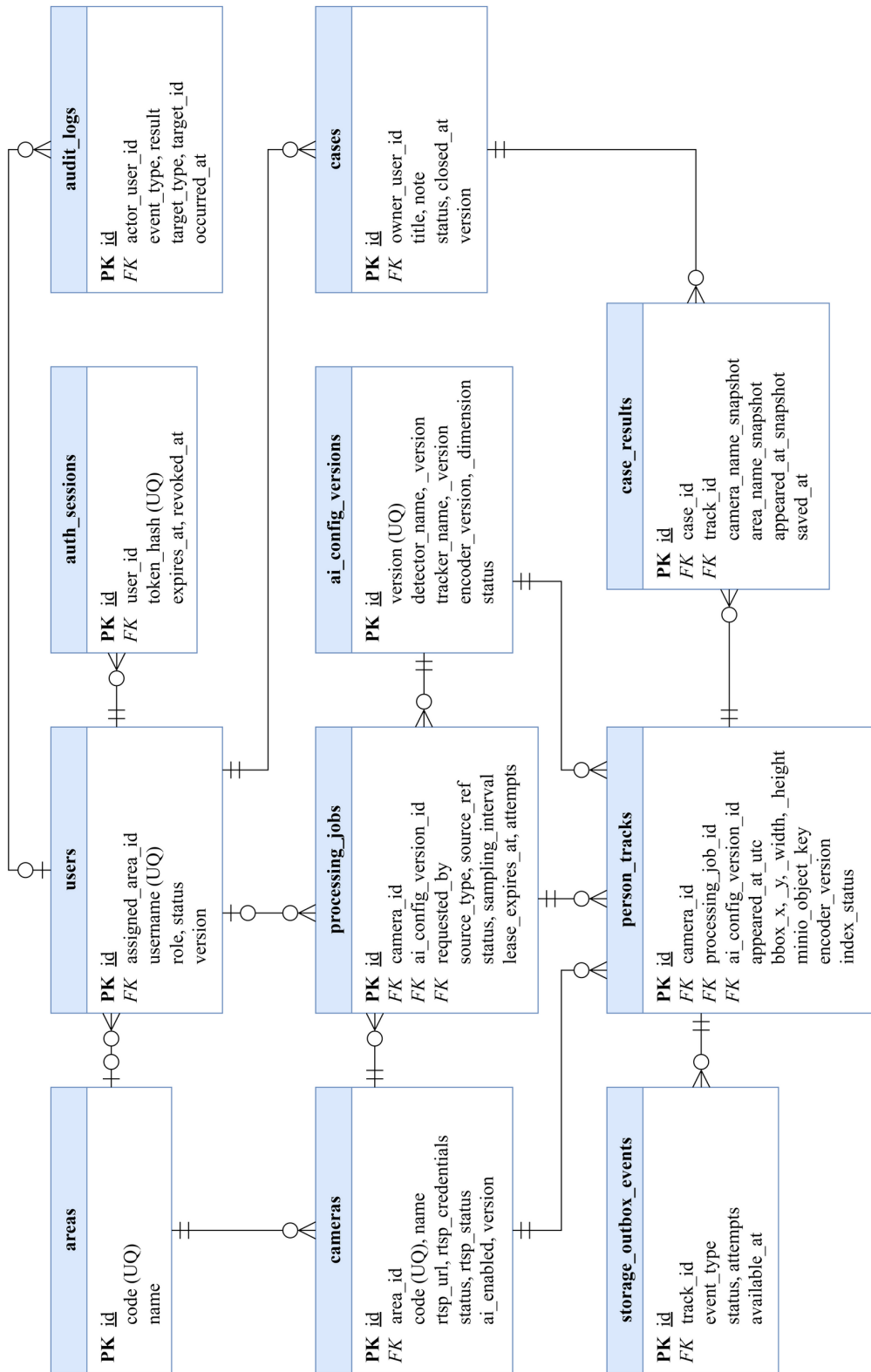


Figure 5.6: Entity-relationship diagram of the relational database

own primary key, stores a snapshot of the information at the time of saving, and has *no* uniqueness constraint on the case-appearance pair, since each save creates a new entry.

Integrity constraints. Many business rules are expressed directly in the database, so that invalid data cannot be written even if the application layer has a bug:

- An Operator account always has an area and the other two roles never do; a case has a closing time if and only if it is CLOSED; only one AI configuration version is ACTIVE; the area code and the area of a camera do not change after creation.
- Most foreign keys use the RESTRICT rule, so data that is still referenced cannot be deleted; accounts and cameras are not deleted but change status (INACTIVE, RETIRED). Only login sessions and result entries are deleted with their parent.
- An appearance can only be READY when it has the object key, hash and size of its frame and the time its vector was written. No table stores a relevance score.
- The users, cameras and cases tables have a version column to prevent overwriting on concurrent updates (Section 5.2.4); the requester-idempotency key pair of `processing_jobs` is unique, so a repeated upload request creates no additional job.

Indexes are also created for frequent queries, such as appearances by camera and time (with a separate index containing only ready appearances), cases by owner, and log entries by actor.

5.3.2 Feature Vector Storage

Milvus serves a single purpose: quickly finding the vectors closest to the query vector within a filtered scope; all other information and every permission decision live in PostgreSQL.

One collection per encoder version. Milvus organises data into *collections*, similar to tables. The system creates a separate collection for each encoder version, for example `person_track_embeddings_rasa_cuhk_pedes_v1`, because vectors from two

versions belong to two representation spaces that cannot be compared. A fixed alias always points to the collection of the version in use; when the background worker starts, the dimension of the collection is checked against the configuration. Each appearance corresponds to one record with the fields of Table 5.2.

Table 5.2: Schema of the Milvus collection of feature vectors

Field	Type	Meaning
track_id	VARCHAR(36), primary key	Appearance identifier, the same as the primary key of person_tracks
embedding	FLOAT_VECTOR, 256 dimensions	Feature vector, L2-normalised
area_id, camera_id	VARCHAR(36)	Area and camera, used for scope filtering
appeared_at_epoch	INT64	Time of appearance (Unix seconds), used for time filtering
encoder_version	VARCHAR(64)	Version of the encoder that produced the vector
index_status	VARCHAR(16)	Readiness; only READY records are searched

The area, camera and time fields are copied from PostgreSQL so that Milvus can pre-filter before searching (Section 3.6.2); since the area of a camera does not change after creation, this copy cannot diverge. The operational status of a camera, by contrast, changes with the Administrator’s actions and is not copied: each search takes the list of operational cameras from PostgreSQL and puts it in the filter, so the data of a retired camera is hidden immediately without updating Milvus.

Index and operations. The vector field has an HNSW index with the inner-product metric, $M = 16$, $efConstruction = 128$, and $ef = 64$ at search time (Section 3.6.3). Vectors are written with an *upsert* keyed by the appearance identifier, so retries create no duplicate records, and are checked for the right dimension, the absence of invalid values and unit length before writing or searching. Every operation uses strong consistency, so a vector just written appears in the very next search.

5.3.3 Frame Storage

Each appearance has exactly one image file in MinIO: the representative full frame, stored as JPEG at quality 90; PostgreSQL only stores the object key, hash and file size. The system does not store cropped person images as separate files: the person image in the result list is rendered from the full frame and the bounding box on request (Section 6.3.2). One file therefore serves both views, the cropping can change without regenerating data, and no standalone person images exist that could be copied out of control. The original video is not stored.

The key of each file is derived deterministically from the appearance, following the pattern

```
tracks/v1/camera code/year/month/day/appearance id/representative.jpg
```

so a retry rewrites the same file instead of creating a new one; the versioned prefix allows the layout to change later, and grouping by camera and day makes clean-up by camera or time range easy.

Before writing, the file is checked for the right format, a size of at most 20 MB and dimensions matching its description; its SHA-256 hash is stored in both MinIO and PostgreSQL. A write to an existing key with the same hash reuses the existing file, and one with a different hash is rejected as a conflict; on reading, the hash is recomputed to detect corruption. The files live in a private bucket, the application accesses it with an account that only has rights on that bucket, and the browser never receives a direct address to MinIO.

5.3.4 Consistency Across the Stores

An appearance is stored as three parts in three stores, linked by the same identifier. The three stores share no transaction, so the process may stop midway or a store may be temporarily unreachable. The system guarantees that incomplete data never appears in search results and can be completed or detected later.

Appearance states. Each appearance record has a synchronisation state, with the allowed transitions of Figure 5.7. Only appearances in the *Ready* state can be searched;

transitions not shown in the figure are rejected by the application layer.

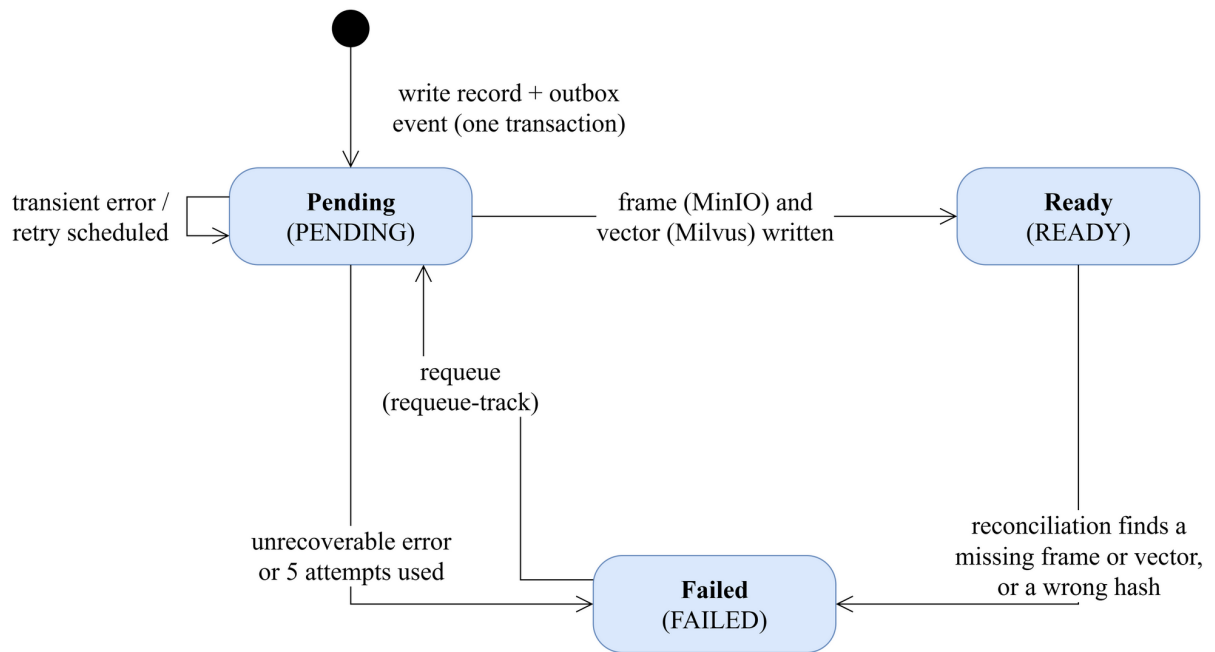


Figure 5.7: State diagram of an appearance

Write order. An appearance is written in a fixed order:

1. In *one* PostgreSQL transaction, create the record in the *Pending* state and an event in `storage_outbox_events` holding everything that must be written to the other two stores, the vector included. This is the *transactional outbox* pattern: the intent to write to another system is stored in the same transaction as the main data.
2. Write the frame to MinIO under its deterministic key.
3. Write the vector to Milvus, then read it back to confirm the record exists with the right area and camera.
4. In a new transaction, move the record to *Ready*, store the hash and size of the frame and the time the vector was written, and mark the outbox event as done.

Because PostgreSQL is always written *first*, every frame and vector can be matched with a known record, and anything without a matching record is orphaned data. Because the *Ready* state is set *last*, an appearance can only be found once all its data is present. Steps 2 and 3 can both be repeated safely thanks to the deterministic key and the upsert.

Error handling and maintenance tools. If step 2 or 3 fails with a transient error, the record stays *Pending* and the outbox event is scheduled for retry, with a delay starting at 2 seconds, doubling each time, up to 300 seconds. After five unsuccessful attempts, or on an unrecoverable error such as a hash conflict, the record moves to *Failed* and the incident is written to the audit log. Since the outbox event holds all the data, a retry does not need to rerun the video analysis. Operators also have a command-line tool to complete pending appearances; to reconcile the three stores, finding records pending for too long, ready records missing their frame or vector, and orphaned data (by default it only reports; on request it quarantines broken records and deletes orphans); to move a failed appearance back to pending; and to rebuild the Milvus data from the outbox events. These commands are currently run by hand.

Protection at search time. Even when the three stores are temporarily out of sync, users never see wrong data: every result returned by Milvus is checked again against PostgreSQL, and results that do not exist or are outside the permitted scope are removed before being shown (Section 3.6.3).

5.4 Application Programming Interface Design

The web application and the application server communicate entirely through a REST-style application programming interface (API). Every business API lives under the prefix `/api/v1` and names resources with plural nouns, such as `/cases`. APIs reserved for the Administrator live under `/admin`. Data is exchanged as JSON, except query images and video files, which are sent as `multipart/form-data`. State transitions with a business meaning, such as locking an account or retiring a camera, are designed as explicit actions, for example `POST /admin/cameras/{id}/retire`, so that each one is checked and logged separately.

Every error is returned in one format with a stable *error code* for the interface to handle programmatically, a *message* in Vietnamese for display, *details* such as per-field errors, and a *request identifier* for matching with the server log. HTTP status codes are used with their standard meaning: 401 when not logged in or the session has expired, 403 when the role is not allowed, 404 when the resource does not exist or is outside the

permitted scope, 409 on a conflict with the current state, 422 for invalid data and 429 when the call limit is exceeded.

To prevent overwriting, requests that update accounts, cameras and cases must include the version number the interface holds. If the data has been changed elsewhere, the server returns 409 instead of overwriting. A video upload request may carry an idempotency key, so resending it creates no additional job. Uploading a video is also asynchronous: the server only creates the job and returns 202 immediately, and the interface follows the progress through the jobs API instead of keeping the request open for the whole analysis.

For access control, no API takes the Operator's area or the case owner as a parameter. The server determines these values from the login session, so editing a request in the browser cannot widen the rights. Search and case responses only contain descriptive information, and images are fetched through separate paths. Images of search results and images of saved results use two different sets of paths, matching the two grounds for viewing images in Section 4.6.

Besides the business APIs, the server has three health-check paths (`/health/live`, `/health/ready`, `/health/storage`) for deployment and monitoring.

5.5 Security and Access Control Design

This section presents the mechanisms implementing the security requirements of Section 4.3.1 and the permission matrix of Section 4.6, following the principle of *defence in depth*: each request is checked in turn for its login session, its role and its data scope, and the database constraints are the last layer that keeps invalid data from being written.

5.5.1 Authentication and Sessions

Passwords are hashed with Argon2 and are at least 8 characters long. When the username does not exist, the system still verifies against a dummy hash and returns the same error message as for a wrong password, so that neither the response time nor the error content reveals which usernames exist; every failed login is written to the audit log.

On a successful login, the server generates a random 32-byte session token and sends

it in a cookie with the `HttpOnly` attribute, so that JavaScript on the page cannot read it, `SameSite=Lax`, so that the browser does not send the cookie with data-changing requests coming from another site, and `Secure` when deployed over HTTPS. The database only stores the SHA-256 hash of the token, so even if the session table leaks, an attacker has no valid token. On each request, the server answers 401 if the session has been revoked, is more than 12 hours old since it was issued, or has been inactive for more than 30 minutes, and revokes the session immediately if the account has been locked or deactivated or is an Operator without an area. While the user is actually active, the interface refreshes the session periodically; each refresh issues a new token whose 12-hour limit counts from the refresh and revokes the old one, so a leaked token is only valid for a short time.

5.5.2 Cross-Site Request Forgery Protection

Since the browser sends the session cookie automatically, a malicious website could try to make the user's browser send requests to the system without the user knowing (a CSRF attack). Besides the `SameSite` attribute, every data-changing request must carry an `X-CSRF-Token` header containing an anti-forgery token, computed with HMAC-SHA256 from the session token and returned to the interface at login; another website cannot read this token and therefore cannot build a valid request. The server only allows the configured origins to call the API with cookies (CORS), and every response carries security headers such as disabling content-type sniffing, forbidding the page from being framed by another site and disabling caching of API responses.

5.5.3 Access Control

Access control is enforced on the server in three layers. First, each API declares the roles allowed to call it; requests from other roles are rejected with 403 before any business processing. Second, the business layer checks the rights on each object: cameras and appearances must belong to the Operator's current area and the camera must be operational, a case must be owned by that Operator, and images are granted on one of the two grounds of Section 4.6; the area and the owner are always taken from the login session. Third, the database constraints keep invalid data from being written even if the business layer has a bug.

For objects outside the scope, the server returns 404 rather than 403: with 403, a user could infer that an object exists by trying identifiers one by one. Hiding the functions of other roles in the interface is only a convenience and is not considered an access control measure.

5.5.4 Data Protection and Abuse Limitation

The credentials in an RTSP address are separated, encrypted with the Fernet symmetric encryption scheme using a key configured through an environment variable, and only decrypted at connection time; they are never returned to the interface or written to logs. Since the connection check makes the server connect to an address entered by the user, the system only accepts IP addresses within the camera network ranges declared at deployment and always rejects loopback, link-local and multicast addresses, so that the function cannot be abused to probe other machines on the internal network (an SSRF attack).

Resource-intensive or probe-prone functions are limited in calls per minute: login 10 times per IP address-username pair, and search 30 times, video upload 10 times, camera connection check 10 times and AI check 6 times per account. When the limit is exceeded, the server returns 429 with the time to wait. The counters live in the memory of the server process, which suits the project's single-process deployment. Uploaded data is also checked before processing: query images must be JPEG or PNG, at most 10 MB and 24 megapixels, and video files at most 500 MB with a valid format signature. File names sent by users are never used as storage paths.

5.6 User Interface Design

The interface was designed and built directly in the web application, without a separate mock-up stage; screenshots of the main screens are presented in Section 6.3.

5.6.1 Screen Structure by Role

After logging in, each role is taken to its own home page and only sees the screens within its responsibility (Figure 5.8). In the figure, a bold arrow points to the page opened right

after login, a dashed frame is a dialog opened on top of the current screen, and the screens of one role are reached from each other through the left-hand menu.

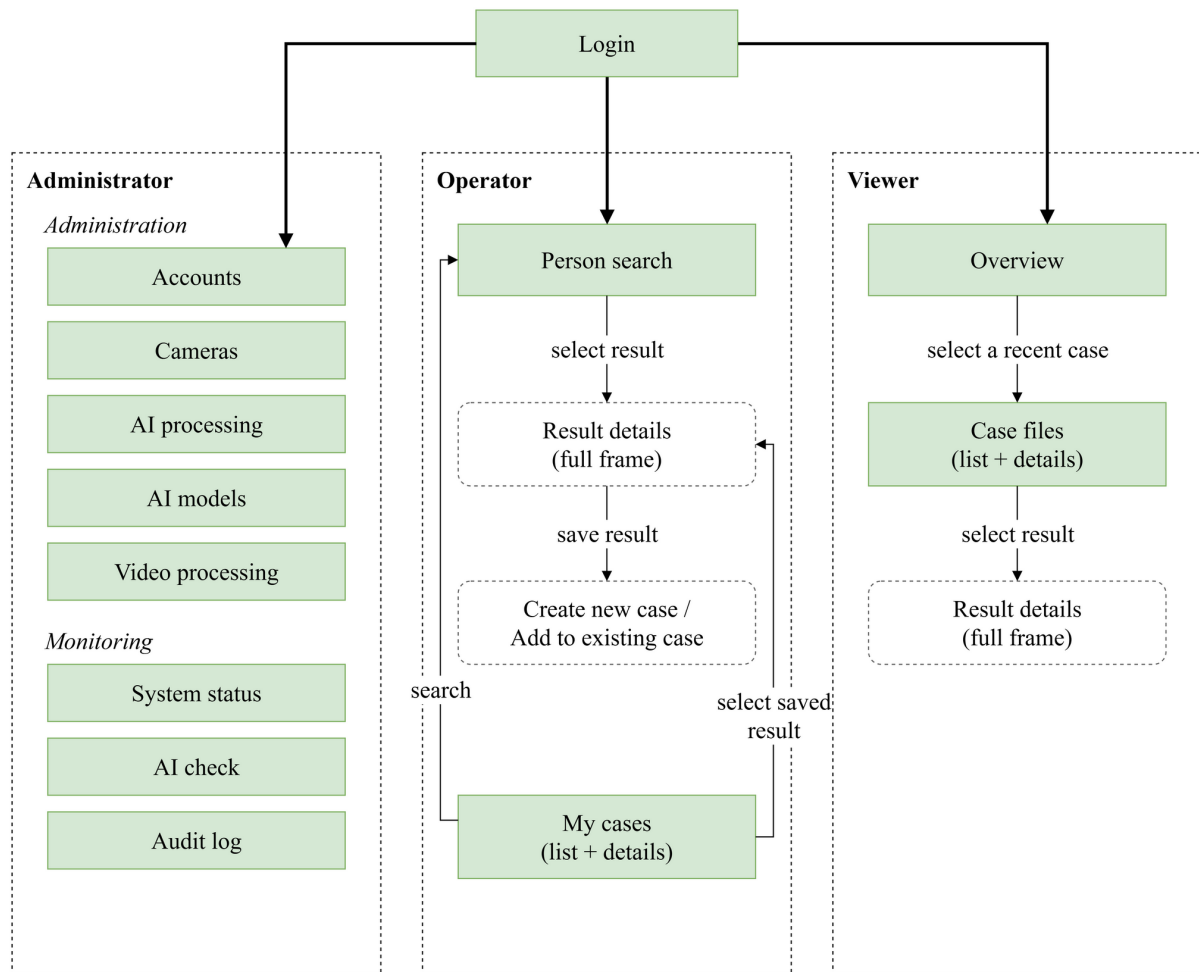


Figure 5.8: Screen flow by role

- **The Administrator** has eight screens, in an *Administration* group (accounts, cameras, AI processing, AI models, video processing) and a *Monitoring* group (system status, AI check, audit log).
- **The Operator** works mainly on the *Person search* screen, where the query form and the result list share one page so that searches can be repeated; selecting a result opens a detail dialog, from which the result is saved into a case. The *My cases* screen shows the list and the details of cases on one page.
- **The Viewer** starts from the *Overview* screen and opens cases on the *Case files* screen, which has the same layout as the Operator's case screen but is read-only.

5.6.2 Design Principles

Since search results always need the user's confirmation, the interface must first of all support visual assessment. Person images are placed in the context of the frame, and the person found is highlighted when the frame contains several people. The rank, camera and time of appearance are shown prominently. The relevance score is only shown as secondary information, and the models' internal confidence is not shown.

The interface also reduces repetitive actions. A query image can be provided by file selection, drag and drop, pasting, or dragging a result directly into the search box to search again for the same person. The result list can be sorted and filtered again without sending a new query.

When there are no results, the interface explains why and suggests how to continue. Errors are shown with the server's Vietnamese messages, and when data has been changed elsewhere, the user is asked to reload instead of overwriting. The interface is also usable on a phone about 390 pixels wide: the left-hand menu collapses into a drop-down list and wide tables scroll horizontally.

Chapter 6

System Implementation

This chapter presents the evaluation and choice of technologies and models, the implementation of the background AI worker and of the user interface, and the operation monitoring tools.

6.1 Evaluation and Choice of Technologies and Models

The options were evaluated against four common criteria: meeting the requirements of Chapter 4; running on a CPU-only machine with 16 GB of memory; being open-source software with a licence suited to a non-commercial project; and the developer's familiarity with them, to reduce risk within limited time. Table 6.1 summarises the choices, and the subsections below explain the important ones.

Table 6.1: Summary of technology and model choices

Component	Choice	Alternatives considered	Main reason
Front end	React 19, Vite, Tailwind CSS	Server-side rendering; Angular, Vue	Many interactions on one screen; front end and server separated by an API
Server	Python, Flask 3.1, SQLAlchemy, Alembic	Django, FastAPI	Same language as the AI models; small and explicit
Queue	A table in PostgreSQL	Celery with Redis or RabbitMQ	One sequential worker; few infrastructure components
Relational database	PostgreSQL 17	MySQL	CHECK constraints, partial indexes, JSONB, SKIP LOCKED
Vector storage	Milvus 2.6	pgvector, FAISS	Pre-filtering, durable storage, one collection per version
Frame storage	MinIO	Files on disk	Bucket-level permissions, hashes, shared with Milvus
Person detection	YOLO11n	YOLO11s, YOLO11m, YOLOX, Faster R-CNN	Fastest on CPU; YOLOX as a licence fallback
Tracking	ByteTrack (default), BoT-SORT	DeepSORT, StrongSORT	No appearance features per person per frame
Encoder	RaSa	Two separate models; CLIP	One shared space for the three forms of query

6.1.1 Front-End Technologies

The interface has many interactions on one screen, such as dragging and pasting a query image, sorting and filtering results without a new query, and opening dialogs over the list. These interactions suit a *single-page application*, in which the browser loads the application once and then only exchanges data through the API, better than server-side

rendering. React was chosen for its large ecosystem with ready-made libraries for routing, API calls and styling, together with Vite for development and bundling, and Tailwind CSS to keep the layout consistent and responsive on phones.

6.1.2 Server-Side Technologies

The server and the background worker are written in Python because all the project's AI models are provided for Python. Using one language lets the server load the query encoder directly and share the data access code with the background worker. Among the three common frameworks, Django is heavier than an API-only server needs, and the asynchronous advantage of FastAPI matters little because the slowest task, video analysis, has already been moved out of the server (Section 5.1.2). Flask is small and explicit, and the developer had already used it. The queue lives in PostgreSQL instead of Celery with Redis or RabbitMQ, because there is only one sequential worker, while each additional infrastructure component costs memory on the target machine.

6.1.3 Database and Storage

PostgreSQL was chosen because it supports the features the design uses directly: complex CHECK constraints, partial indexes, the JSONB type for logs and measurements, and row locking that skips locked rows (SKIP LOCKED) for safe job claiming. For vector storage, pgvector [42] was suggested at first because it only needs one database, but with an approximate index the filter is applied *after* the index scan, so a query may return fewer results than requested (Section 3.6.2). FAISS is only an in-memory library, so the application would have to handle storage, filtering and concurrent access itself. Milvus [33] was chosen because it filters before searching [34], directly meeting the area-based access control requirement; the trade-off is that Milvus needs etcd and an object store, which makes deployment more complex, although in practice all the services in Docker only use about 515 MiB of memory (Section 8.3). Frames are stored in MinIO rather than as files on disk to control access per bucket and keep a hash per object. Milvus already needs an object store, so the application shares that MinIO server in a separate bucket.

6.1.4 AI Models

Person detection. Two-stage detectors such as Faster R-CNN are too slow on a CPU, so the project chose the YOLO family. Within YOLO11, the model size directly determines the speed on a CPU (Table 6.2); even with YOLO11n, the pipeline is about seven times slower than real time (Section 7.2), so the project chose the smallest size and compensates for accuracy with a low confidence threshold combined with tracking. YOLO11 is licensed under AGPL-3.0, which suits a non-commercial project, but a commercial product would need a commercial licence; YOLOX [43], licensed under Apache-2.0, is therefore registered as a fallback.

Table 6.2: Comparison of YOLO11 sizes on the COCO validation set [22]

Model	mAP _{50–95}	CPU time (ms/image)	Parameters (millions)
YOLO11n	39.5	56.1	2.6
YOLO11s	47.0	90.0	9.4
YOLO11m	51.5	183.2	20.1

Tracking. Algorithms that add appearance features, such as DeepSORT [44] and StrongSORT [45], keep tracks better through long occlusions, but must run an extra model for every person in every frame, a heavy burden on a CPU in crowded scenes. With static cameras, the project chose ByteTrack [27] as the default and integrated BoT-SORT [28] as an alternative with ReID and camera motion compensation disabled.

Encoder. Using two separate models for image and text queries would require storing two vectors per appearance and running two models during indexing. General image-text models such as CLIP [29], including the multilingual variants considered to support Vietnamese, are weaker at separating small differences in clothing. The project chose RaSa [19] on the supervisor’s recommendation: a single model serves all three forms of query with one vector per appearance (Section 3.5.3). Its limitation is that queries are English only. The evaluation on the project’s data is presented in Section 8.2. Part II of this report studies a different choice for text queries, CLIP ViT-H/14 followed by a reasoning layer, outside the application (Chapter 9).

6.2 Background AI Worker Implementation

Section 5.2.1 presented the processing steps and life cycle of a job. This section presents how the background worker is implemented to run reliably on a CPU-only machine.

6.2.1 Model Management and Packaging

The system only loads the models declared in a *model registry*, where each entry records an identifier, version, weight file with its SHA-256 hash, preprocessing version and provenance, including the licence; Trackers also list the compatible Detectors. A model can only be used when its weight file exists, the hash matches, the licence has been approved and the model has been trial-loaded successfully; models failing these conditions are still listed on the *AI models* screen with the label *Unavailable*. Currently YOLO11n, ByteTrack, BoT-SORT and RaSa are usable, while the larger YOLO sizes and YOLOX have no weight files yet (Section 6.1.4). The hash is recomputed every time a pair of models is selected to run, so a weight file replaced on disk cannot be used silently. When the Administrator applies a new Detector-Tracker configuration, the server trial-loads all three components before saving, and each job uses exactly the configuration version current when it was created.

Each kind of model is wrapped in an *adapter* with a common interface: *open* to load the model, *process* one input, and *close* to release resources. The adapter checks the output before passing it on: the Detector only keeps the person class and valid boxes; RaSa vectors must have 256 dimensions, contain no invalid values and be normalised. As a result, adding a new model, for example enabling YOLOX, only needs the weight file and a registry entry, plus an adapter if the model is of a new kind.

6.2.2 Resource Limits and Error Handling

Since the target machine has limited memory, the background worker is limited at several levels. The number of threads of the numerical libraries is set to 2, so that the models do not take every CPU core that the application server and the data stores also need. The Detector and the image encoder run in a separate child process; if an inference does not return within 120 seconds, the child process is killed, because a call hanging inside

a native library cannot be interrupted from within the same process. Each job runs in its own process with a default total deadline of 3,600 seconds; when the job ends, the process ends with it, so the memory of the models and frames is fully reclaimed by the operating system. Frames are processed one at a time and released immediately, except for the representative frame candidates, which are already bounded.

Each error is tied to the step that caused it and a specific error code, stored with the job for display on the administration screens, and classified in advance as *transient* or *unrecoverable*. For video files, transient errors are retried after 5 seconds, doubling each time up to 300 seconds, at most three times. RTSP sessions are not retried, since that part of the video has passed; the scheduler creates a new session after 5 minutes (Section 5.2.1). When the process receives a stop signal, the running job is left *Running* so that it can be claimed again when its lease expires, instead of being marked failed immediately. A global lock in PostgreSQL guarantees that only one process executes jobs at a time, even if the background worker is accidentally started twice.

AI check. The *AI check* screen trial-runs the models on real data without creating a job: for a camera, the system reads at most 90 frames from the RTSP stream, takes 6 of them and runs the Detector, Tracker and image encoder in turn; for the search group, the system encodes a sample image and a sample description. Each check is limited to 180 seconds, and only one check runs at a time.

6.3 User Interface Implementation

6.3.1 Navigation and Communication with the Server

The web application is a single-page application built with React, with screen navigation handled by React Router. The routes are split into three groups by role: a user who is not logged in is sent to the login screen, and a user who opens a route of another role is sent to the home page of his or her own role. This guard only avoids showing unusable screens; the actual permissions are always checked by the server on every request.

Every request to the server goes through a common API layer. This layer attaches the CSRF token to every data-changing request (Section 5.5.2). The CSRF token is

only kept in the page's memory, never in browser storage. Error responses are turned into Vietnamese messages. For server-side errors, the message includes a *lookup code*, the first eight characters of the request identifier, so that the Administrator can find the matching log line (Section 6.4). When any request receives 401, the application clears the login state and returns the user to the login screen. The session is only refreshed when the user is actually active. Automatic background requests do not keep the session alive, and the time of the last user action is shared between browser tabs.

6.3.2 Rendering Person Images in the Results

The person images of the result list are rendered by the server from the full frame and the bounding box on each request (Section 5.3.3). A result card in the interface has a 3:5 aspect ratio; the crop is widened from the bounding box with the real surrounding scene until it reaches this ratio, only ever growing, never cutting into the person. Since a wide crop may contain other people, the surroundings are dimmed to 60% brightness and the person found is outlined in an accent colour. The full frame in the detail dialog has a red bounding box drawn around the person. Images are only returned when the requester has the right to see the corresponding result, with instructions not to cache them; if the frame is no longer in the store, the result card shows an *Image unavailable* icon.

6.3.3 Main Screens

The figures below show the main screens of the application by role, captured on the demonstration data (Section 7.2). The application's interface is in Vietnamese (NFR-19); the names used below are the English meaning of the screen and label names.

Operator. The *Person search* screen (Figure 6.1) has the query form on the left, with the query image, the cameras of the area, the date range and the number of results, and the ranked result list on the right. Each result shows the person image in context, the camera, the time of appearance and the relevance score; the shared area and date are only shown once at the top of the list, and the list can be filtered again by time or camera or sorted by time without sending a new query. When searching by attributes (Figure 6.2), the English description generated from the selected attributes is shown in the form, so that

users know which description the system actually searches for. Selecting a result opens the detail dialog with the full frame (Figure 6.3a), from which the result is saved into a new case, with the logged-in account as owner and the option to close the case right away (Figure 6.3b). The *My cases* screen (Figure 6.4) shows the list of cases by status and the details of the selected case; a closed case is locked and can only be reopened.

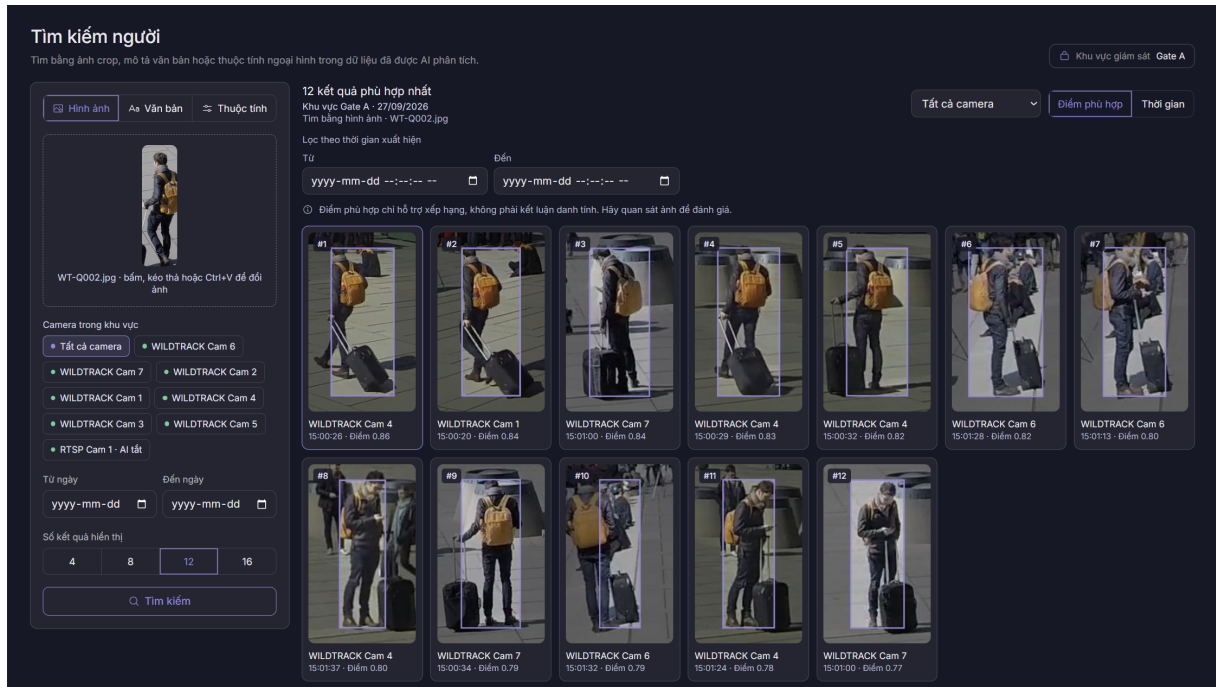


Figure 6.1: Person search by image

Viewer. The *Overview* screen (Figure 6.5) shows the number of cases by status, the number of saved results and the list of recent cases; selecting a case opens the *Case files* screen, which has the same layout as Figure 6.4 but is read-only.

Administrator. The *Cameras* screen (Figure 6.6) lists the cameras by area with their connection status, where a camera processed only through video files is shown as *Uploaded video*, their AI processing status and the actions to check the connection, edit and retire them. The *AI models* screen (Figure 6.7) lets the Administrator choose the Detector and Tracker; models that do not meet the conditions are marked *Unavailable*, and the encoder is shown as a fixed component (Section 6.2.1). The *Video processing* screen (Figure 6.8) uploads a video for a camera with a predefined sampling mode and follows the progress of each job by the number of frames and ready tracks. The two

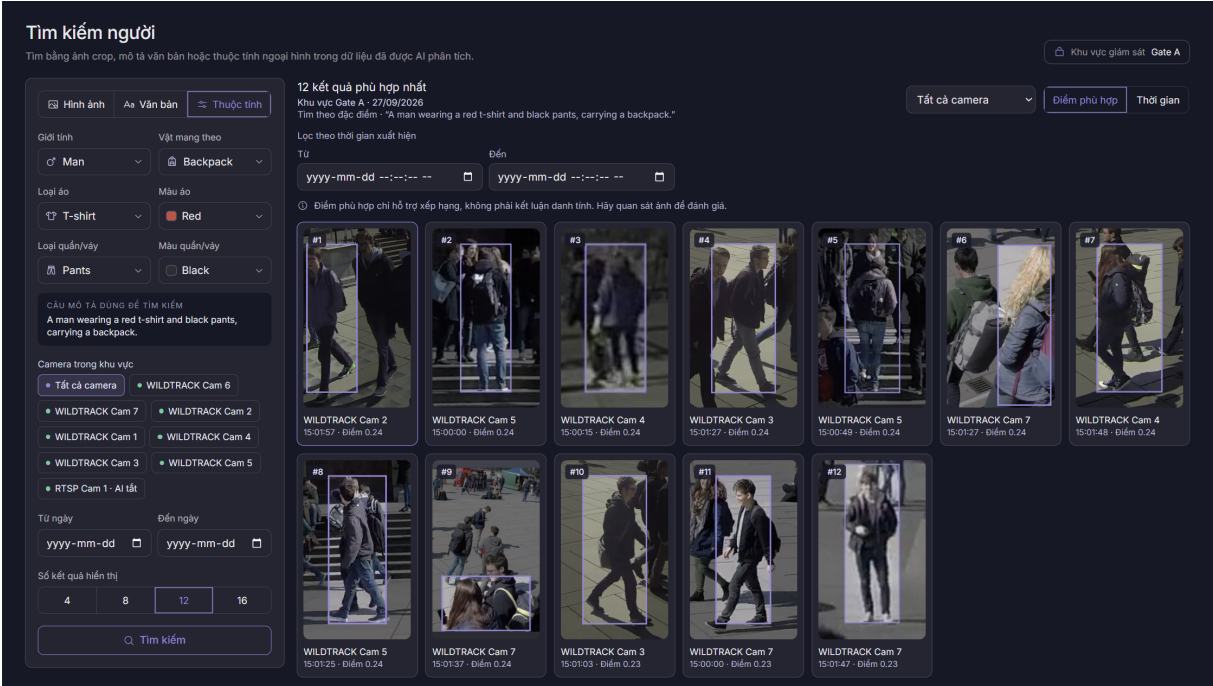
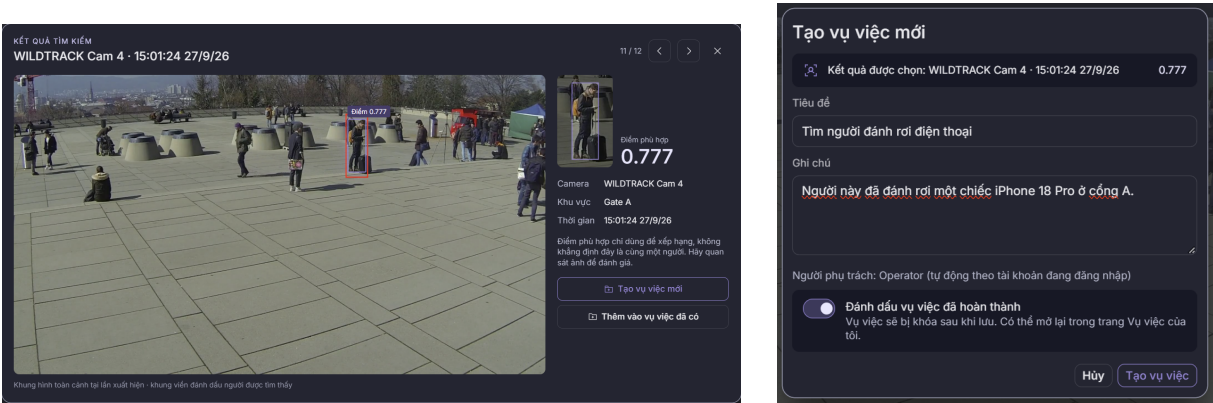


Figure 6.2: Person search by attributes



(a) Details of a result

(b) Creating a case from a result

Figure 6.3: The result detail dialog and the case creation dialog

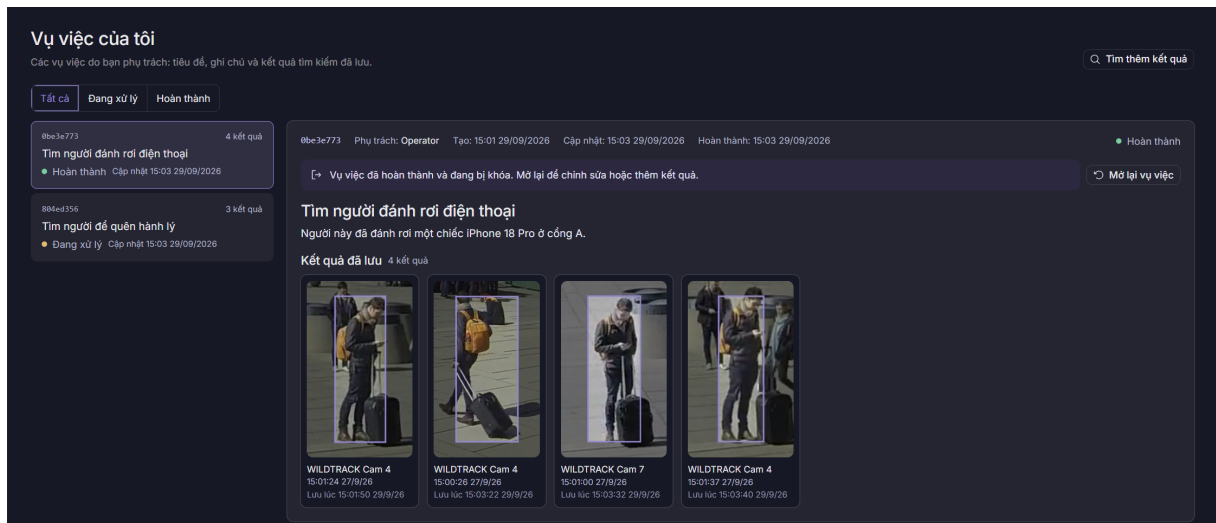


Figure 6.4: The Operator's case management screen

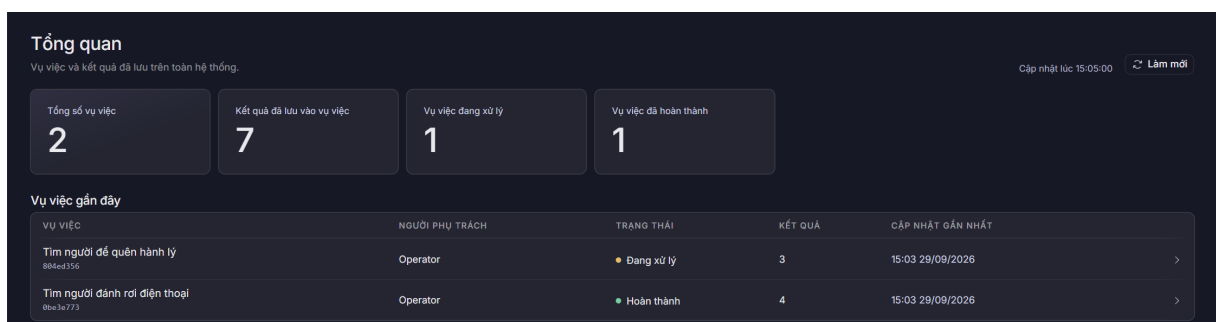
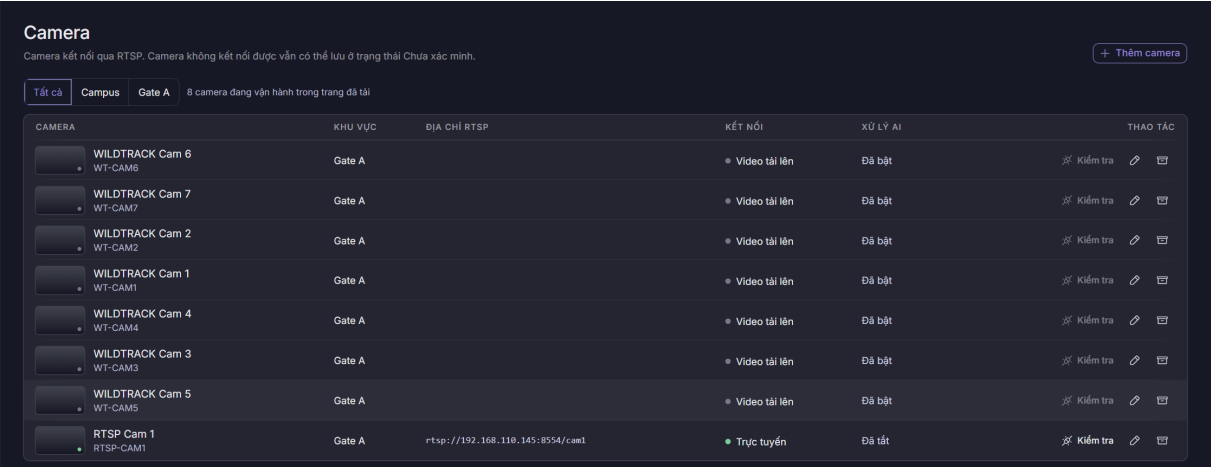


Figure 6.5: The Viewer's overview screen

monitoring screens are *System status* (Figure 6.9, Section 6.4) and *AI check* (Figure 6.10, Section 6.2.2).



Camera

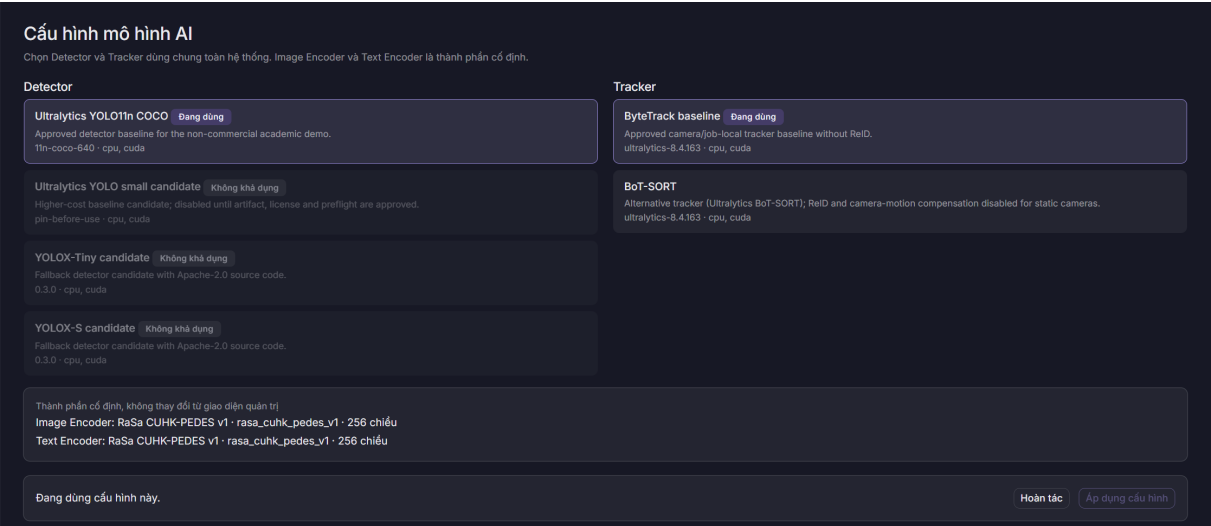
Camera kết nối qua RTSP. Camera không kết nối được vẫn có thể lưu ở trạng thái Chưa xác minh.

[+ Thêm camera](#)

Tất cả Campus Gate A 8 camera đang vận hành trong trạng đã tải

CAMERA	KHU VỰC	ĐỊA CHỈ RTSP	KẾT NỐI	XỬ LÝ AI	THAO TÁC
WILDTRACK Cam 6 WT-CAM6	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
WILDTRACK Cam 7 WT-CAM7	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
WILDTRACK Cam 2 WT-CAM2	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
WILDTRACK Cam 1 WT-CAM1	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
WILDTRACK Cam 4 WT-CAM4	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
WILDTRACK Cam 3 WT-CAM3	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
WILDTRACK Cam 5 WT-CAM5	Gate A		Video tải lên	Đã bật	Kiểm tra 🔗 🗑️
RTSP Cam 1 RTSP-CAM1	Gate A	rtsp://192.168.110.145:8554/cam1	Trực tuyến	Đã tắt	Kiểm tra 🔗 🗑️

Figure 6.6: Camera management screen



Cấu hình mô hình AI

Chọn Detector và Tracker dùng chung toàn hệ thống. Image Encoder và Text Encoder là thành phần cố định.

Detector	Tracker
Ultralytics YOLO11n COCO Đang dùng Approved detector baseline for the non-commercial academic demo. 11n-coco-840 · cpu, cuda	ByteTrack baseline Đang dùng Approved camera/job-local tracker baseline without ReID. ultralytics-8.4.163 · cpu, cuda
Ultralytics YOLO small candidate Không khả dụng Higher-cost baseline candidate; disabled until artifact, license and preflight are approved. pin-before-use · cpu, cuda	BoT-SORT Alternative tracker (Ultralytics BoT-SORT); ReID and camera-motion compensation disabled for static cameras. ultralytics-8.4.163 · cpu, cuda
YOLOX-Tiny candidate Không khả dụng Fallback detector candidate with Apache-2.0 source code. 0.3.0 · cpu, cuda	
YOLOX-S candidate Không khả dụng Fallback detector candidate with Apache-2.0 source code. 0.3.0 · cpu, cuda	

Thành phần cố định, không thay đổi từ giao diện quản trị
Image Encoder: RaSa CUHK-PEDES v1 · rasa_cuhk_pedes_v1 · 256 chiều
Text Encoder: RaSa CUHK-PEDES v1 · rasa_cuhk_pedes_v1 · 256 chiều

Đang dùng cấu hình này. Hoàn tác Áp dụng cấu hình

Figure 6.7: AI model configuration screen

6.4 Operation Monitoring

Technical logs. The application server and the background worker write logs as JSON, one event per line, automatically tagged with the correlation identifiers of the context: the request identifier for the application server; the job identifier, camera identifier and AI configuration version for the background worker. From the *lookup code* in an error message in the interface (Section 6.3.1) or the identifier of a failed job, all related log

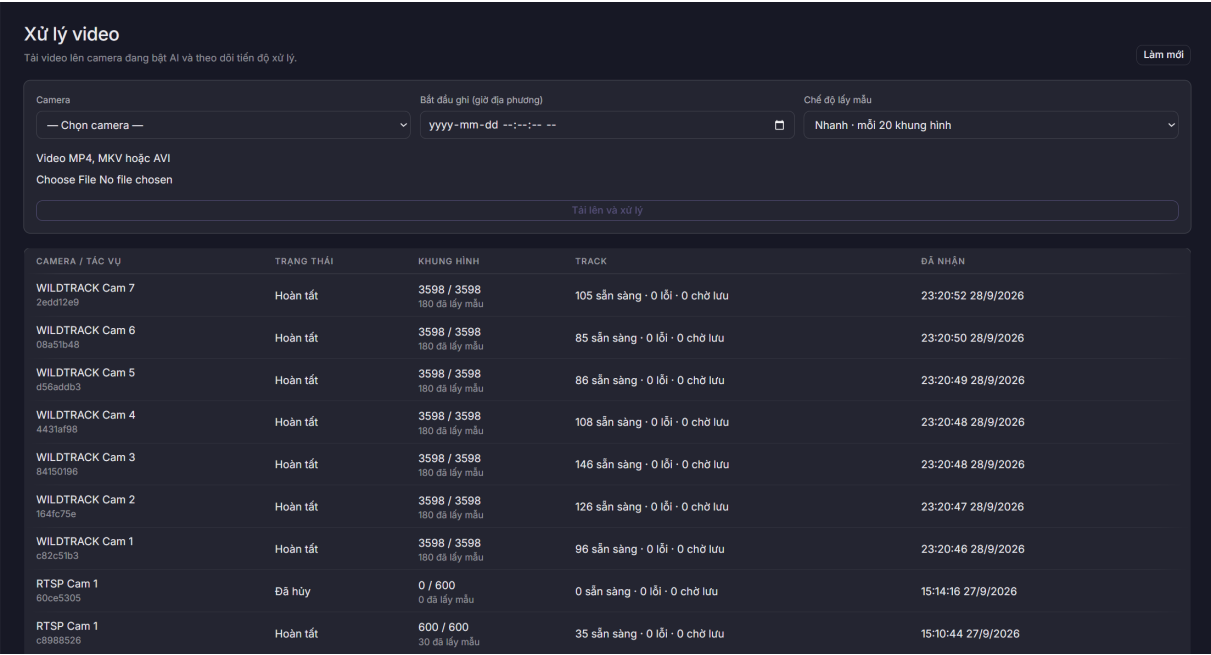


Figure 6.8: Uploaded video processing screen

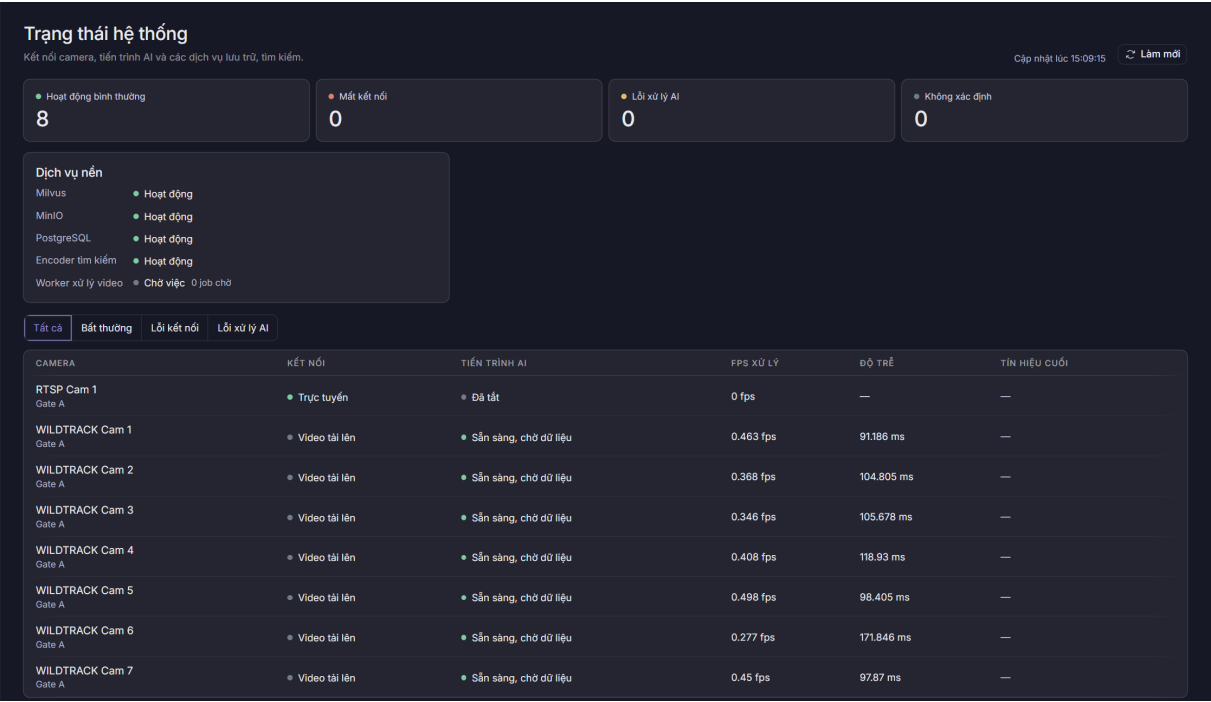


Figure 6.9: System status screen

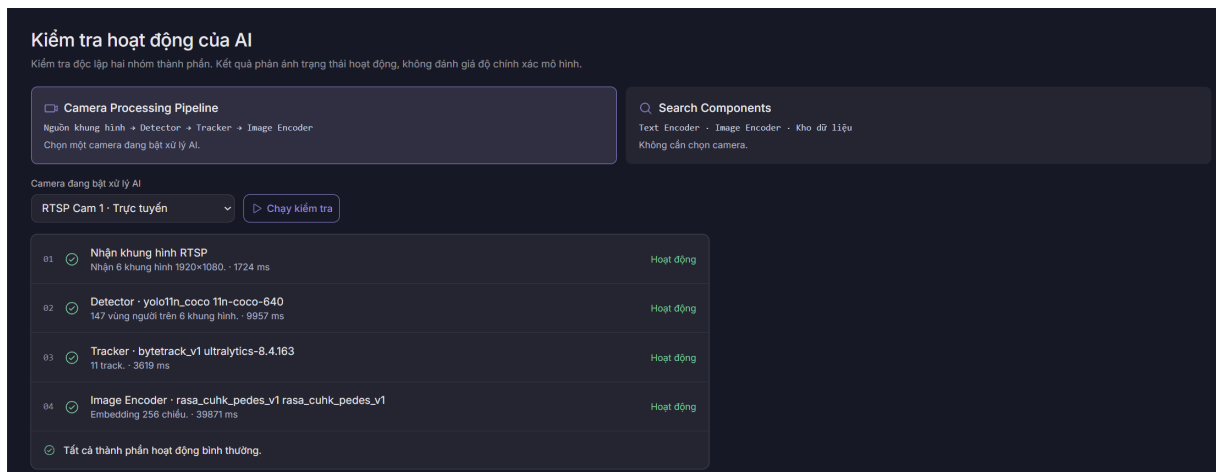


Figure 6.10: AI check screen

lines can be filtered out. Before writing, a filter replaces the values of sensitive fields, such as passwords, session tokens, RTSP addresses, feature vectors, images or query sentences, with [REDACTED], and masks credentials, vector-like number sequences or encrypted data embedded in free text.

Audit log. Important user actions and technical events of operational significance are stored in PostgreSQL as 32 event types, including logins and session expiry, management of accounts, cameras and AI configuration, processing errors and AI check errors, jobs, cases and data maintenance tools. The Administrator consults them on the *Audit log* screen, filtering by date range, event type, actor and outcome.

Metrics and system status. While running, each job updates its metrics about every 2 seconds, such as the frame processing rate, the average time of the Detector, Tracker and encoder, memory and CPU usage; the background worker sends a heartbeat every 10 seconds and is considered down if it has been silent for more than 45 seconds. The *System status* screen (Figure 6.9) aggregates this information: for each camera, its RTSP connection status, AI processing status, processing rate and latency, and the number of failed jobs in 24 hours, with the camera placed in one of four groups (normal, offline, AI processing error, unknown); for the components, the health of the three data stores, the search encoder and the job queue. Each piece of information is shown with its age, so that an old number is not mistaken for the current state.

Chapter 7

System Deployment

This chapter presents the deployment environment and procedure of the system, together with the test data and the simulation of the camera system.

7.1 Environment and Deployment Procedure

Environment. The whole system is deployed on a laptop with an Intel Core i5-11300H processor (4 cores, 8 threads, 3.1 GHz), 16 GB of memory and no discrete graphics card, running Windows 11. The infrastructure services run in Docker Compose: PostgreSQL 17, Milvus 2.6 in Standalone mode with etcd, MinIO shared by Milvus and by the application's frames, and MediaMTX to stream the simulated cameras. The Flask application server, the background worker and the React web application run directly on the machine. The AI configuration is YOLO11n with a confidence threshold of 0.1, ByteTrack, the RaSa encoder and a sampling interval of $N = 20$, all running on the CPU.

Start-up procedure. The components are started in order: infrastructure services, simulated camera streams, application server, background worker, then the web application. Since the query encoder is only loaded at the first search, a warm-up search is run before use (Section 8.3).

Backup and restore. The backup of the demonstration data of the three stores is about 510 MB in 1,525 files. The restore procedure was rehearsed with 1,555 tracks, 10 of whose frames had been deleted on purpose: the restore finished in 28.8 seconds (PostgreSQL 1.8 seconds; frames 6.2 seconds, with all 10 of 10 files re-uploaded; rebuilding

the Milvus vectors 8.8 seconds; reconciling the three stores 12 seconds). The first rehearsal, when vectors were still written one by one, took 1,485 seconds and had one vector time out, which led to the switch to batch writes. After each demonstration rehearsal, the data is reset to its baseline in about 70 seconds.

7.2 Test Data and Simulated Cameras

Dataset. The project uses the WILDTRACK dataset [46], made of the videos of seven static 1920×1080 cameras observing one crowded outdoor pedestrian area with overlapping fields of view, together with person position labels on a set of frames. Each video is treated as one camera, and all seven cameras are assigned to the same surveillance area.

Simulated cameras. The seven videos are streamed as seven RTSP streams through MediaMTX as in Section 3.7.3. On the target machine, however, the analysis is about seven times slower than real time, so the streaming server drops frames and most of the stream content is not analysed. The RTSP streams are therefore used to demonstrate and verify camera ingestion (Section 8.3), while the demonstration data is indexed from the video files themselves through the video upload function, which goes through the same pipeline.

Preparing the demonstration data. Indexing is done by a tool that calls the system’s API as an Administrator: it cuts the segment of video to process, uploads it for each camera with the same time origin for all seven cameras, and follows the jobs until they finish. The demonstration data covers the first 120 seconds of each video: the 0-60 second segment produced 736 tracks, and the 60-120 second segment was processed by 7 jobs in 55 minutes, adding 752 tracks without errors. In total there are 1,488 ready tracks, distributed as in Table 7.1. Together with 35 tracks from one RTSP processing session, the demonstration data has 1,523 tracks and one sample case with 3 results.

Table 7.1: Number of tracks in the demonstration data per camera (first 120 seconds of each video)

Camera	C1	C2	C3	C4	C5	C6	C7	Total
Tracks	208	254	308	188	182	159	189	1,488

Chapter 8

Testing and Evaluation

This chapter presents the results of software testing, the evaluation of search quality, and the measurements of processing speed, RTSP handling and resource use of the application.

8.1 Software Testing

The system was tested at five levels, from individual functions to the whole demonstration script; the results are summarised in Table 8.1. The server-side code is tested with pytest and the interface with automated Playwright tests in the Edge browser. Integration and end-to-end tests only run on a disposable database: the test suite refuses to run if the database name does not carry the suffix reserved for tests, so that the demonstration data is never written by mistake.

Table 8.1: Test results by level

Level	Scope	Result
Unit	Each component of the server and the background worker, including API contract, security and fault injection tests	713 passed
Integration	With real PostgreSQL, Milvus, MinIO and MediaMTX: store connections; business services; storage adapters; database migrations and schema; RTSP streams	37 passed, 1 skipped (fault injection needs a service to be fully stopped)
End-to-end (E2E)	The whole flow through the real API with the real models: video upload, indexing, search, saving a case, viewing as a Viewer	2 passed; the AI flow with a 10-second video took 217 seconds (model loading 44 seconds, processing 154 seconds, first text search 16 seconds)
User interface	20 screens at three widths, 390, 768 and 1440 pixels	No overflowing elements, no browser or HTTP errors; 3 bugs found and fixed
Demonstration script	The steps of the three roles, from adding an RTSP camera to viewing a case file, run automatically	Twice in a row 15/15 steps passed, 85-88 seconds of interaction

The three interface bugs found were: a model could not be selected on the AI models screen, the first search reported a lost connection because the timeout was shorter than the time needed to load the encoder, and the AI processing status was shown wrongly on the camera screen.

Access control and security tests. These tests belong to the unit tests and cover the permission matrix of Section 4.6:

- Every API must declare an access rule; one test calls every API with every role and without login, and checks that only the permitted roles are not rejected.
- An Operator opening another Operator's case receives 404; a Viewer can only read cases; appearances outside the area and images belonging to another case are

blocked. The end-to-end tests further confirm that an Operator in another area can neither find nor open the frame of an appearance, and that a Viewer cannot call the search function.

- Every write requires the anti-forgery token. Login is rate-limited per IP address and username, and search per account. The security headers are present in both successful and error responses, and only the configured origins may call the API with cookies.

8.2 Search Quality Evaluation

Method. The system’s pipeline was run on the labelled frame sets of the seven WILD-TRACK cameras. Each track was labelled with the majority person by matching its boxes with the dataset labels, for evaluation only. The evaluation set has 6 queries, each in three forms: a cropped image, an English description and a set of attributes. The observation used as the query image is removed from the gallery. $Recall@k$ is the fraction of queries with at least one correct result in the top k , with $k \in \{4, 8, 12, 16\}$, and MRR is the mean reciprocal rank of the first correct result. Run 1 used a confidence threshold of 0.25 and a gallery of 1,552 tracks, for all three forms. Run 2 used the current threshold of 0.1, a gallery of 1,526 tracks and an extended attribute set, but only re-evaluated attribute search. Search by image and by text at threshold 0.1 has *not been measured*.

Table 8.2: Search quality results (6 queries)

Form	Conditions	R@4	R@8	R@12	R@16	MRR
By image	Run 1: threshold 0.25; 1,552 tracks	0.833	1.0	1.0	1.0	0.867
By text	Run 1: threshold 0.25; 1,552 tracks	0	0	0	0	0.01
By attributes	Run 1: threshold 0.25; 1,552 tracks	0	0	0	0	0.005
By attributes	Run 2: threshold 0.1; 1,526 tracks; extended attribute set	0	0	0	0	0.0045

Person detection. In run 1, detection reached a precision of 0.58 and a recall of 0.54; in run 2, after lowering the confidence threshold to 0.1, precision was 0.43 and recall 0.64. The two runs differ in both threshold and attribute labels, so they are only compared qualitatively: lowering the threshold misses fewer people but produces more false detections, most of which are removed by tracking (Section 3.2.5).

Discussion. Search by image performs well: in run 1, every query has a correct result in the top 8. Search by text and by attributes, in contrast, brings no correct result into the top 16; in run 2, the first correct result of the attribute queries only appears at ranks 87, 241 and 365, and the extended attribute set does not improve the results. The main cause is the domain gap between CUHK-PEDES, on which RaSa was trained, and the person images cropped automatically from the full frames of WILDTRACK (Section 3.5.3). Search by image is therefore the main form of the system, while search by text and by attributes plays a supporting role and needs careful visual assessment by the user. This weakness of text search is the starting point of the research in Part II (Chapter 9).

Qualitative check on the demonstration data. On the demonstration data (Section 7.2), five query images were tried with the top 8 results and assessed visually: three queries had 8/8 correct results, one 7/8 and one 4/8; the correct results lay on 4 to 5 different cameras. This is only a qualitative check on a dataset different from the evaluation set and cannot be compared with the Recall values of Table 8.2.

8.3 Performance and Camera Stream Handling

Sampling interval. Table 8.3 compares the two sampling presets on the same 10-second video (600 frames) with the current configuration, each preset run 3 times, interleaved, on the target machine. The first run of $N = 10$ had to load the models (taking 178.5 seconds), so the time of $N = 10$ is the average of the other two runs. Other applications on the machine were not closed during the measurement, so the time varied by about ± 12 seconds between runs. CPU and memory were sampled every 0.25 seconds, so they are lower bounds of the actual values.

Table 8.3: Comparison of the two sampling intervals on a 10-second video (confidence threshold 0.1)

Metric	$N = 10$	$N = 20$
Processing time with the models loaded	149.5 s	132.6 s
CPU time (main process and model processes)	about 212 s	about 160 s
Peak memory of the main process	695 MiB	576 MiB
Peak memory of the model processes	4,284 MiB	4,267 MiB
Tracks	44	36
Short tracks (under 2 seconds)	22	24
Size of the representative images	10.2 MB	6.9 MB

With $N = 20$, CPU time drops by about a quarter and the image size by about a third, while the number of short tracks, a sign of broken tracks, hardly changes. The processing time only drops by about 9%, because most of the time is spent in the image encoder: about 1.85 seconds per track on average, while the Detector only takes about 0.16 to 0.23 seconds per sampled frame. Halving the number of frames through the Detector therefore saves only a small part, and the processing time depends mainly on the number of people appearing in the video. The benchmark video is more crowded than the demonstration data: 36 tracks in 10 seconds, against 752 tracks in 420 seconds of video from the seven cameras in the 60-120 second segment (Section 7.2). The ratio of processing time to video length is therefore higher here than the roughly sevenfold slowdown measured when indexing the demonstration data. The memory of the model processes hardly differs between the two presets, since it mostly holds the model weights. Since a CPU-only machine is already many times slower than real time, $N = 20$ is kept as the default (Section 3.4).

RTSP stream handling. The ingestion of simulated RTSP streams was verified in three situations, with a confidence threshold of 0.25, so the track counts are only indicative: a normal session processed 300 frames, produced 29 tracks and a text search returned 8 results, in 136 seconds; when the streaming source was cut for 5 seconds mid-session, the background worker reconnected once and continued, completing 600 frames with 38 tracks in 140 seconds; when the Administrator disabled AI processing

mid-session, the job became *Cancelled* and left no incomplete track. With the current configuration, in the rehearsals, a newly added RTSP camera was *Online* and the background worker created a processing session 13 to 24 seconds after AI processing was enabled.

Search latency. Latency was measured on the client side by calling the search API with $k = 8$ on the 1,523-track demonstration data, 30 calls per form after one warm-up call, interleaved and 2.5 seconds apart, with the background worker idle. The results are in Table 8.4; no call was rejected by the rate limit.

Table 8.4: Search latency through the API (30 calls per form, $k = 8$)

Form	Median	95th percentile	Maximum
By image	1,269 ms	1,976 ms	2,289 ms
By text	106 ms	142 ms	151 ms
By attributes	103 ms	136 ms	136 ms

Search by image is more than ten times slower than the other two forms, because the query image must go through RaSa’s image encoder on the CPU, while a description only goes through the text encoder. When operating through the interface in the rehearsals, the first search after the server started took about 31 seconds, mostly to load the encoder. Later searches took about 8 seconds, because they were measured while the background worker was running an RTSP session and include the interface rendering time. Without warm-up and with a busy background worker, the first search took about 97 seconds. The deployment procedure therefore includes a warm-up search (Section 7.1).

Storage and memory. Each track takes about 358 KB on average for its full JPEG frame and 1 KB for its 256-dimensional vector; the relational database is about 24 MB. Memory was measured for each component with the whole system running as in the demonstration, taking the maximum in each state. When idle and after the search warm-up, the services in Docker use 515 MiB (828 MiB including the Docker virtual machine), the application server 1,978 MiB (153 MiB before loading the query encoder), the background worker 338 MiB, the front-end development server 318 MiB and the FFmpeg processes streaming the cameras 286 MiB, about 3.7 GiB in total. While searching, the

application server rises to 2,037 MiB. While processing a job, the analysis pipeline uses up to 4,772 MiB more, bringing the system’s total to about 8.0 GiB.

The memory figures come with four conditions. First, to avoid writing into the demonstration data, the processing state was measured with a tool that runs exactly the system’s analysis pipeline on a 10-second video in place of the background worker. Second, the peak of 4,772 MiB adds the peaks of the main process and the model processes, which may not occur at the same time, so it is an upper bound. Third, on Windows, the measured memory is the part currently in RAM, which the operating system may trim when RAM runs low. Fourth, other applications were open during the measurement, so the RAM used by the whole machine rose from 13.2 GiB when idle to 13.9 GiB while searching and 15.2 GiB while processing, out of 15.7 GiB. The main reason is that the application server and the analysis pipeline each load their own models: about 1.8 GiB for the query encoder and about 4.2 GiB for the Detector and the image encoder. Unneeded applications should therefore be closed during a demonstration.

Part II

Semantic Reasoning for Text-Based Person Search

Chapter 9

Semantic Reasoning: Motivation and Overview

This chapter opens Part II, the research on the search block. It states the problem and the contributions, reads the person search pipeline as a template of replaceable blocks, argues for improving the search block with a reasoning layer inspired by ZARA, and fills each block with a concrete model.

9.1 Motivation and Contributions

9.1.1 Problem

Given a free-text description of a person, such as “*a teenage woman, with short hair, wearing a short-sleeve red top and shorts, with a bag*”, and a gallery of person crops extracted from surveillance video, the task is to return the crops that show that person. This is text-based person search: a retrieval problem whose query and gallery live in different modalities, over imagery that is small, motion-blurred, badly lit and taken from arbitrary viewpoints.

9.1.2 Why a reasoning layer

A dual-encoder model such as CLIP answers this problem in one shot: encode the sentence, encode every crop, sort by cosine similarity. This is fast and is the right first stage, but it has two properties that limit what it can do.

- It gives one opaque number per crop. A cosine similarity cannot be interrogated: when the right person is ranked 37th, nothing in the score says whether the top was read in the wrong colour, the sleeves were wrong, or the gender was read wrong, so nothing can be corrected.
- It makes no use of the track. The gallery is not a set of independent images: tracking groups crops into tracks, so the same person is observed several times under different blur, lighting and pose. CLIP scores each crop alone and discards that redundancy.

An attribute-level representation addresses both: it is readable, and it can be aggregated across the frames of a track. The difficulty is that the attributes themselves come from a small VLM and are noisy. This is the subject of Section 10.1.4, and it is the reason the aggregation is done by an LLM that reasons over evidence rather than by a majority vote.

9.1.3 Contributions

1. A statement of where the improvement should go: the pipeline is read as a template of replaceable blocks, the requirement of each block is stated, and the search block is chosen because it is the one that must both generalise semantically and tolerate the errors of every block upstream (Section 9.2.2).
2. An attribute-centric index over tracked person crops. Each crop has 18 attributes read by Qwen2.5-VL-3B and image-quality statistics for each body region (sharpness, contrast, brightness, saturation), which let a later stage reason about *why* two readings disagree.
3. An adaptation of the ZARA evidence-and-reasoning pattern from motion-sensor activity recognition to image-text person retrieval (Section 9.2.3).
4. A four-stage multi-agent funnel (query analyst, feature selector, evidence reader, ranker) run over two rounds with different cut semantics, and the prompt mechanisms that make it work: a citation rule, a reliability-not-direction ranking rule, a rarity table and a round-dependent coverage floor.

5. An evaluation protocol and tooling that reconstruct every ranking from the saved model replies, so that results survive a corrupted record file, and that separate retrieval failures from reasoning failures by reporting the pool ceiling next to every recall number.
6. A measured result on 100 identities, decomposed into what the layer rescues and what it costs, with an error analysis of both.

9.2 The Pipeline, and Where to Improve It

9.2.1 The pipeline in the abstract

Figure 9.1 shows the system with every block named by what it *does* rather than by what implements it. Seen this way, a person search system has five replaceable parts and two phases.

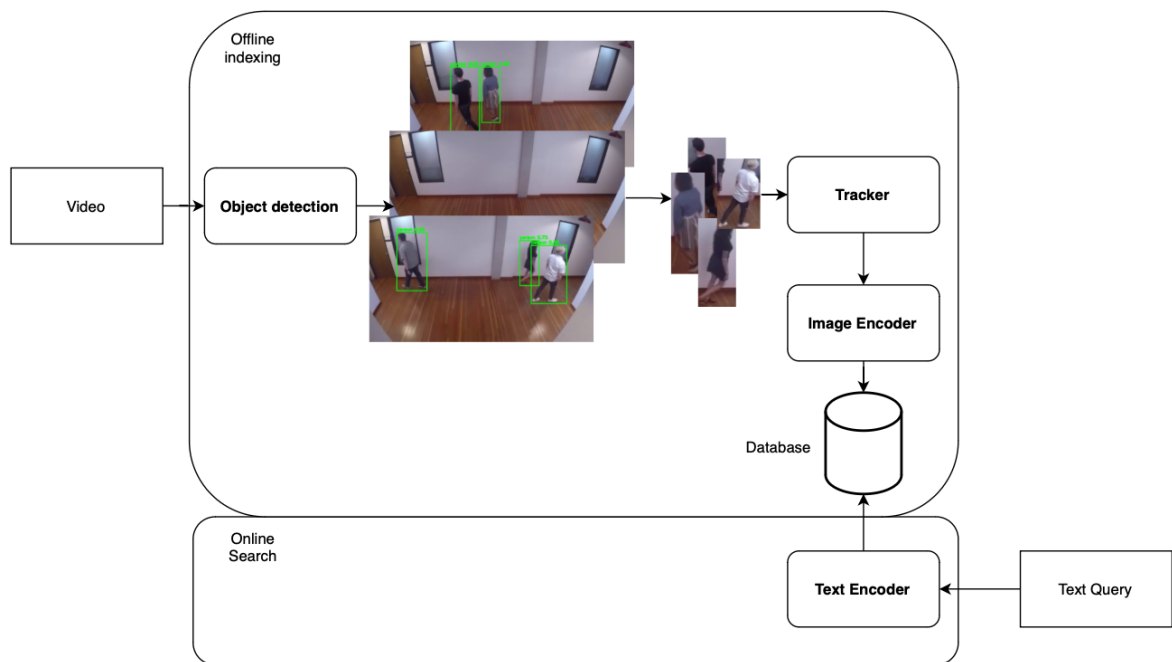


Figure 9.1: The pipeline as a template. Offline: video → object detection → tracker → image encoder → database. Online: text query → text encoder → similarity search against the same database. Every box is a slot for any detector, tracker or encoder.

This view makes the next question precise: with a fixed budget, which block should the work go into?

9.2.2 Possible directions for improvement

Each block has a requirement it must meet and a way it fails. Table 9.1 is the comparison we used to choose.

Direction	Main requirement	Our consideration
Object detection	Detect people correctly	Errors propagate downstream.
Tracking	Maintain identity across frames	Tracker errors can contaminate a whole track.
Image/text encoders and online search	Match the query to visual appearances	Needs semantic generalisation and robustness to upstream errors, at the image quality of deployed cameras.

Table 9.1: The three places the pipeline can be improved, what each has to do, and what that costs.

The first two directions are well studied, and improving them mostly means substituting a better model into a slot. Their failures are also *upstream*, so they reach the search block whatever is done about them: a missed detection is a crop that never exists, and a tracker that merges two people gives the search block a track that contains two people.

The search block is different. It is the only block that must do three things at once: generalise semantically from a sentence to an appearance, tolerate what the detector and the tracker got wrong, and do both on crops from a deployed camera rather than on photographs. It is also where the system’s answer is decided. For these reasons, the work of this part is placed in the search block.

Why an LLM-based search block. Once the block is chosen, the question is what to put in it. A dual-encoder such as CLIP gives one opaque number per crop: it cannot be interrogated when it is wrong, and it scores each crop alone, so the redundancy of a track is discarded. Fine-tuning the encoder addresses neither problem: it changes the number, but there is still only one number and nothing explains it. The block needs a representation that can be *read*, namely attributes with the evidence behind them, and a component able to weigh several partially contradictory readings of one person. This is a

reasoning problem over text, which suits an LLM, and ZARA made the same observation in another domain.

9.2.3 ZARA: related work and inspiration

The closest prior work to this system is ZARA [47]. The reasoning pattern comes from ZARA; the domain and the evidence are those of this project.

The key idea of ZARA is that an LLM reasons poorly over raw numerical signals and well over *textual* evidence. Instead of feeding the model the signal, ZARA first converts statistical patterns of the reference data into a textual knowledge base, and at inference retrieves the discriminative evidence relevant to the candidate classes.

We keep this structure and change the domain. In ZARA the statistics come from motion signals; here they are human-readable person attributes extracted offline from images by Qwen-VL. In ZARA several sensors give complementary views of one activity; here the frames of one track give complementary views of one person, with the caveat that a tracker can group two different people. This failure mode does not exist for ZARA’s sensors, and Section 9.2.2 has already identified it as coming from upstream.

Unlike conventional statistical aggregation, we do not treat the offline attribute statistics as ground truth. Qwen-VL makes mistakes, especially with the image quality and viewpoints of real surveillance cameras, and the tracker adds its own. The statistics are therefore given to an LLM as *evidence to be reasoned over*, and the aggregated track-level reading is used to re-rank the candidates against the query:

ZARA	motion signals → statistical features → multi-sensor evidence → LLM reasoning
Ours	person images → human-readable attributes → multi-frame track evidence → LLM evidence aggregation → candidate re-ranking

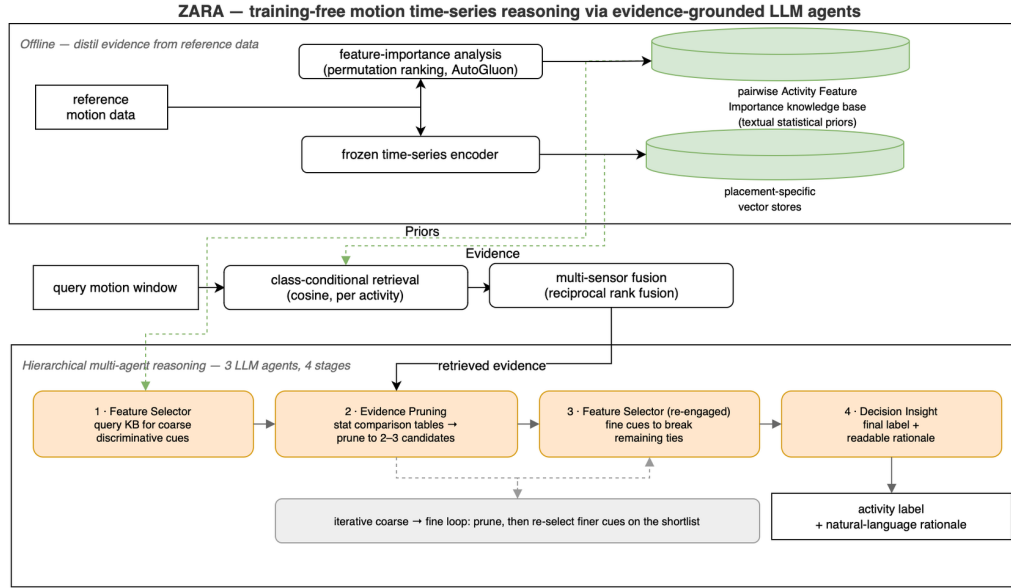


Figure 9.2: ZARA’s retrieval of discriminative evidence for candidate classes, the pattern this project adapts.

One more element is taken directly from ZARA: the coarse-to-fine iteration. ZARA prunes before it decides, and the funnel of Section 10.2 keeps this structure, with a coarse round over the whole pool followed by a finer round over the candidates that survive.

9.3 System Overview

9.3.1 Filling the slots

Table 9.2 puts a concrete model into each block of Figure 9.1. The detector and the tracker are current, well established defaults, chosen so that the upstream blocks are not the bottleneck and not studied further in this part. The work is in the search block: CLIP is kept as the *base encoder* that builds the candidate pool, and an LLM-based reasoning layer is added after it.

Block	Choice	
Object detection	YOLO11n	[48]
Tracker	ByteTrack	[27]
Image/text encoder	CLIP ViT-H/14 (laion2B), as the base encoder	[49]
Online search	LLM-based reasoning layer over the CLIP pool	
Database, similarity search	Milvus	[50]
VLM (attribute describer)	Qwen2.5-VL-3B-Instruct	[51]
LLM (reasoning)	Gemini Flash-Lite 3.5	[52]

Table 9.2: One model per slot. None of them is fine-tuned on this dataset.

9.3.2 Offline indexing

Figure 9.3 shows the indexing phase with the choices of Table 9.2. Detection (YOLO11n) and tracking (ByteTrack) are the unchanged blocks of the template; the new element is that each tracked crop now passes through *two* describers. CLIP produces the embedding used for retrieval, and Qwen2.5-VL-3B produces the attribute record used for reasoning. Both are written to the database, keyed by (video, track, frame), so a crop returned by a similarity search can be joined with its attributes and with every other frame of its track in one lookup.

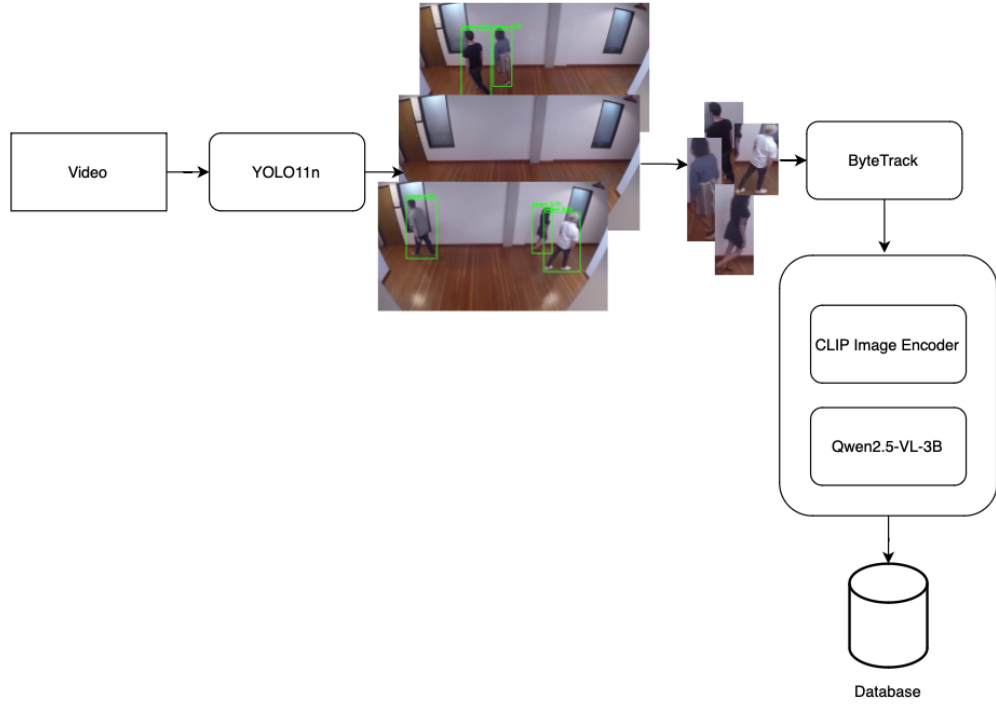


Figure 9.3: Extended offline indexing. The Qwen2.5-VL-3B branch turns each crop into 18 readable attributes alongside its embedding, which the reasoning stages need.

9.3.3 Online search

Figure 9.4 shows the search path. CLIP retrieval is kept as it was, but its output is no longer the answer: it becomes an *intermediate candidate pool*. The multi-agent semantic reasoning block takes this pool together with the original text and returns the final top- K .

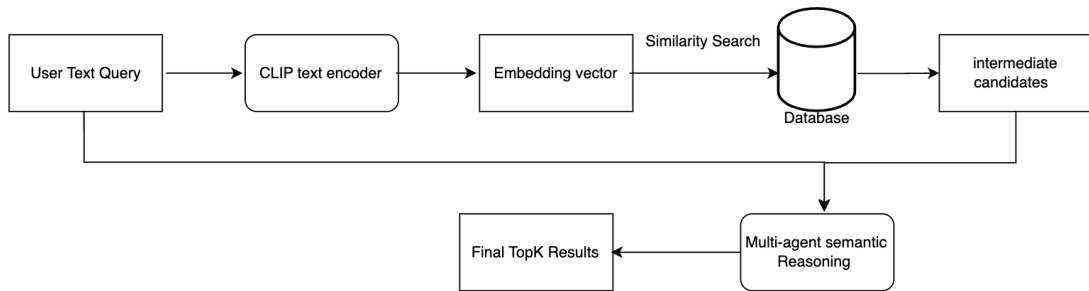


Figure 9.4: Online search with the reasoning layer. CLIP produces the candidates and the funnel orders them. The user’s text reaches the reasoning block directly and is not summarised by the retrieval stage.

This arrangement has a hard ceiling, reported throughout: the reasoning layer can

only reorder what retrieval gave it. If the right person is not in the pool, no reasoning can return them. Section 11.1.5 therefore reports this ceiling, cap@75 , next to every recall number.

Chapter 10

Attribute Index and Multi-Agent Reasoning

This chapter describes the attribute index built offline with a vision-language model, and the multi-agent funnel that re-ranks the candidates of CLIP retrieval from that evidence.

10.1 The Attribute Index

10.1.1 Schema

Each described crop carries the 18 attributes of Table 10.1. The schema is deliberately centred on garments, since these are what a witness usually describes. `carrying_canon` is list-valued (a person can carry a backpack *and* a handbag); every other attribute is single-valued per crop, although the same attribute read across a track often produces several values, which is the signal the reasoning stages use.

Attribute	Region	Meaning
gender	whole	apparent gender
age_group	head	apparent age band
build	whole	apparent build
height_class	whole	apparent height, words only
hair_color	head	dominant hair colour
hair_style	head	visible hairstyle
upper_inner_type	upper	the top worn under any outer layer
upper_inner_color	upper	the colour of that top
upper_outer_type	upper	the outermost upper garment (jacket, coat)
upper_outer_color	upper	the colour of that outer garment
sleeves	upper	sleeve length
pattern	upper	visible pattern on the upper body
lower_type	lower	lower-body garment
lower_color	lower	its colour
lower_length	lower	how far it reaches
shoes_type	feet	footwear type
shoes_color	feet	footwear colour
carrying_canon	whole	non-clothing objects carried (LIST-valued)

Table 10.1: The attribute schema, and the visual band each attribute is judged from. The region column is a manual routing table: a wrong entry is invisible at run time, because the evidence line still reads correctly but quotes the sharpness of the wrong part of the body.

10.1.2 Visual context

Attributes alone cannot explain a disagreement. If one frame reads the top as red and another as black, the useful question is whether the two frames were taken under conditions that make this confusion likely. Every crop therefore also carries numeric quality statistics, computed per band:

Band	Extent of the crop
whole	the entire crop
head	the first 15%
upper	the next 30%
lower	the next 40%
feet	the last 15%

Each band carries four numbers: sharpness, contrast, brightness and saturation. Because `upper_inner_color` is routed to the upper band, a note about a colour disagreement can cite the saturation of the upper body in the frames that read red and in those that read black, giving a mechanism with its figure rather than a guess.

10.1.3 Three different counts

The same attribute value can be counted in three ways. Confusing them is the easiest mistake in this design, so the implementation keeps them as three separate functions:

- The *global prior* is the value’s share over the *whole index*, with lists *split*: a reading of [red, white] plus a reading of red gives red 66% and white 33% over three observations. This is the base rate against which rarity is judged.
- The *track distribution* is the set of readings of one track, kept *whole*: the same two readings give “red+white 50%, red 50%”. This is what the evidence reader quotes.
- *Membership* tells whether a query value red matches a frame that read [red, white], which it does.

The local (pool) prior adds one more rule: it is computed over *deduplicated tracks*, not over pool crops. Two pool crops from the same five-frame track give a five-frame statistic, not a ten-frame one. Without this rule, a person retrieved many times would dominate the pool statistic simply because the retriever returned that person several times.

10.1.4 The image-quality problem, and why voting does not work

Table 10.2 shows one real track, described over ten frames. It is the motivating example for Section 10.2.

Attribute	Across the 10 described frames
gender	female 80%, male 20%
age_group	young adult 100%
build	slim 100%
height_class	average height 80%, average 20%
hair_color	black 40%, brown 30%, blonde 30% (split)
hair_style	short 50%, straight 30%, long+straight 20% (split)
upper_outer_type	hoodie 20%, not visible 80% (split)
upper_outer_color	black 20%, not visible 80% (split)
upper_inner_type	sweater 70%, tank top 10%, not visible 20%
upper_inner_color	gray 40%, black 40%, not visible 20% (tied, the aggregate is arbitrary)

Table 10.2: VLM noise on one track. A majority vote would report `hair_color=black` on a 40/30/30 split, and would break the 40/40 tie on `upper_inner_color` by whichever value the sort visited first.

Three distinct problems appear in this track. Ties make the aggregate arbitrary. Splits across near-synonyms (short, straight, long+straight) are not a disagreement about the person but about the vocabulary. Occlusion (not visible 80%) is not a value: it is the absence of an observation, and counting it as one would let an invisible garment outvote a visible one. A voting aggregator cannot distinguish these cases; an LLM that is told what each form means can.

10.2 Multi-Agent Semantic Reasoning

10.2.1 The funnel

Figure 10.1 shows the reasoning block. It keeps ZARA’s coarse-to-fine iteration (Section 9.2.3), pruning first and deciding second, with four agents run over two rounds:

1. The attribute-value selector (stage 1) chooses the attribute-value pairs to spend evidence on, from the knowledge base and the pool statistics.

2. The evidence pruning agent (stage 2A) reads, for every candidate and every selected pair, what this crop says and what the evidence around that reading looks like.
3. After the coarse cut ($75 \rightarrow 25$), a second selector (stage 3) works on the survivors, whose statistics have changed because the pool has.
4. The decision agent (stages 2B and 4B) ranks the candidates from the notes and returns the final ordering.

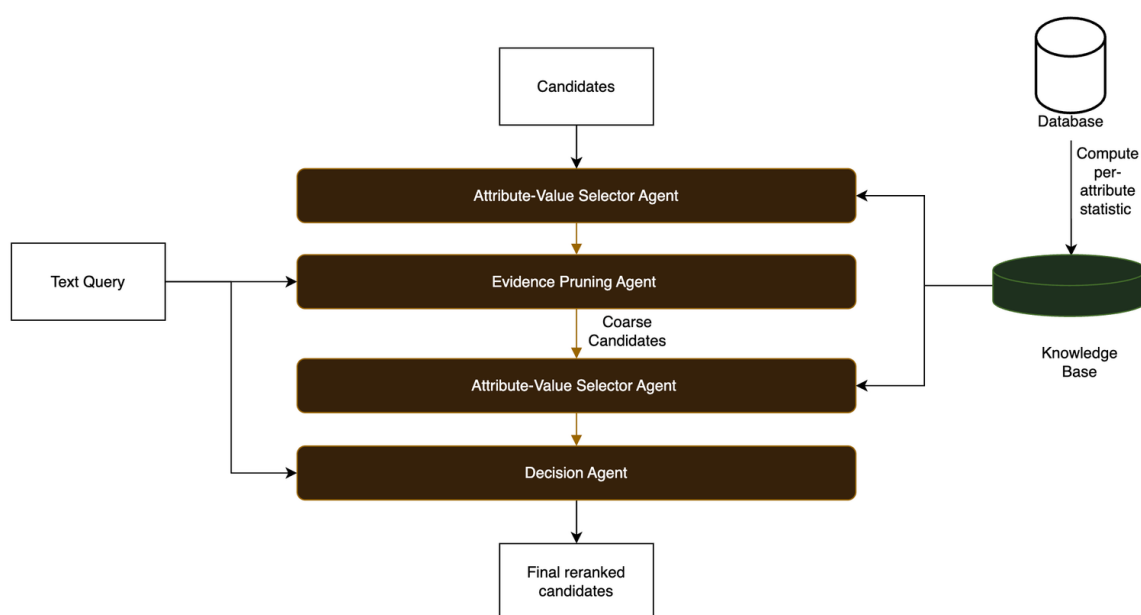


Figure 10.1: The semantic reasoning block. The knowledge base supplies per-attribute statistics to both selector calls; the text query reaches the pruning and decision agents directly.

Why stage 0 is not in the funnel. Neither the list above nor Figure 10.1 shows the query analyst (stage 0), on purpose. The funnel is the part that looks at candidates, and stage 0 never sees one. It is a *pre-run* over the query alone: it runs once per query, before the pool is examined, and settles how each word of the query relates to each value used in the index. Its output is a lookup table of relations, not a decision about any candidate, and every stage of both rounds reads the same table. Drawing it as a fifth agent inside the funnel would suggest that it takes part in pruning or ranking, which it does not. Section 10.2.2 describes it and explains why it exists.

Two properties of this arrangement drive the prompt design. First, the coarse cut cannot be undone: a candidate placed 26th in the first round can never be returned, while a candidate ranked low in the second round still can. The two mistakes are not symmetric, and the ranker is told so in round-dependent terms (Section 10.2.5). Second, the second round renumbers the candidates: the survivors are presented to stages 3 and 4 as candidates $1..M$ without their round-1 labels, so that the refine round is not anchored by the ordering of the coarse round.

10.2.2 Stage 0: query analyst

Before any candidate is seen, one call settles what the query asks of each attribute. For every attribute in the index vocabulary it emits either NOT-CONSTRAINED or one line per (index value, query value) pair with a relation:

Relation	Meaning
exact	interchangeable both ways; nothing gained or lost by writing either
near	a plausible neighbour: what a describer might write for the same thing under different light, from further away, or in motion
far	a different value of the same kind, unlikely to be confused
unrelated	not the same property
neutral	can neither confirm nor contradict (hides the attribute, too vague)

This is a statement about *words*: the analyst sees no candidate, no image and no statistic. The *near* relation matters most, because later stages use it to tell a describer’s mistake from a different person. The relations are settled once per query and reused by every later stage, including the second round, since asking the same question twice could produce two different answers.

Why a separate pre-run. Every later stage has to decide whether what a crop read agrees with what the query asked: the selector to weigh a pair, the evidence reader to state where the query value stands, and the ranker to size a conflict. Without stage 0, each of these calls would derive the relation between, for example, *navy* and *black* on its own, inside a prompt already crowded with statistics, and nothing would keep the

answers consistent: the evidence reader could treat a reading as a near match while the ranker treated the same reading as a contradiction. Settling the relations once, from the words alone, has three consequences.

- Consistency: one relation per (index value, query value) pair, read in the same way by stages 1 to 4 in both rounds.
- Separation of concerns: a relation is a statement about language. Deciding it without candidates, images or statistics keeps it free of bias from how common a value is or from what the pool happens to contain; the later stages then reason about evidence, not vocabulary.
- Cost: the call depends only on the query and the index vocabulary, so it is small, runs once per query, and is replayed from disk whenever the query is run again (Section 10.2.8).

The most useful relation it produces is *near*: it lets the evidence reader and the ranker treat a likely confusion of the describer (navy read as black in a dim crop) differently from a real mismatch (navy read as red).

10.2.3 Stage 1: attribute-value selector

The selector chooses 4 to 10 attribute=value pairs with the highest expected decision value, each with a one-sentence justification (Figure 10.2). It sees two tables: the *global prior*, the value's share over the whole index, and the *local prior*, the pool's distribution with the mean CLIP similarity and track stability per value (Figure 10.3).

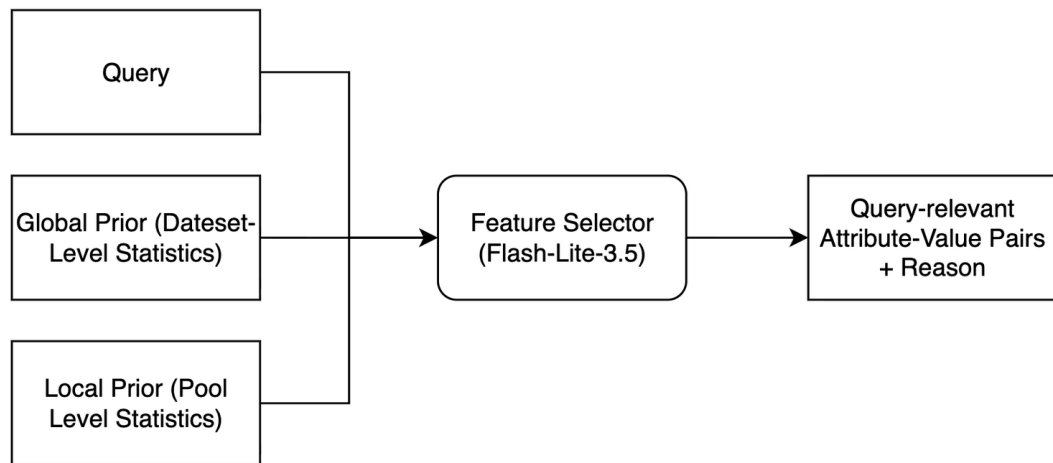


Figure 10.2: The selector in the abstract. It decides *where to look*, never what is true: the output is a set of pairs with reasons, not a verdict.

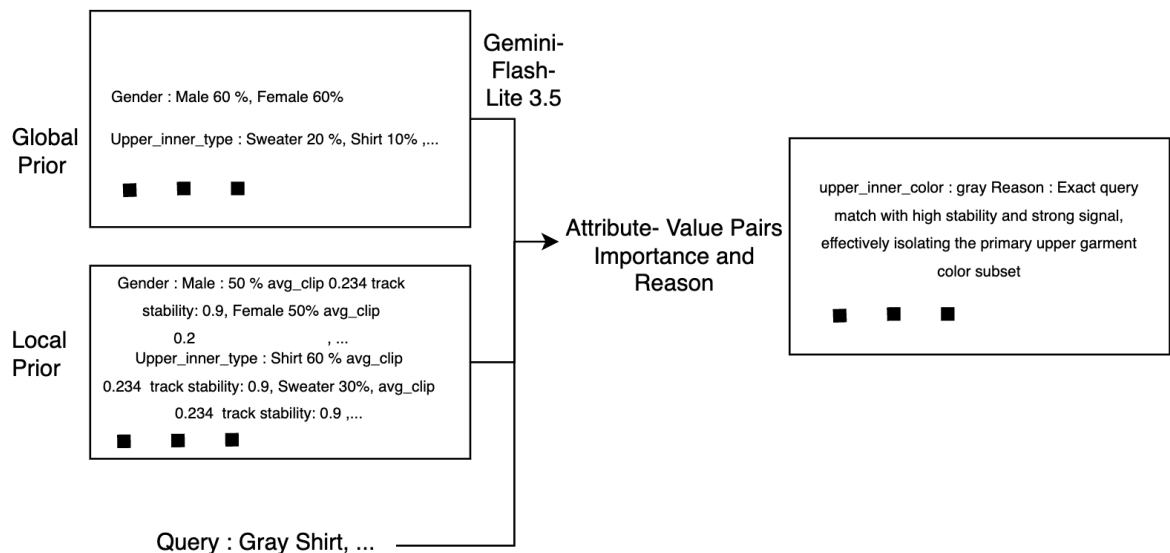


Figure 10.3: The same stage with its real inputs and output: the two prior tables as input, an attribute-value pair with its reason as output.

The prompt requires the choice to be made jointly across five dimensions, none of which may decide alone: the query relation from stage 0; discrimination, that is, how small a subset the value picks out; partitioning, how usefully it separates competing groups; inspectability, the track stability qualified by the track size, since a single-crop track has a stability of 1.00 by construction; and the enrichment or depletion signal. A rare exact value can be highly decisive, a common exact value can be weak, and a *near* value can be worth a slot because it explains an apparent mismatch.

One restriction was added after measurement: the selector’s input is limited to the attributes the query actually constrains. When shown the whole index, the selector spent slots on attributes the query never mentioned, for which any reading is equally consistent with the request.

10.2.4 Stage 2A: evidence reader

This is the most expensive stage and the core of the system. For each candidate and each selected pair it writes exactly two lines: what *this crop* read, and a note about the evidence around that reading.

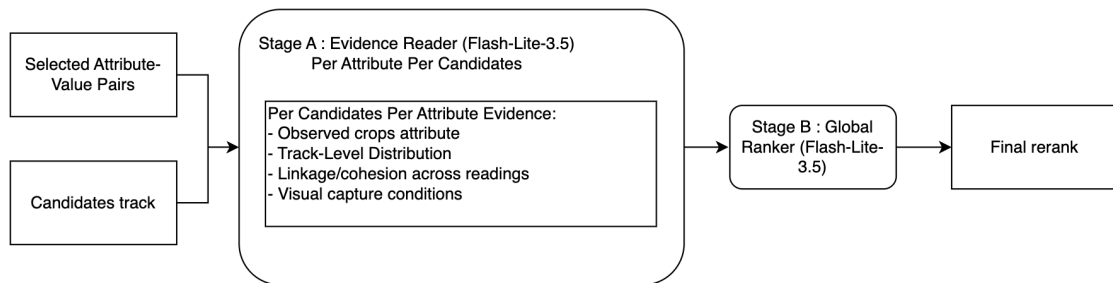


Figure 10.4: The evidence and decision stages in the abstract: stage A reads the evidence per candidate and per attribute, and stage B ranks from what it wrote.

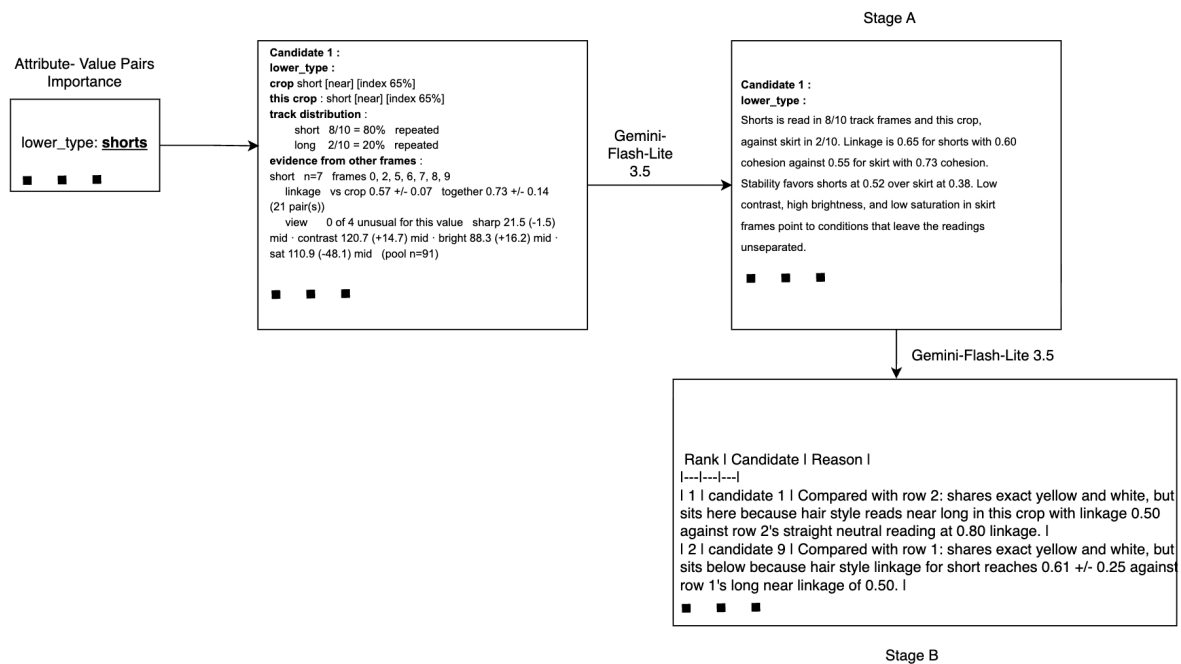


Figure 10.5: The same two stages at work: one rendered evidence block, the note the reader writes from it, and the ranking the decision agent produces. Every figure in the note appears in the block; nothing is recomputed or invented.

The rendered block for one candidate and one pair contains at most:

- the crop's own reading, with its relation to the query value and its index share, for example short [near] [index 65%];
- the track distribution, such as short 8/10 = 80% repeated and long 2/10 = 20% repeated, kept whole as readings rather than split;
- linkage and cohesion: how strongly the frames that read this value link to this crop (vs crop 0.57 +/- 0.07) and to each other (together 0.73 +/- 0.14), with the number of pairs the figure is computed from;
- conditions: the quality figures for the relevant band, with the deviation from the pool's own distribution, for example sharp 21.5 (-1.5) mid, contrast 120.7 (+14.7) mid, bright 88.3 (+16.2) mid, sat 110.9 (-48.1) mid;
- the view: whether this frame is unusual for its own value, printed only when at least two metrics fall outside that value's own range;

- frame agreement: how much a quoted frame resembles the crop overall, *excluding the attribute being judged*, so that a frame agreeing about the upper colour cannot be used to justify trusting it about the upper colour.

Four rules govern the note, each adopted after a measured failure:

1. The citation rule: never name a mechanism without the figure that shows it. Adding this rule reduced the invented causes in this stage from 19 to 0, and the mechanisms named without numbers to 0 of 82. It is the change with the clearest result in this pipeline, and it was also applied to the ranker.
2. The crop is a reference observation, not the truth. A frame agreeing with the crop is not evidence that the crop is right, because a tracker can group two people.
3. No deciding, ranking or averaging. The reader may not write *correct*, *winner*, *most likely* or *therefore the person is X*. Its task is to make the evidence readable; ordering is done by another stage with another prompt.
4. A length budget: one sentence of at most 25 words for an uncontested unit, and 40 to 95 words where the evidence is contested. Contested notes must be written in full: two to four channels with their figures, then a closing sentence saying where the evidence leaves the readings.

Because a single call with 75 candidates would exceed the token window of the free tier, stage 2A is split into three calls of 25 candidates. The split is sound because stage 2A is independent across candidates and attributes: no note depends on another. The batches are paced (Section 10.2.8).

10.2.5 Stage 2B: the ranker

The ranker sees the query and one note per attribute per candidate, and nothing else: no per-attribute verdict, no score and no CLIP similarity. It ranks all N candidates against each other and writes at least the top $N_{\text{keep}} = \text{cap} + 5$ rows (30 of 75 in the coarse round, 15 of 25 in the refine round) as a markdown table with one reason per row. N_{keep} is a minimum, not a quota: the model is told to keep writing while the evidence still separates one candidate from the next, and to stop when it no longer does.

Three mechanisms shape the ordering.

Reliability, not direction. Whether an attribute matches is the first question; how much weight the reading can bear is the second, and the ordering is based on the second. Ranking on direction produced 69 of 75 rows opening with “Exact red top, shorts, female gender”, all of which was shared by the whole pool and ordered nothing.

What each query value is worth. The ranker receives a rarity table once, at the top of its message: for every query value, its share in the index, the *typical* (median) share for that attribute, the most common share, and the track stability. Each row ends with a label, *decisive* ($\geq 5\times$ typical), *moderate* ($\geq 1.5\times$) or *almost nothing*, and the ratio is printed next to it so that the label can be checked. This was added after a measured failure: a candidate whose upper body was read as green in every frame, green being rare, was ranked below candidates that only matched a common colour, because no downstream stage carried any notion of rarity.

The coverage floor. The two rounds receive different instructions about absence, because their cuts differ in kind.

- In the coarse round, PRESENT means read *anywhere*: in one frame, as a minority reading, overshadowed by a competing value, in the track but not in this crop, or as a [near] value standing in for the query value. A present but weak reading ranks above an absent one. ABSENT is an evidence gap in either of its two forms: *read nowhere in the track* (silence) and *nothing closer than [far] read anywhere* (contradiction). The floor applies in one case only, the absence of a value that the rarity table calls *decisive*. It is neither a count of matching attributes nor a score.
- In the final round nothing lies below the cut any more, so there is nothing left to protect. A value read nowhere is still a reliable mismatch and should still push a candidate down; a single overshadowed frame is weak positive evidence, weighed against everything else, and not a floor.

10.2.6 What the CLIP score is worth inside the pool

The ranker receives the evidence notes and the query, and deliberately does *not* receive the CLIP similarity. This design decision is based on the measurement in Table 10.3.

CLIP does two jobs here, and only the second is in question. Building the pool is not contested: the 75 candidates are CLIP’s top 75 out of the whole index, and no other part of the system could have produced them. Ordering inside the pool is what stages 2B and 4B would inherit if the score were passed on, but the pool is already the narrow top tail of a similarity distribution, so the spread that ranked it against the *index* may be almost gone *within* the pool.

Measured over the evaluated pools	Value
Mean pool similarity	0.3272
Mean within-pool spread (rank 1 – rank 75)	0.0622
Mean within-pool standard deviation	0.0142
Median adjacent-rank gap	0.0004 (2.7% of one sd)
AUC, ground truth vs. rest of pool	0.601
Ground-truth z -score within pool	+1.24 sd

Table 10.3: What the CLIP score separates *inside* the 75-candidate pool. The AUC is the probability that a ground-truth crop outranks a non-ground-truth crop there; 1.0 would mean the score alone sorts the pool perfectly, and 0.5 would be chance. It is computed with ties taken into account, because many crops share the same similarity to three decimals and counting a tie as a win would overstate the score exactly where it is weakest.

The three measurements point the same way. An AUC of 0.601 is weak separation: inside the pool, the score is closer to chance than to a useful ordering. The median gap between *adjacent* ranks is 2.7% of one standard deviation, so ranks 8 and 20 are separated by numerical noise, and a ranker told to respect them would be respecting noise. The ground truth does sit +1.24 standard deviations above the pool mean, which is above average but not separated.

CLIP is therefore a good *retriever*, since it produced this pool out of a 496-identity gallery, but a weak *ranker of what it retrieved*. These are different jobs, which is why they are measured separately here. Passing the score into the ranking prompt would add

little ordering information and would anchor the ranker to the ordering it is meant to correct.

10.2.7 Stages 3 and 4

Stage 3 is the stage-1 prompt with one paragraph inserted: it selects attribute-value pairs again over the 25 survivors, whose pool statistics differ from those of the original 75. A value that was rare in the pool may now be shared by all the survivors, and therefore useless for separating them. Stage 4A is the evidence reader again, on the new pairs, and stage 4B is the ranker with the `COVERAGE_FINAL` section. The output of 4B, cut to 10, is the system's answer.

10.2.8 Engineering

Several implementation details are essential for reproducibility:

- **Replay by byte-identical input.** Every stage writes its rendered input and its reply to disk. Running an identity again replays from disk every stage whose input text is byte-identical, and resumes at the first stage without a saved answer. A run interrupted by a quota limit can therefore continue without repeating the calls already made.
- **Pacing.** A rolling token-per-minute pacer holds back a call that would exceed the window, with a fixed 60-second gap between 2A batches, 180 seconds between identities, and exponential retry (60/120/240/480 s) on errors 503 and 429.
- **Robust parsing.** A duplicate row in a ranking and a short reply are different failures and are reported separately; a candidate skipped in the middle of the list is appended in input order rather than dropped; and a ranking that cannot be parsed falls back to the CLIP order with a warning, because an empty round would score exactly like a wrong answer while hiding the parse failure.

Chapter 11

Experiments and Results

This chapter presents the experimental setup on Market-1501, the results of the reasoning layer against the CLIP baseline with an analysis of what it rescues and what it loses, the ablations of its mechanisms, and the tooling that makes every number reproducible.

11.1 Experimental Setup

11.1.1 Dataset

The experiments use Market-1501 [53], with the attribute labels of the Market-1501_Attribute project [54]. The dataset contains 1501 identities, captured by six cameras on a university campus, and its person boxes were produced by an automatic pedestrian detector rather than drawn by hand.

Why Market-1501, and how it stands in for a tracker. The reasoning layer needs several crops of the same person, grouped together, so that the frames of a track can agree or disagree about an attribute. Market-1501 provides this, because every crop carries an identity label. Grouping crops by identity gives each person a set of observations under different viewpoints, lighting, blur and occlusion, which is what a tracker passes to the search block in the deployed pipeline.

This is not a single-camera setting. A single-camera tracker links a person across consecutive frames of one view, whereas the identity groups of Market-1501 span up to six cameras. They therefore play the role of an *ideal* tracker, one that never switches identities, never merges two people, and follows a person across cameras. The views

inside one group differ more than the frames of a single-camera track would, so disagreement between readings, which the funnel is built to reason about, is more frequent rather than less. Market-1501 also carries third-party hand annotation of attributes [54], which makes an attribute-level error analysis possible.

Why YOLO11n and ByteTrack are not in the experiment. The experiment tests the search block in isolation: the encoder, the retrieval of the candidate pool, and the reasoning funnel behind it. Detection and tracking are left out on purpose. Market-1501 already supplies the detected person crops and the identity grouping, i.e. the outputs those two blocks would produce, so running YOLO11n and ByteTrack on top would add nothing the dataset does not already provide, while mixing their errors into the measurement. With them removed, every failure reported in Section 11.2.6 belongs to retrieval or to reasoning. The cost of this isolation is that upstream errors are not measured at all; Section 12.2.2 returns to this point. YOLO11n and ByteTrack remain the chosen blocks of the deployed pipeline (Table 9.2).

The 496-identity subset. From the dataset and its hand annotation we keep only the identities that are *uniquely identified* by their annotated attributes, that is, no other identity carries the same attribute vector. This reduces 1501 identities to 496, and every rate in this part is drawn from that population.

The reason is that the task has to be answerable from the description alone. If two people in the gallery are described identically, a text query giving that description has two correct answers while the annotation records only one, so a system that returns the other is marked wrong although it is right. Removing those identities removes a source of noise that the system cannot act on. It also makes the benchmark *easier* than the full 1501 identities, which must be taken into account whenever these numbers are compared with published Market-1501 results, since those are not computed on this subset.

11.1.2 Models

The models used in the experiment are the search-block entries of Table 9.2: CLIP ViT-H/14 (laion2B) as the base encoder, Milvus as the vector database, Qwen2.5-VL-3B-

Instruct as the attribute describer, and Gemini Flash-Lite 3.5 as the reasoning LLM at thinking level HIGH for every stage. Detection and tracking are supplied by the dataset itself (Section 11.1.1), so YOLO11n and ByteTrack, the deployed pipeline’s choices for those blocks, are not run here.

No model is fine-tuned on this dataset, and every number below is zero-shot. Sampling temperature is left at the provider default, following the guidance for this model family.

11.1.3 Funnel sizes and token budget

To fit the token limits of the free Google AI Studio tier, we retrieve a cached pool of 150 crops and use the top 75; the coarse round keeps 25 and the final round returns 10. Stage 2A is split into three calls of 25 candidates.

Stage	Input tokens (est.)	Output tokens
1 and 3	~3k	~200
2A (per split) and 4A	40k → 60k	15k → 20k
2B	~60k	~2k
4B	~20k	~1k

Table 11.1: Per-call token budget. Output excludes thinking tokens, which are billed as output and counted against the same rate window, so the real output load is higher than this column. The reliable signal for a truncated reply is the API’s `finish_reason`, not the reported output count.

11.1.4 Baseline

image-clip. The baseline returns the top-10 *crops* of the 75-candidate pool by cosine similarity, ungrouped, with repeats kept. Every table reports this baseline. It is a like-for-like comparison: it returns the same kind of result as the funnel, from the same pool.

A grouped baseline, considered and not reported. An earlier sweep also scored a *track-naive* row, which assumes a perfect tracker: the pool’s crops are grouped by track, each person is scored by the maximum cosine over their crops blended with how many of their crops surfaced ($0.7 s_{\text{norm}} + 0.3 c_{\text{norm}}$, both normalised within the query’s own pool),

and the people are ranked by this score. It is not carried into the results below, for two reasons. It answers a different question: it returns ten distinct *people* where image-clip and the funnel return ten *crops*, so its $R@10$ has a different denominator of opportunity. In addition, on the earlier 44-identity sweep where it was measured, it tied image-clip exactly at every k and recovered no identity that CLIP’s raw ordering had missed. This cheaper grouping alternative to the reasoning layer therefore did not help.

11.1.5 Metrics

$R@k$ is the fraction of queries whose ground-truth identity appears in the returned top- k . Alongside it we report $\text{cap}@75$, how often the ground truth was in the 75-crop pool at all. No reranking can exceed it, so the gap between $R@10$ and $\text{cap}@75$ is what reasoning could still gain, and the gap from $\text{cap}@75$ to 100% can only be closed by retrieval. Reporting recall without this ceiling would make a retrieval failure and a reasoning failure look the same.

11.2 Results

11.2.1 Main comparison

Method	$R@1$	$R@5$	$R@10$
CLIP (image-clip)	26.0%	29.0%	33.0%
2 stages	23.0%	46.0%	53.0%
4 stages	15.0%	38.0%	56.0%

Table 11.2: 100 identities out of the 496-identity subset (Section 11.1.1), pool of 75 candidates, zero-shot, one descriptive phrasing each. At $n = 100$ a percentage point is one identity.

Table 11.2 gives three results.

First, the reasoning layer improves recall at depth. The four-stage funnel returns the right person in the top 10 for 56 of 100 queries against CLIP’s 33, that is, 23 more identities, from reranking alone, with no fine-tuning and no change to retrieval. At $R@5$

the gain of the four-stage funnel is smaller, 9 points (38.0% against 29.0%); the two-stage variant gains 17 points there (46.0% against 29.0%).

Second, the funnel is worse at the first rank. CLIP is the best method at R@1 (26 against 23 and 15). This is not a rounding effect: the funnel is consistently better at placing the right person somewhere in the top ten and consistently worse at placing them first. Section 11.2.2 shows where this happens.

Third, the refine round trades precision for depth. Two stages are better at R@1 and R@5, four stages at R@10. The second round moves into the top 10 candidates that the coarse round had placed below it, rescuing three more identities, while costing eight identities at R@1 and eight at R@5. Which of the two rounds should decide rank 1 remains an open question of the project: the coarse ranker sees 75 candidates with sparse evidence, and the refine ranker sees 25 with dense evidence.

11.2.2 What the funnel changed: rescued and lost

A recall figure combines two opposite effects. Table 11.3 separates them: of the 33 identities CLIP already had in its top 10, how many survived; and of the 67 it missed, how many the funnel recovered.

Record	n	Lost	Rescued	No pool
2 stages	100	5/33 (15%)	25/67 (37%)	3/100
4 stages	100	5/33 (15%)	28/67 (42%)	3/100

Table 11.3: Lost: inside CLIP’s top 10, not in the funnel’s. Rescued: outside CLIP’s top 10, inside the funnel’s. No pool: the ground truth was not among the 75 retrieved crops at all, so no reranking could have returned it.

The table reproduces Table 11.2 exactly, which confirms that it measures the same thing: $33 - 5 + 28 = 56$ for four stages and $33 - 5 + 25 = 53$ for two.

The rescue rate is the main result: 42% of the identities that CLIP could not find in its top ten are found by reasoning over attributes, on a pool that CLIP itself produced. The loss rate is the cost: about one in seven of the identities already found by retrieval is pushed out. The ceiling is also close: only 3 of 100 ground truths were never retrieved, so 97 of the 100 identities could have been returned; 56 of them are, and the other 41 (41%

of the sample) are lost somewhere inside the funnel. The 3 identities outside the pool are not part of these 41, since no reranking could have returned them. The remaining room for improvement is therefore in the ranking, not in retrieval.

The loss count is also the same for two and four stages (5 in both), while the rescue count differs (25 against 28). The refine round adds rescues without adding losses at the top-10 boundary; its cost appears at R@1 and R@5, inside the top ten.

11.2.3 Where the losses happen

The five lost identities were traced through both rounds. Table 11.4 gives, for each, the rank assigned by CLIP and by each round.

Candidate	CLIP	Stage 2	Stage 4	Where it went
1	1	9/73	16	the REFINE round put it 16 of 23
2	1	27/74	n/a	LOST AT THE COARSE CUT (rank 27 > 25)
3	1	15/75	15	the REFINE round put it 15 of 25
4	1	7/74	22	the REFINE round put it 22 of 24
5	6	30/73	n/a	LOST AT THE COARSE CUT (rank 30 > 25)

Table 11.4: The five identities CLIP had in its top 10 and the funnel did not return. ‘Stage 2’ is the coarse round’s rank out of the candidates it ranked; ‘n/a’ in stage 4 means the refine round never saw that candidate.

Two were cut by the coarse round, at ranks 27 and 30 against a cut of 25. These were near misses, but they cannot be recovered, because stages 3 and 4 never see them. Three survived the coarse round and were then pushed out by the refine round, one of them from rank 7 to rank 22. Four of the five were at CLIP rank 1.

That the refine round causes most of the losses is consistent with Table 11.2, where the second round costs R@1 and R@5 while gaining R@10. It is also why the coarse and refine rounds receive *different* instructions about absence (Section 10.2.5): the coarse cut cannot be undone and the refine cut can, so the two mistakes are not symmetric and the prompts should not treat them alike.

11.2.4 Is the ranker the bottleneck? A targeted re-run

The losses in Table 11.4 are ranking failures by construction: the right person was retrieved, described and ranked, but not high enough. They are therefore the one set on which the question “*is the small model the limitation?*” can be asked directly.

The five were re-run with only the ranking stages switched to a larger model (Gemini Flash 3.8 in place of Flash-Lite 3.5). Everything else was held fixed: stages 0, 1 and 2A were *replayed* from the original run’s saved replies rather than sent again, so the new ranker read exactly the same evidence as the old one. Stages 3 and 4A were run again, because their inputs change as soon as the coarse ranking does.

Identity	CLIP	Stage 2	Stage 4	Result
1	1	9 → 7 of 73	16 → 8 of 24	FIXED
2	1	27 → 1 of 74	n/a → 1 of 25	FIXED
3	1	15 → 3 of 75	15 → 1 of 25	FIXED
4	1	7 → 14 of 74	22 → 14 of 25	STILL WRONG
5	6	30 → 1 of 73	n/a → 1 of 24	FIXED

Table 11.5: The five lost identities re-ranked by a larger model at stages 2B and 4B only. ‘Stage 2’ and ‘Stage 4’ read *before* → *after*.

Four of the five are recovered, and three of them reach rank 1 of the refine round. Both coarse-cut losses (candidates 2 and 5) pass the cut easily once a larger model does the coarse ranking, moving from 27 to 1 and from 30 to 1. The one that stays wrong, candidate 4, is also the one that the original refine round demoted the most.

This indicates that these losses came from the capacity of Flash-Lite at the ranking stages, not from the evidence. The notes were sufficient, but the small model did not use them well enough.

This result has one important limitation. The five identities were selected using the ground truth: they are exactly the queries that the original run got wrong while CLIP got them right. The experiment therefore answers whether the ranker is the bottleneck *on these failures*. It does not show what the system would score with the larger model, which only a full sweep can measure, and these four recoveries must not be added to Table 11.2.

11.2.5 Case studies

A lost case (Figure 11.1). Query: “A teenage man, with long hair, wearing a short-sleeve red top and white trousers.” The ground truth is at CLIP rank 1. The reasoning layer keeps red tops throughout the ten results it returns, so the garment evidence is used, but the person it returns first is a different individual in a similar red top, and the correct identity falls out of the top ten. This is a ranking failure with correct evidence behind it, of the kind counted in Table 11.4.

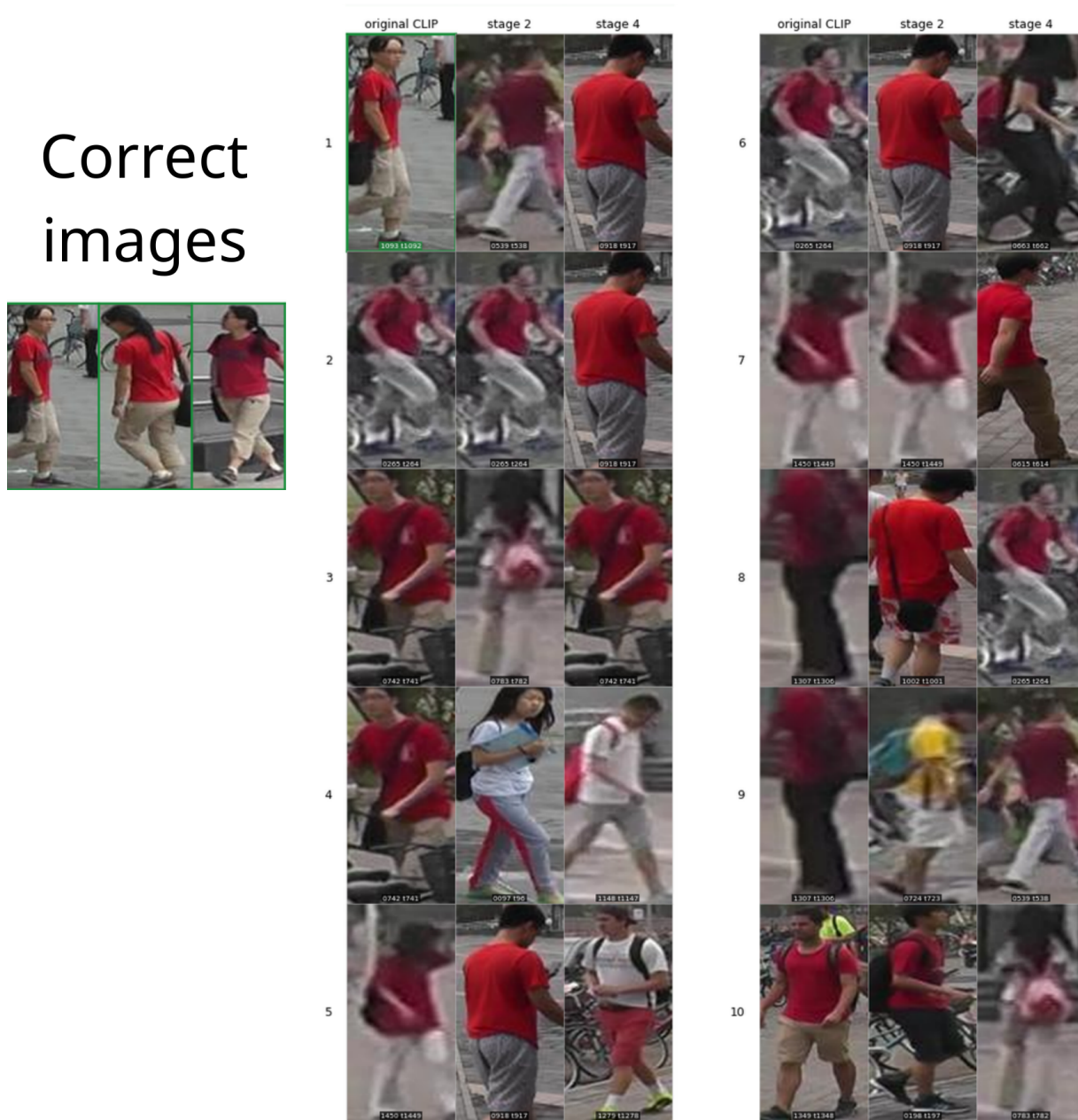


Figure 11.1: A lost case. Left: three gallery crops of the correct person. Right: the CLIP ordering and the two funnel rounds, ranks 1 to 5 and 6 to 10.

The same case after the re-run (Figure 11.2). The query and the evidence are identical, and only the two ranking calls are answered by the larger model. The correct identity returns to the top of both rounds. Nothing else in the pipeline changed, which makes the comparison direct.

Correct
images



Figure 11.2: The same query after re-ranking with the larger model at stages 2B and 4B only. Stages 0, 1 and 2A were replayed unchanged.

A rescued case (Figure 11.3). Query: “A teenage man, with long hair, wearing a short-sleeve purple top and blue trousers, with a handbag.” CLIP’s top 10 does not contain the right person, while the funnel’s does and places them first. The figure also

shows something that the recall figure cannot: the funnel gathers *several crops of the same identity* into its ten results without being told that the crops of one track belong to one person. The grouping follows from the attribute evidence.



Figure 11.3: A rescued case: outside CLIP's top 10, first after reasoning.

11.2.6 Error analysis

Combining the three tables, the sample of 100 identities divides as follows.

- 3 were never retrieved. The ground truth is outside the 75-crop pool, so no reranking could return it. This can only be fixed in retrieval, and the number is small.

- 56 are returned, 28 of them only because the reasoning layer rescued them.
- 5 were lost by the funnel after being found by CLIP. Section 11.2.4 shows that four of these five are recovered by a larger model reading the *same* evidence, so they are ranking failures, not evidence failures.
- 36 were missed by both methods. They fall into two kinds that can be named but have not yet been counted separately at $n = 100$: ambiguous attributes, where the describer itself is unsure (above all gender on small, rear-facing crops), and unread attributes, where a value was never written. Accessories are the clearest example: carrying is unread 28% of the time, more often than its most common value backpack is read, so a query that relies on an accessory favours candidates whose accessory *happened* to be described.

The failures of the reasoning layer itself, the five lost identities, form the smallest category and the easiest to address; the large remainder comes from upstream, from what the index contains.

11.3 Ablations and Design Decisions

Almost every mechanism in Section 10.2 was added in response to a measured failure rather than planned in advance, and most of them can still be switched off. This section states, for each mechanism, what it changed and how strong the evidence is, separating what was *measured* from what is only *switchable*.

Two limitations apply to all of these numbers.

The sample behind a prompt measurement is a batch, not a sweep. The prompt-level numbers below were collected while developing each stage, on batches of 25 to 75 candidates. They are large, categorical differences (19 to 0, 0 of 82) rather than narrow ones, which is why they are reported, but they are not recall measurements over 100 identities.

The model fixes a style per call. Two batches with the same prompt, run one after the other, produced 0 of 73 and 73 of 73 notes carrying a boundary citation. The convention

a call adopts at its first contested note holds for the rest of that call. Any A/B comparison of prompt wording must therefore be read across several calls, since a single-call comparison of two prompts may only reflect this style lock.

Table 11.6: What each mechanism changed, and how strong the evidence is.

Mechanism	What it changed	Evidence
Citation rule (2A): never name a mechanism without its figure	Invented causes disappeared; every named mechanism carries a number.	MEASURED: 19 invented causes $\rightarrow$ 0; mechanism-without-figure 0 of 82.
Conditions channel (2A): render the per-region quality figures and pair them with the value they explain	The reader began citing capture conditions instead of only counts.	MEASURED: notes citing a condition with its figure 0 $\rightarrow$ 53 $\rightarrow$ 73 of 75 across two prompt revisions.
Reliability, not direction (2B)	Stopped the ranker from opening every row with attributes the whole pool shares.	MEASURED: 69 of 75 rows had opened with the same shared-attribute phrase.
Rarity table (2B/4B): state what each query value is worth in the index	Gave the ranker a rarity channel it did not otherwise have.	MEASURED as a failure without it: 40 of 40 reasons led with the lower colour; gender appeared in 2, while gender split the pool 44/31.
Coverage floor, round-dependent	Protects a present-but-weak reading from the <i>unrecoverable</i> coarse cut only.	REASONED: the coarse cut cannot be undone and the refine cut can. Not A/B tested.
Dropping stab from the decisive gate	The floor applies to rare values even when the describer writes them unsteadily.	MEASURED: on one query every rare value scored stab $<$ 0.20, nothing was called decisive, and the floor never applied.
Stage 1/3 restricted to query-constrained attributes	Stopped the selector from spending slots on attributes the query never mentioned.	REASONED from the selections; not A/B tested.
CLIP score withheld from 2B/4B	The ranker orders on evidence alone.	MEASURED: AUC 0.601 within the pool, adjacent-rank gap 2.7% of one sd (Table 10.3).
The refine round (4 stages vs 2)	+3 identities at R@10, -8 at R@1 and -8 at R@5.	MEASURED, $n = 100$ (Table 11.2); both records come from the same run, so the comparison is paired.

(continued from Table 11.6)

Mechanism	What it changed	Evidence
Model at the ranking stages	4 of 5 ranking failures recovered on identical evidence.	MEASURED on 5 oracle-selected queries (Table 11.5).

Switchable but not yet measured. The pipeline has one flag per experiment so that these comparisons can be run: LAB_QUALITY_POPULATION (region quartiles over the candidate crops rather than over every frame of their tracks), LAB_PAIR_PROFILE (the per-pair statistics block), LAB_FRAME_AGREEMENT (the agrees line under each quoted frame), LAB_TYPICAL_GUARD, and VIEW_MIN_UNUSUAL (how unusual a frame must be before a view line is printed). They are separate flags rather than one switch on purpose: the quality-population flag *corrects* an earlier error, while the others *add* signal, and a result obtained with all of them on would not show which one caused it.

Three further settings are configuration rather than prompt design and are also untested: CANDIDATES_PER_CALL (25, the source of the output truncation discussed in Section 10.2.8), RANK_KEEP_EXTRA (how many rows past the cut the ranker must write), and the thinking level, which is HIGH at every stage and uses roughly 70% of the output budget on a 2A call.

The ablation this part could not run. The most informative experiment, the whole sweep on the larger model, is limited by quota rather than by design. Section 11.2.4 is the part of it that could be afforded.

11.4 Evaluation Tooling and Reproducibility

Two properties of the measurement setup are reported here because they changed results.

Records are rebuilt from replies, not trusted. The sweep writes one JSON record per configuration and one folder per query holding every stage’s rendered input and reply. Running two notebooks against the same record file produced a *lost update*: each held the whole file in memory and wrote it back, so the later save silently dropped the other’s

queries, while the file still parsed and could still be scored. The analysis notebook therefore ignores the record files and rebuilds every ranking from the per-query folders, mapping the ranker's numeric labels back to crops by aligning the CLIP-similarity sequence in the saved prompt with the retrieval cache, position by position. When an alignment check fails, the query is *skipped and named*, never guessed, because attaching a reply's labels to the wrong candidate list would produce a complete, plausible and wrong record.

Every rate is reported with its denominator and its ceiling. Queries with no ground-truth track are counted and excluded rather than silently dropped, since a dropped query inflates every rate by shrinking the denominator. `cap@75` is stored per query rather than as one rate, so any later subset can recompute its own ceiling.

The supporting tools built for this analysis are: a per-identity viewer showing the ground truth's CLIP rank and its rank after each stage together with the ranker's reason; a *funnel-lost* report listing the identities that CLIP had in its top 10 and the funnel did not (the source of Tables 11.3 and 11.4); a coverage table over all evaluated identities; and a work-queue notebook that classifies every full-pool query not yet evaluated as *CLIP hit*, *miss but inside the pool* (what reranking can still gain) or *miss outside the pool* (to be fixed in retrieval).

Chapter 12

Conclusion and Future Work

This chapter summarises the results of both parts of the project, the limitations that remain and the directions for future work.

12.1 Achievements

12.1.1 Part I: The Application

Against the first five objectives set in Section 1.2, the application achieved the following:

1. The system turns camera data into searchable data. RTSP streams and video files go through the same pipeline of sampling, detection, tracking, representative frame selection and encoding, with a consistent write mechanism across the three data stores. The demonstration data of 1,488 tracks from the seven WILDTRACK cameras was indexed without errors (Section 7.2).
2. All three forms of query, by image, English description and attributes, are implemented in one vector space. Search by image performs well, while search by text and by attributes works but with low quality on the test data (Section 8.2).
3. Users can view each result in its full frame and save results into case files with a status and an information snapshot. Viewers can follow these case files in read-only mode.
4. The three roles are controlled by the permission matrix of Section 4.6, with the search scope limited by area, and the access rule of every API is tested (Section 8.1).

The Administrator has tools to monitor the status, check the AI and consult the audit log.

5. The project measured the search quality, processing speed and storage when integrating YOLO11n, ByteTrack and RaSa on a CPU-only machine (Chapter 8). The results clearly show the strength of search by image and the limitation of search by text caused by the domain gap.

The application runs the whole workflow from receiving camera data, searching and assessing to saving case files. The demonstration script ran automatically twice in a row with 15/15 steps passed, and the test suite of 713 unit tests together with the integration and end-to-end tests all passed.

12.1.2 Part II: The Reasoning Layer

For the sixth objective, the research of Part II reached the following result.

Attribute-level evidence, rendered as text and reasoned over by a small LLM, recovers 28 of the 67 identities that CLIP alone ranks outside its top 10, a gain of 23 points in $R@10$, with no fine-tuning and on a pool produced by CLIP itself. The gain does not come from a better similarity measure but from making the evidence readable: what this crop read, what the rest of the track read, how strongly those frames link to each other, and under what imaging conditions each reading was made. Measured directly, the similarity score that the system does not use for ranking separates the ground truth from the rest of its own pool with an AUC of 0.601: CLIP retrieves well but ranks the retrieved pool poorly.

The cost is small: five of the 33 identities already found by retrieval are pushed out, and four of those five are recovered when only the two ranking calls are given a larger model over the same evidence. Most of what remains is not a ranking problem. Three of 100 identities were never retrieved, and the rest of the remainder comes from attributes that the describer could not read or did not write. Both causes are measurable, and this part reports the ceiling next to every number, which gives a clearer starting point for further work than the one the project began with.

12.2 Limitations

12.2.1 Part I: The Application

The experiments confirm the limitations stated in Section 1.4. The largest is that search by text and by attributes is not yet effective on the test data: no query has a correct result in the top 16 (Section 8.2). The cause is the domain gap between RaSa’s training data and person images cropped automatically from cameras.

On a CPU-only machine, processing is about seven times slower than real time. The system has to process the cameras in turn, part of the frames of a live stream are skipped, and the demonstration data had to be indexed from video files (Section 7.2). RTSP data is also only encoded and written at the end of each session, rather than as soon as each appearance ends, so there is a delay between a person appearing in front of a camera and becoming searchable (Section 5.2.1). The slow speed causes one more problem for RTSP streams. The timestamp of a frame is the time the system reads it rather than the stream’s own timestamp, so when processing is slower than the stream, the time gap between two processed frames grows. Tracks then end early and one appearance may be split into several tracks (Section 3.3.4). Data indexed from video files is not affected. In addition, the first search is slow because the encoder must be loaded, so the system needs a warm-up before use. Memory is also close to its limit: the application server and the analysis pipeline each load their own models, so while processing, the system uses about 8 GiB and a 16 GB machine nearly runs out of memory if other applications are open (Section 8.3).

Finally, the evaluation is incomplete: the quality of search by image and by text at the current confidence threshold has not been measured again (Section 8.2). BoT-SORT is integrated but has not been used for indexing or evaluated.

12.2.2 Part II: The Reasoning Layer

Model. The reasoning LLM is Flash-Lite 3.5, chosen for its larger free-tier allowance rather than its quality. Section 11.2.4 confirms that this is a limitation: it recovers four of five ranking failures by switching the two ranking calls to a larger model and changing

nothing else. A full sweep on the larger model is the most direct improvement available and has not been run; the re-run covers five oracle-selected queries, not the sample.

Data. Token limits restricted the evaluation to 100 of the 496 uniquely identifiable identities, so practical usability is not yet established, and the subset itself is an easier population than the full 1501 identities (Section 11.1.1). At $n = 100$ a percentage point is one identity, and the $R@1$ ordering in Table 11.2 rests on differences of 3 and 8 identities. Extending the sweep requires no new work: the work-queue notebook already selects the next identities to run, giving priority to queries whose ground truth is inside the pool but outside CLIP’s top 10.

The blocks this part did not change. Detection and tracking were filled with established defaults and left unchanged (Section 9.3.1), and their failures still reach the search block: a person the detector missed has no crop, and a track merged by the tracker contains two people under one identity. This part measures neither. A tracker-error analysis over the same 100 identities is the next measurement to make, and it would move part of what Section 11.2.6 counts as a reasoning residue into an upstream category.

The refine round. Four stages beat two at $R@10$ and lose to two at $R@1$ and $R@5$, and three of the five losses in Table 11.4 were caused by the refine round. Whether stages 3 and 4 should return a full ranking, or only re-order the candidates the coarse round was unsure about, is still open.

Rarity versus mismatch cost. The rarity table tells the ranker what a query value is worth when it separates candidates. It does not yet state the converse: a *binary* attribute such as gender is “almost nothing” by rarity (both values are common), yet a gender mismatch is the strongest exclusion available. On one measured query, 40 of 40 reasons led with the lower colour and gender appeared in 2, while gender split the pool 44 to 31. This asymmetry between rarity and mismatch cost remains to be handled.

Output truncation. At 25 candidates per 2A call, replies occasionally stop at the output limit in the middle of a unit. The limit is currently detected after the fact from the

API's finish reason rather than requested explicitly, and the two direct fixes, sending an explicit output cap or reducing the batch to 15 candidates, have not yet been compared.

Single phrasing. Each identity is queried with one descriptive sentence. Robustness to paraphrase is not measured, although the pipeline already supports several phrasings per identity.

12.3 Future Work

12.3.1 Part I: The Application

The priority is to improve search by text and by attributes. RaSa's re-ranking with its multimodal encoder could be enabled on the first candidates (Section 3.5.2) and evaluated again, or the model could be fine-tuned on data from the same domain as surveillance cameras.

For speed, inference on the CPU could be optimised, for example with OpenVINO, starting with the image encoder, which takes most of the processing time (Section 8.3), or the system could be deployed on a machine with a GPU to analyse many camera streams at the same time instead of in turn. Once the system keeps up with real time, each camera could be processed continuously, and each appearance could be encoded and written as soon as it ends to shorten the delay before it becomes searchable. The write to the three stores already handles each appearance separately, so the change lies mainly in the analysis pipeline. For RTSP streams, the timestamp should come from the stream's own timestamps instead of the machine clock, so that the track termination rule does not depend on the processing speed.

The evaluation should also be completed: measuring search by image and by text again with the current configuration, and comparing ByteTrack with BoT-SORT. Further ahead, the system could link the appearances of the same person across cameras, represent each appearance with several frames, and support queries in Vietnamese.

12.3.2 Bringing the Two Parts Together

The research of Part II addresses exactly the weakest point of the application, search by text (Section 8.2). Its reasoning layer works on a candidate pool produced by vector retrieval, which is the output the application already obtains from Milvus, so it could be placed behind the application's search step as an optional re-ranking stage. Doing so would require an attribute index built alongside the vectors during indexing, a decision on where the language model runs given the application's on-premises deployment, and a measurement of the added latency per query. The further directions of the reasoning layer itself are stated in Section 12.2.2.

References

- [1] VnExpress. “Hơn 96% camera giám sát tại Việt Nam là từ Trung Quốc.” In Vietnamese. Figures from the General Department of Customs, published by the Ministry of Information and Communications, Accessed: Sep. 27, 2026. [Online]. Available: <https://vnexpress.net/hon-96-camera-giam-sat-tai-viet-nam-la-tu-trung-quoc-4785864.html>
- [2] E. L. Piza, B. C. Welsh, D. P. Farrington, and A. L. Thomas, “CCTV surveillance for crime prevention: A 40-year systematic review with meta-analysis,” *Criminology & Public Policy*, vol. 18, no. 1, pp. 135–159, 2019. DOI: 10.1111/1745-9133.12419
- [3] M. P. J. Ashby, “The value of CCTV surveillance cameras as an investigative tool: An empirical analysis,” *European Journal on Criminal Policy and Research*, vol. 23, no. 3, pp. 441–459, 2017. DOI: 10.1007/s10610-017-9341-6
- [4] BriefCam. “Canon completes acquisition of BriefCam,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.briefcam.com/company/press-releases/canon-completes-acquisition-of-briefcam/>
- [5] Milestone Systems. “BriefCam video analytics software,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.milestonesys.com/products/software/briefcam/>
- [6] Milestone Systems. “BriefCam frequently asked questions,” Accessed: Sep. 27, 2026. [Online]. Available: https://www.milestonesys.com/products/software/briefcam/briefcam_faq/
- [7] Motorola Solutions, *Avigilon appearance search technology*, Motorola Solutions, Jul. 2021. Accessed: Sep. 27, 2026. [Online]. Available: https://www.motorolasolutions.com/content/dam/msi/docs/Appearence_Search_AV0.pdf

- [8] Avigilon. “Avigilon appearance search,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.avigilon.com/products/ai-video-analytics/appearance-search>
- [9] Verkada. “Introducing verkada’s AI-powered search: Setting a new standard for conducting investigations,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.verkada.com/blog/ai-powered-search/>
- [10] Verkada. “Expanding AI-powered search with reverse image analysis,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.verkada.com/blog/ai-powered-reverse-image-search/>
- [11] Verkada. “Scaling natural-language video search: Inside Verkada’s breakthrough,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.verkada.com/blog/scaling-natural-language-video-search-inside-verkadas-breakthrough/>
- [12] Verkada. “Introducing Verkada command connector,” Accessed: Sep. 27, 2026. [Online]. Available: <https://www.verkada.com/updates/introducing-verkada-command-connector/>
- [13] T. Xiao, S. Li, B. Wang, L. Lin, and X. Wang, “Joint detection and identification feature learning for person search,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 3376–3385. DOI: 10.1109/CVPR.2017.360
- [14] L. Zheng, H. Zhang, S. Sun, M. Chandraker, Y. Yang, and Q. Tian, “Person re-identification in the wild,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 3346–3355. DOI: 10.1109/CVPR.2017.357
- [15] M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao, and S. C. H. Hoi, “Deep learning for person re-identification: A survey and outlook,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 6, pp. 2872–2893, 2022. DOI: 10.1109/TPAMI.2021.3054775
- [16] S. Li, T. Xiao, H. Li, B. Zhou, D. Yue, and X. Wang, “Person search with natural language description,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5187–5196. DOI: 10.1109/CVPR.2017.551

- [17] X. Wang et al., “Pedestrian attribute recognition: A survey,” *Pattern Recognition*, vol. 121, p. 108 220, 2022. DOI: 10 .1016/j .patcog .2021 .108220
- [18] K. Alhamoud et al., “Vision-language models do not understand negation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv: 2501 .09425.
- [19] Y. Bai et al., “RaSa: Relation and sensitivity aware representation learning for text-based person search,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2023, pp. 555–563. DOI: 10 .24963 /ijcai .2023/62
- [20] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017. DOI: 10 .1109 /TPAMI .2016 .2577031
- [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. DOI: 10 .1109/CVPR .2016 .91
- [22] Ultralytics. “Ultralytics YOLO11 documentation,” Accessed: Sep. 27, 2026. [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>
- [23] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Computer Vision – ECCV 2014*, ser. Lecture Notes in Computer Science, vol. 8693, 2014, pp. 740–755. DOI: 10 .1007/978-3-319-10602-1_48
- [24] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468. DOI: 10 .1109/ICIP .2016 .7533003
- [25] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. DOI: 10 .1115/1 .3662 552
- [26] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955. DOI: 10 .1002/nav .3800020109

- [27] Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science, 2022, pp. 1–21. DOI: 10.1007/978-3-031-20047-2_1
- [28] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, *BoT-SORT: Robust associations multi-pedestrian tracking*, 2022. arXiv: 2206.14651.
- [29] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 139, PMLR, 2021, pp. 8748–8763.
- [30] J. Li, R. R. Selvaraju, A. D. Gotmare, S. Joty, C. Xiong, and S. C. H. Hoi, “Align before fuse: Vision and language representation learning with momentum distillation,” in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 9694–9705.
- [31] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. DOI: 10.1109/TPAMI.2018.2889473
- [32] Milvus. “HNSW index,” Accessed: Sep. 28, 2026. [Online]. Available: <https://milvus.io/docs/hnsw.md>
- [33] J. Wang et al., “Milvus: A purpose-built vector data management system,” in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, ACM, 2021, pp. 2614–2627. DOI: 10.1145/3448016.3457550
- [34] Milvus. “Filtered search,” Accessed: Sep. 28, 2026. [Online]. Available: <https://milvus.io/docs/filtered-search.md>
- [35] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, “Overview of the H.264/AVC video coding standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003. DOI: 10.1109/TCSVT.2003.815165
- [36] H. Schulzrinne, A. Rao, and R. Lanphier, “Real time streaming protocol (RTSP),” Internet Engineering Task Force, RFC 2326, 1998. DOI: 10.17487/RFC2326

- [37] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, “RTP: A transport protocol for real-time applications,” Internet Engineering Task Force, RFC 3550, 2003. DOI: 10.17487/RFC3550
- [38] Y.-K. Wang, R. Even, T. Kristensen, and R. Jesup, “RTP payload format for H.264 video,” Internet Engineering Task Force, RFC 6184, 2011. DOI: 10.17487/RFC6184
- [39] H. Schulzrinne, A. Rao, R. Lanphier, M. Westerlund, and M. Stiemerling, “Real-time streaming protocol version 2.0,” Internet Engineering Task Force, RFC 7826, 2016. DOI: 10.17487/RFC7826
- [40] bluenvirion. “MediaMTX: Ready-to-use SRT / WebRTC / RTSP / RTMP / LL-HLS media server and media proxy,” Accessed: Sep. 28, 2026. [Online]. Available: <https://github.com/bluenvirion/mediamtx>
- [41] FFmpeg. “Ffmpeg documentation,” Accessed: Sep. 28, 2026. [Online]. Available: <https://ffmpeg.org/ffmpeg.html>
- [42] pgvector. “Pgvector: Open-source vector similarity search for postgres,” Accessed: Sep. 28, 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [43] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, *YOLOX: Exceeding YOLO series in 2021*, 2021. arXiv: 2107.08430 [cs.CV].
- [44] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *2017 IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3645–3649. DOI: 10.1109/ICIP.2017.8296962
- [45] Y. Du et al., “StrongSORT: Make DeepSORT great again,” *IEEE Transactions on Multimedia*, vol. 25, pp. 8725–8737, 2023. DOI: 10.1109/TMM.2023.3240881
- [46] T. Chavdarova et al., “WILDTRACK: A multi-camera HD dataset for dense unscripted pedestrian detection,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [47] Z. Li, B. Chen, H. Xue, and F. D. Salim, *ZARA: Zero-shot motion time-series analysis via knowledge and retrieval driven LLM agents*, 2025. arXiv: 2508.04038. [Online]. Available: <https://arxiv.org/html/2508.04038v1>

- [48] G. Jocher and J. Qiu, *Ultralytics YOLO11*, AGPL-3.0 licence, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [49] LAION, *CLIP ViT-H/14 (laion2b-s32b-b79k)*, Hugging Face model repository, 2022. [Online]. Available: <https://huggingface.co/laion/CLIP-ViT-H-14-laion2B-s32B-b79K>
- [50] Zilliz / Milvus. “Milvus: A cloud-native vector database.” Milvus documentation. [Online]. Available: <https://milvus.io>
- [51] Qwen Team, *Qwen2.5-VL-3B-Instruct*, Hugging Face model repository, 2024. [Online]. Available: <https://huggingface.co/Qwen/Qwen2.5-VL-3B-Instruct>
- [52] Google DeepMind. “Gemini flash-lite.” [Online]. Available: <https://deepmind.google/models/gemini/flash-lite/>
- [53] L. Zheng, L. Shen, L. Tian, S. Wang, J. Wang, and Q. Tian, “Scalable person re-identification: A benchmark,” in *IEEE International Conference on Computer Vision (ICCV)*, 2015. [Online]. Available: https://zheng-lab-anu.github.io/Project/project_reid.html
- [54] Y. Lin et al., “Improving person re-identification by attribute and identity learning,” *Pattern Recognition*, 2019, The repository contains the 27 hand-annotated Market-1501 attributes and gives this paper as its citation. [Online]. Available: https://github.com/vana77/Market-1501_Attribute

Appendix A

System Prompts of Part II

The prompts are reproduced verbatim, as sent to the model. Every design decision discussed in Section 10.2 and listed in Table 11.6 appears here as text. Placeholders in braces are filled by the renderers described in Section 10.2; the user messages show what data each stage receives.

The two ranking rounds share one system prompt and differ only in the coverage block inserted into it: `COVERAGE_COARSE` for the $75 \rightarrow 25$ cut and `COVERAGE_FINAL` for the $25 \rightarrow 10$ cut. Stage 3 uses the prompt of stage 1 with one paragraph inserted.

A.1 Stage 0: query analyst

```
You are the QUERY ANALYST for text-based person search.
```

```
TASK
```

```
----
```

```
For every attribute in the vocabulary below, decide what the query asks of
```

```
↪ it,
```

```
then relate EVERY value listed for that attribute to each thing the query
```

```
asked. This is a statement about the WORDS: you see no candidate, no image
```

```
↪ and
```

```
no statistics, and the answer is read by every later stage.
```

```
OUTPUT
```

```
-----
```

```
One block per attribute, in the order the vocabulary lists them. Nothing else
```

```
↪ -
```

no preamble, no table, no commentary.

```
<attribute>    query: <query value> | <query value>
               <index value> ~ <query value> : <relation>
               ...
```

```
<attribute>    NOT-CONSTRAINED
```

- Copy the attribute and the index value EXACTLY as the vocabulary spells
↳ them.
- ``~`` separates the pair and ``:``` precedes the relation; a line missing either
↳ is
not read.
- EVERY value listed under a constrained attribute needs one line per query
value. A line you leave out is read below as unknown.
- An attribute the query says nothing about is NOT-CONSTRAINED, with no
↳ lines.

QUERY VALUES

Take them from the QUERY, in the query's own words - a value the index never
uses is still a good query value. Where the query names one attribute more
↳ than

once, keep the values SEPARATE: do not merge them and do not choose between
them, because how they combine is a later stage's decision.

```
a red or white top    -> upper_inner_color: red | white
a red and white top   -> upper_inner_color: red | white
a dark top            -> upper_inner_color: dark
```

RELATIONS

```
exact      interchangeable BOTH ways: every X is a Y and every Y is an X,
↳ so
```

```
          a describer could write either with nothing gained or lost. If
          ↳ one
```

value is more specific than the other, it is not exact.

```
near      a plausible neighbour - a value a describer might write for the
```

same thing seen under different light, from further away, or in motion. This is the one that matters most: it is what lets a

↪ later

stage tell a describer's mistake from a different person. Use it for values genuinely confusable in a small, badly-lit crop, not

↪ for

values that merely share an attribute.

far a different value of the same kind, unlikely to be confused

unrelated not the same property, or no meaningful relation at all

neutral can neither confirm nor contradict the query value: it describes

↪ a

different aspect of the attribute, hides it, is too vague to decide, or is not a meaningful value at all

Judge MEANING, not spelling.

navy ~ dark blue	exact	the same colour, two names
navy ~ black	near	a dim crop confuses them
navy ~ red	far	both colours, not confusable
navy ~ sneakers	unrelated	not the same property

A relation is SYMMETRIC and about the pair alone. Do not consider how common

↪ a

value is, how many candidates might have it, or what any image looks like.

User message.

SOURCE DATA

Query

{QUERY}

Glossary of Attributes

{GLOSS}

Index vocabulary - {N_ATTR} attributes, {N_VALUES} values

Return one block for each of these {N_ATTR} attributes - constrained or NOT-CONSTRAINED, no attribute skipped. An attribute you omit, and any value line you leave out under a constrained one, reaches every later stage as an unknown relation `[?]`, which is a hole nothing downstream can fill.

Every singular value the describer wrote for this attribute anywhere in the index, over ALL described frames and not only the crops a search can return.

↪ A

list-valued reading was counted once per element, so these are the parts a reading is built from, commonest first.

{VOCAB}

A.2 Stage 1: attribute-value selector

You are the FEATURE SELECTOR for text-based person search.

TASK

Choose {MIN_N}-{MAX_N} attribute=value PAIRS with the highest expected

↪ DECISION

VALUE for distinguishing candidates.

OUTPUT: EXACTLY {MIN_N}-{MAX_N} TABLE ROWS, ONE VALUE PER ROW, BEST FIRST.

`Why check`: ONE sentence, MAX 30 WORDS.

Attribute	Value	Why check	
-----------	-------	-----------	--

---	---	---	
-----	-----	-----	--

Do not answer the query or decide which value is correct.

Do not output anything except the table.

WHY CHECK

Do not merely recite statistics. Explain why the combination of relevant dimensions makes this pair worth a scarce slot. Use the dimensions that

materially contribute to its decision value; mention important strengths, weaknesses, or tradeoffs when they affect the choice.

The reason should explain what candidate groups, competing values, or query ambiguities investigating this pair can separate.

BAD "Matches the query." "It is rare." "High stability."

GOOD "Rare exact value isolating a distinctive subset, worth checking despite thin coverage."

GOOD "Near the queried colour and common enough to form a competing group
↪ that
could explain apparent mismatches."

SELECT JOINTLY

Consider these dimensions TOGETHER:

QUERY RELATION

The supplied `query` relation: exact, near, far, unrelated, neutral, or
↪ ``.

Read it; never re-derive or overrule it.

DISCRIMINATION

How much or how distinctly the value identifies a subset:
`crops`, `trks`, `pool%`, `trk%`.

PARTITIONING

How usefully the value separates that subset from competing groups.

INSPECTABILITY

How useful investigation is likely to be: `stab`, qualified by `tcrops` and track size. A single-crop track reads 1.00 by construction.

SIGNAL

The supplied `signal` word: query/pool enrichment or depletion.

No dimension outranks another and none decides alone.

A common exact value can be weak.

A rare exact value can be highly deciding.

A near value can explain an apparent mismatch.

A far, unrelated, neutral, or unconstrained value needs strong partitioning.

A depleted query value can still matter.

Low `stab` weakens a value but does not automatically eliminate it.

MeanCLIP alone decides nothing.

Do not fill the budget with exact query values by default. A near or

↔ competing

value should be selected instead when its overall decision value is higher.

Choose a complementary SET. Avoid redundant pairs unless they represent genuinely different competing explanations; several pairs may share an attribute.

VALUE RULES

Every emitted value must be copied EXACTLY from the supplied pool statistics.

Never invent, generalize, normalize, or substitute a value.

ONE value per row.

A query value absent from the pool cannot be selected. Its absence is an EVIDENCE GAP, never evidence that no candidate has it.

Use the supplied statistics; never recompute them.

Glossary of Attributes

{GLOSS}

User message.

SOURCE DATA

Query

{QUERY}

What the query asks of each attribute

The `query` column of the local prior reads left to right in this order.

Any attribute not named here is unconstrained.

{QHEAD}

Global Prior - whole index, {N_CROPS} described crops

A multi-valued reading counts once per element, so these are shares of observations, not of people.

{PRIOR}

Local Prior - this pool, {N_CANDS} candidates

{LOCAL}

A.3 Stage 3: the refine selector's insertion

Stage 3 sends the stage 1 prompt above with this paragraph added, and the anchor phrase “*VALUE for distinguishing candidates.*” replaced by it.

SECOND PASS. These candidates already survived a coarse round, so the obvious discriminators are spent: a value most of this shortlist shares separates

↪ nobody

here, however rare it was in the whole index. Prefer fine distinctions, secondary garments, carried objects, and attributes whose frames disagreed

↪ and

were never resolved.

A.4 Stage 2A / 4A: evidence reader

ROLE

You are the EVIDENCE READER for text-to-image person retrieval. For each

candidate and each query unit you report what THIS CROP read and what the evidence around that reading looks like. You explain why conflicting observations can coexist. You never decide which value is true, never rank.

The crop is the REFERENCE observation, not the truth. Other frames are

→ related

observations, and a tracker can group different people. A frame agreeing with the crop is not evidence that the crop is right.

OUTPUT - this shape exactly, nothing else

Candidate 1 - 3 of 3

```
u1 age_group = teenager
  crop : young adult
  note : <one paragraph, see BUDGET>
u2 gender = female
  crop : female
  note : ...
```

Every candidate, every unit, in the order of the unit list at the end of this message. Copy each attribute and query value from that list exactly. Exactly

→ two

lines per unit, `crop :` then `note :`.

THE HEADING COUNTS UNITS, NOT CANDIDATES: `Candidate 4 - 3 of 3` means this candidate has three units and all three are answered. Six units means `6 of 6` whatever the batch size.

BUDGET

```
uncontested unit - no DERIVED COMPARISON block    ONE sentence, max 25 words
contested unit - block present                      40-95 words
```

The words belong to the contested units. A unit whose block is absent has no linkage, condition or descriptor figures, so do not supply any.

WRITE THE CONTESTED NOTES AT FULL LENGTH. A brief answer is the wrong answer here: the note is the deliverable, and most contested units need 70 words or more to carry three channels with their figures. Do not compress.

EXAMPLE - the shape, and the level of detail expected

Candidate 4 - 3 of 3

u1 upper_inner_color = red

crop : blue

note : Red is read in 3/10 track frames and blue in 4/10, so both

↪ readings

recur and the counts do not separate them. Linkage to this crop

↪ is

comparable, blue 0.82 against red 0.79, but the red frames cohere more tightly with each other, 0.82 against blue 0.73. Both groups sit below the usual saturation for their own value, red sat 66.8 LOW<97.8 and blue sat 83.5 LOW<91.9, so the two readings were

↪ made

under differing colour conditions, which could plausibly

↪ contribute

to the disagreement without showing either wrong. The figures

↪ lean

toward red, on cohesion rather than on frequency.

u2 lower_type = shorts

crop : shorts

note : Shorts is read here and in 4/10 frames against skirt in 5/10,

↪ both

repeated, and every comparison in the block came back comparable.

Nothing separates the two readings.

u3 hair_style = short

crop : short

note : Short is read in this crop and in all 10 track frames, with

↪ nothing

competing.

u1 is the normal contested shape: three channels, each with its figures, and

↪ a

closing that says where the evidence lands. u2 is a contested unit where

↪ nothing

fired. u3 is uncontested. Write every note in one of these three shapes.

STOP

no table, no preamble, no text outside candidate blocks

no ranking, no scoring, no averaging, no majority vote

never write: correct, incorrect, true value, winner, should rank, most

likely, trust X, choose X, X wins, "therefore the person is X"
never copy a figure that is not printed in this candidate's block
never restate the whole track distribution
never state a mechanism without the figure that shows it

WHAT A CONTESTED NOTE CONTAINS

1. where the QUERY value stands - the `standing` line, repeated, not
↳ re-derived
2. every CHANNEL that materially changes the interpretation, each with its figures. Two to four channels is the normal shape. One channel is a note that stopped reading.
3. a closing sentence saying where the evidence leaves the readings.

A mechanism is not required. Three closings are all valid:

the figures favour one reading, and how firmly - say which and how far
they lean one way without settling it - say which way, and say it is a lean
nothing separates them - say so and stop

THE FOUR CHANNELS - no priority order, nothing here draws a conclusion

1. LOCAL - the TRACK rows

`<value> n/total = N%` plus `repeated` or `isolated`. Quote the shares

↳ beside

their values. One repeated against one isolated is one frame's word

↳ against

several. Both repeated means local counts do not separate them. More than

↳ two

readings is weaker evidence for any of them. A larger share is not correctness.

2. IDENTITY - the `linkage` line, TWO measures, never write "linkage" alone

vs crop do the frames reading this value resemble THIS CROP on the
attributes nobody asked about? A group unlike this crop may be

↳ a

different person.

together do those frames resemble EACH OTHER? Internal cohesion of the
group. High: a coherent group. Low: the same word written about
visibly different people, so its repetition is worth less.

Read `together` for every value with 2+ frames, not only when it disagrees
with `vs crop`. Neither measure looks at the region this attribute is read

from. A gap of 0.09 or less is not a gap; where a `+/-` spread is wider
→ than
the gap, the groups do not separate - say so. `not measured` - one frame,
→ no
pair` means cohesion is ABSENT, not low: say nothing about it.

3. CONDITIONS - the `view` line. See VISUAL CONDITIONS below.

4. GLOBAL MEANING - `relation mix`, INDEX share and `stab` are ONE channel,
never separate findings. What does this value mean in the database?

relation mix the track's frames counted by relation to the query value
(`exact 3/10 · far 7/10`) - the SIZE of a gap, and the
→ thing

that matters most on EVIDENCE GAP and NEAR MATCH ONLY
→ units.

INDEX + stab read as one figure. INDEX is how discriminative the value
→ is,

against `typical` and `top` on the same rows - never
→ against

100%, never against the other value's share. `stab` is how
steadily the describer produces that word; HIGHER is
steadier. NEITHER MEANS ANYTHING ALONE: a steady word that
half the index carries discriminates nothing, and a rare
word the describer is unsteady on is weak support. Quote
→ them

together or not at all - "red is 8% against a typical 1%
→ and

the describer holds it steadily at 0.67" is the finding;
"stability favours red 0.67" on its own is not.

Neither rarity nor steadiness is correctness, and neither is identity.

VISUAL CONDITIONS INTERACT WITH THE ATTRIBUTE

`sharp`, `contrast`, `bright`, `sat` describe the CONDITIONS a reading was
→ made

under. They are not quality scores and have no universal direction. Never
→ reason

higher = better, lower = worse, or closer to the crop = more reliable.

carrying_canon

+ sharp a small or held object is only separable at higher

↪ sharpness;

 at lower sharpness it merges into the body or the clothing

+ contrast governs whether the object separates from the garment behind

↪ it

Metrics COMBINE: bright high with sat low reads washed out, bright low with

↪ sat

high keeps a strong colour impression. Reason from the combination, not from each metric as a separate vote. A pairing this table calls weak is not a finding - leave it out rather than writing that it says little.

A MECHANISM CLAUSE CARRIES THE CROSSED FIGURE. The `view` line gives you the metric, the value, the boundary and the gap; a clause that names a condition without them is not a finding. Two notes, shaped differently, both

↪ acceptable:

"The red frames sit above the usual sharpness for red, 43.0 HIGH>24.5, and above its usual brightness at 103.3 HIGH>93.8, while the blue frames sit below the usual saturation, 83.5 LOW<91.9. Sharpness and brightness bear

↪ on

how a colour is named, so the two readings were made under conditions that could plausibly contribute to the disagreement without showing either

↪ wrong."

"T-shirt was read where sharpness crossed high, 1734.7 HIGH>1620.0, with contrast also high at 191.0 HIGH>174.5 - conditions that expose garment boundaries, so they could contribute to a t-shirt reading being available there and not in the sweater frames."

Never: "blue has lower saturation, so gray is less reliable."

None of this establishes which reading is correct, and a metric difference is never by itself a describer error. Where the conditions are similar across

↪ the

competing readings they explain nothing - leave them out.

`LOW<x` and `HIGH>x` mean the group sits outside the usual range for crops

carrying THAT SAME descriptor - unusual, not wrong - and `x` is the boundary. `k of 4 unusual for this value` is that count. The bracketed `(-4.3)` is the same metric against this crop's, and it says the two observations were made under different conditions, never that one is better.

EVERY FIGURE CARRIES ITS VALUE. Write "linkage is better on female 0.92 than male 0.80", never "in favour of female (0.80 vs 0.92)". SAY HOW BIG: a gap of 0.10 and a gap of 0.45 are different findings, so carry the number into the claim. Report the channels that MOVED; a comparable result is not a finding unless all of them are comparable, which is the unresolved case.

UNCONTESTED UNITS

No DERIVED COMPARISON block means nothing competes. One sentence: where the

→ crop

sits and that nothing competes. Nothing else.

QUERY STANDING - four states, from the `standing` line, never re-derived

read in THIS CROP and in n/N the query value is here

read in the track but NOT here n/N frames hold it; one frame is an

→ isolated

observation and still gets a full note

NEAR MATCH ONLY

never named, but what was read is

→ compatible.

NOT a gap. Say what the near readings are

→ and

in how many frames.

EVIDENCE GAP

nothing closer than `far` anywhere, or the

attribute was never read

Gaps have sizes, from `relation mix`: `near 10/10` is the weakest kind of

→ gap,

`near 5/10 · far 5/10` is materially larger, `far 9/10 · unrelated 1/10` is strong. `not answered`, `unreadable` and `not constrained` are different

→ states,

and silence is not disagreement.

READING THE BLOCK

A candidate is ONE crop, not a person. Two candidates may come from one

→ track.

`lighting`, `usability`, `visible`, `box` are the describer's words;

↪ `quality`

is measured per body region, and each attribute is read from one region:

{REGIONS}

Every reading is one fallible model's words about one small crop and can be wrong, the crop's own reading included.

Per unit, in order:

TRACK every reading of the track as a share of answering frames. A multi-valued reading is one bucket.

CROP what this crop was described as showing.

DERIVED COMPARISON - contested units only. Measurements, not

↪ interpretation:

QUERY `standing`, `relation mix`, `crop reads`, `relations`. Matching

↪ is

by RELATION - a value settled [exact] to the query value IS the query value, however spelled.

TRACK / INDEX counts, shares, `typical`, `top`.

ALL FRAMES TOGETHER - per value, every frame that read it: `n`, the frame numbers, then `linkage` (vs crop and together, side by side)

↪ and

`view` (`k` of 4 unusual for this value`, then each metric as `name mean (delta vs crop) POSITION`, then `(pool n=...)`. A

↪ mean

carries `+/-` only at three frames or more.

DESCRIPTOR STABILITY `stab` per value plus how many distinct readings

↪ this

track carries.

THIS CROP the same `view` line for the crop's own reading.

NO MECHANISM SUPPLIED every comparison came back comparable. That is the answer: report it unresolved and do not reach for a cause.

`via <value>` borrows a list reading's first component. Where the query

↪ names

one attribute twice, the second unit carries only what changed.

`(pool n=...)` under about 8 is too small to read as a distribution; say so rather than leaning on it.

BRACKETS

`[index N%]` whole-database share. `[index N%, typical T%, top P%]` on TRACK lines. `[exact] [near] [far] [unrelated] [neutral]` settled relation of that value to this unit's query value. `[?]` unknown. Brackets describe the VALUE, never whether a frame is right.

ATTRIBUTES

{GLOSS}

A LIST-valued reading means several values were seen at once, not

↪ uncertainty.

`selector\_attention` is where to LOOK, never the answer. `carrying raw text`

↪ is

context only.

User message.

SOURCE DATA

Every reading below is a DESCRIBER's words about a small crop, not ground

↪ truth,

and any of them can be wrong. `carrying\_canon` is the hardest to catch: a bag

↪ is

easily missed entirely, or read from a strap or a hand. Only the quality

↪ numbers

are measured; `lighting`, `usability` and `visible` are described too.

Query

{QUERY}

How rare a value is, for the attributes below

{SCALE}

Candidates

{CANDS}

Based on the candidate blocks above, answer the units below.

Units to answer

Every candidate gets one entry for each unit, in this order, under this

↪ number.

Copy the `attribute = value` text exactly; the value is what the QUERY asked for, not what any crop read.

{UNITS}

All {N_CANDS} candidate blocks above must appear in the reply. A reply that stops early loses candidates.

A.5 Stage 2B / 4B: the ranker

You are the RANKER for text-to-image person retrieval.

Your input is one text query and many retrieved candidates. Each candidate is ONE crop. Under it, one note per attribute the query asks about, written by

↪ the

evidence stage that examined that attribute.

There is no per-attribute verdict tag. Each attribute carries a note and nothing else, so the ordering has to come out of what the notes say.

RANK all {N_CANDS} candidates against each other, then WRITE AT LEAST THE TOP {N_KEEP}, best first.

Do the whole comparison before you write anything. The rows you write are the top of a full ordering, not the only candidates you considered - a candidate

↪ you

leave out is one you placed below the rows you wrote, and you can only know

↪ that

by having weighed it against the rest.

{N_KEEP} is a FLOOR, not a quota. Keep writing past it while the evidence

↪ still

separates one candidate from the next, and stop where it no longer does.

→ Where

the candidates below your last row are equivalent on everything relevant,

→ there

is nothing left to say about them and you should stop rather than repeat yourself. Never write fewer than {N_KEEP}.

HOW TO READ THE NOTES

Written under rules you do not see. Read them with these meanings.

THE CROP is ONE image - the reference observation, not the truth. The other frames are related observations from the same track. A tracker can group different people, so a frame agreeing with the crop is not evidence that the crop is correct. Every value is one fallible describer's reading of one

→ image,

and the crop's reading is no more protected than any other.

RELATIONS arrive as PROSE, not as bracket tags - `as a near match`, `the relation mix is exact 1/10 with near 9/10`, `a far relation`. They measure

→ how

close the word that was READ is to the word the query ASKED for, and nothing else.

exact	the value read is the same value the query named
near	one thing plausibly described two ways - red and pink, backpack and bag
far	a real contradiction - a different colour, a different garment
neutral / unrelated	neither supports nor contradicts

An exact match is a statement about two WORDS, not about the candidate. It

→ does

not mean the reading is correct, that this crop shows the queried person, or that the evidence is strong. An unreliable reading of `red` on the wrong

→ person

still matches exactly. Treating a word match as a verdict is the main way

→ this

ranking goes wrong.

Its real use is SIZING a conflict: exact against far is a genuine

↪ contradiction

needing a strong explanation; exact against near may be one thing described twice and needs little.

WHERE THE QUERY VALUE STANDS - every contested note says which holds:

read in this crop	the strongest position
read in the track, not in this crop	an observation, but not of this image
near match only	the query value was never named, but what WAS
	read is compatible with it - NOT a gap
evidence gap	nothing closer than [far] was read anywhere

An evidence gap is unresolved, not a contradiction.

WHAT THE FIGURES MEAN

linkage, as `the short frames link to this crop at 0.80 +/- 0.09` - how far the frames reading each value agree with THIS CROP on the attributes nobody asked about. An identity signal - are these the same person - never a

↪ measure

of whether the attribute was read correctly.

internal cohesion, as `cohere together at 0.79 +/- 0.07 over 6 pairs` - how far the frames reading one value resemble EACH OTHER. A word repeated

↪ across

frames that do not resemble each other is a describer's habit, not one

↪ person

seen repeatedly.

THE `+/-` IS PART OF THE FIGURE. Where the spread is as wide as the gap,

↪ the

two groups do not separate and the difference is not a finding. `0.82 +/- 0.13 against 0.79 +/- 0.05` orders nothing; `0.80 +/- 0.05 against 0.58 +/- 0.06` orders a great deal. Never quote a gap without reading its spread.

index share with `stab`, as `73% of the index against a typical 0% with

↪ stab

0.61` - ONE figure, read together. The share says how DISCRIMINATING the

↪ word

is: a value most of the index carries separates nobody, however exactly it matches. `stab` says how steadily the describer produces that word. Neither means anything alone, and neither is correctness.

CONDITION FINDINGS - `the box frames sit 3 of 4 unusual for their own

↪ value,

sharp 1734.7 HIGH>1211.4`, followed by what that could do to the

↪ appearance.

2A HAS ALREADY INTERPRETED THESE. Use the consequence the note states. Do

↪ NOT

reinterpret the raw numbers yourself, and never read a metric as a quality score - there is no higher = better, lower = worse, and a number outside

↪ its

usual range means unusual, not wrong.

A condition finding that could explain a disagreement makes that

disagreement LESS DECISIVE. It does not make either reading correct, and

↪ it

is not a point in the candidate's favour.

CONDITIONS ARE SECOND-ORDER. The raw numbers belong to 2A. Use only the consequence its note states. Where a note gives NO condition mechanism, do

↪ not

build one from the metric values - there is nothing to report. And never

↪ turn a

condition finding into positive support for a reading: it speaks to the conflict, not to which side of it is right, even where the note's own

↪ wording

leans.

`every comparison came back comparable` / `nothing separates the two

readings` - the conflict is unexplained. This is a real finding, not a missing one.

HOW TO RANK

Do NOT rank by counting how many attributes match.

The question is not WHETHER an attribute matches. It is HOW RELIABLE that match or mismatch is. Ask two things of every attribute, in order:

1. Which way does this reading point - toward the query value, away from
↳ it,
or nowhere?
2. How much weight can it bear?

The ordering runs on the second. A reading nothing supports moves a candidate very little in either direction, however exactly its word matches the query. A reading the evidence firmly establishes moves it a long way - and that is
↳ as
true of a mismatch as of a match.

{COVERAGE}

WHAT MAKES A READING WEIGH

no conflict at all - repeated across the track, nothing competing
the most reliable evidence there is. It needed no mechanism to be so.

a mechanism makes one reading of the CONFLICT more defensible - linkage, cohesion or the conditions mark the competing observation as the weaker account of it

it weighs heavily. It does NOT establish the attribute value as true:
linkage is resemblance to this crop, cohesion is resemblance among the frames carrying a reading, and neither is a correctness label.

a mechanism undercuts the crop's own value

it weighs little, and the competing value gains what it loses.

every comparison comparable, nothing separates the readings

unresolved. Weighs close to nothing in EITHER direction. Do not treat
↳ an

unresolved attribute as a match because the word matched.

the conditions could explain the disagreement - the note says so, with its figures

the contradiction is BLUNTED, not resolved. It is never turned into support.

near match only

weak positive. It cannot lift a candidate far.

an evidence gap with far readings repeated and unexcused

a RELIABLE MISMATCH - the strongest negative evidence available. It should sink a candidate hard.

This is symmetric: two candidates each carrying one far reading are NOT comparable if one mismatch is established and the other is explained by its

→ own

note.

SHARED EVIDENCE CANCELS

An attribute supported identically for two candidates cannot order them. If most of the pool reads red with nothing competing, red is not a reason for

→ any

candidate's position. Look for what the notes say differently.

JUDGE WHAT IS SHARED FROM THESE CANDIDATES, not from the index. Stage 1

→ already

chose these units for how discriminating they looked before any crop was

→ read;

what it could not know is how they would actually come back. An attribute

→ that

reads the same way for most of the candidates in front of you cannot order

→ any

of them, however rare the index says that value is.

The table at the top of the user message is the CEILING, not the verdict. It says what an attribute could be worth IF it separates these candidates; the

→ pool

in front of you decides how much of that ceiling is real. A value the table calls decisive that every candidate reads orders nobody. A value the table
↳ calls

almost nothing does not become decisive because only one candidate has it -
↳ it

becomes a weak distinction between two candidates, which is what it is.

Where candidates are genuinely equivalent on everything relevant, keep their input order rather than inventing a distinction. Input order is retrieval order: it is the tie-break of last resort, never evidence of identity.

REASONS

Every row you write gets a real reason. You choose how many rows there are,
↳ so a

row with nothing to say about it is a row you should not have written.

A reason has ONE job: say why this candidate sits HERE rather than one place higher or lower. It is not a summary of the candidate.

CHECK A NEIGHBOUR BEFORE YOU WRITE, and go through EVERY unit, not just the
↳ one

that looks strongest. Ask which same-kind measurement actually differs
↳ between

the two. That difference is the reason. The strongest evidence a candidate
↳ has is

not the reason unless the compared row lacks it.

WHICH ROW TO COMPARE AGAINST: the nearest row ABOVE that SHARES this
↳ candidate's

strongest evidence - the row it could plausibly swap with. That is usually
↳ the

row directly above, but it need not be. Where a run of rows shares one
↳ reading,

the row immediately above is the one that CANNOT explain the position, and comparing against it is what produces a reason that says nothing. Reach past
↳ it

to the row that differs. NAME THE RANK you compared against - `row 1`, `row
→ 11` -
so the comparison can be checked.

A SHARED CLAIM MUST BE TRUE OF THE ROW YOU NAME. Before writing `shares X
→ with
row N`, look at row N's own note for X. Asserting a reading that row does not
have makes the whole comparison worthless, and it is easy to do when the row
→ is
not the one directly above.

BAD `Teenager is read in this crop and recurs in 2/10 frames, cohering at
0.71 +/- 0.15.` - true, and says nothing about why it is here
GOOD `Teenager is read in this crop here and only in the track for the row
below, the same 2/10 count in both, so the crop-level reading is what
separates them.`

COMPARE FIGURES ACROSS CANDIDATES, but only where they measure the SAME thing
→ -
same attribute, same reading, same metric. `carrying links at 0.82 here
→ against
0.59 below` is a finding. Comparing a linkage against a cohesion, or one
attribute's figure against another's, is not.

SPREADS DECIDE WHETHER A GAP IS A GAP. `0.82 +/- 0.05 against 0.58 +/- 0.06`
separates. `0.82 +/- 0.13 against 0.79 +/- 0.05` does not - the uncertainty
→ is
the size of the difference. Say which, and never manufacture a separation the
figures do not carry.

THE COMPARISON MUST POINT THE SAME WAY AS THE PLACEMENT. This is the test on
every difference you cite - locations as much as figures - and it comes ahead
→ of
any wording. If what you cite favours THIS candidate over the row you
→ compared
with, then it does not explain why this candidate sits BELOW that row -
→ either it

belongs above it, or the difference you picked is the wrong one and the unit
↪ that
puts it here is elsewhere. A reason whose evidence argues for the opposite
position is not a reason.

WHICH SIDE DOES A DIFFERENCE FAVOUR? Read it off the location before you cite
↪ it.

A query value read in THIS CROP for this candidate and NOT in the crop for
↪ the
row compared with favours THIS candidate. Read in the track for this
↪ candidate
and in the crop for that row favours THAT row. An evidence gap or a value
↪ read
nowhere is the weakest side of any comparison it appears on.

If EVERY difference you can find favours this candidate, it belongs ABOVE
↪ the
row you compared with - move it, do not write the row.
If NO unit differs at all, the row is TIED. Say so, name the units you
↪ checked,
and let input order break it. Restating what the two rows SHARE as though
↪ it
were a distinction is the same failure as calling them equivalent without
looking: `carrying does not answer in this crop` explains nothing when it
↪ is
equally true of the row you are comparing against.

And where the gap is inside its spread the comparison points NEITHER way, so
↪ no
wording can make it place anything. Write it as the tie it is: `0.69 +/- 0.13
against 0.70 +/- 0.15, inside the spread, so this figure does not place
↪ them`.

BEFORE EACH ROW, IN THIS ORDER. Not once for the table - once per row,
↪ against
the row you chose to compare with above.

1. LOCATION, unit by unit. For each of the units in turn, where does the
 ↪ query
 value sit for this candidate and for the row above: read in THIS CROP,
 ↪ read
 in the track but NOT in this crop, near only, or never read anywhere?
 ↪ This
 is a comparison of words, not a judgement. THE FIRST UNIT WHERE THE TWO
 DIFFER IS THE REASON - take it and stop looking.
2. COUNTS, for the units whose location matched: does `n/N` differ?
3. FIGURES, for the units where location and counts both matched: does a
 linkage or cohesion gap survive its spread?
4. Only when all of 1, 2 and 3 come back the same on EVERY unit is the row
 TIED.

A location difference on hair_style or lower_type places a candidate exactly
 ↪ as
 much as one on age_group. The unit you ordered the list by is not the unit
 ↪ that
 separates every neighbouring pair, and by step 1 you will usually find the
 separator in a unit you have not mentioned yet.

EVERY ROW IS ONE OF TWO SHAPES, and you choose only after the four steps
 ↪ above.

DISTINGUISHING the measurement that differs from the neighbour, with both
 figures, and what it means for the two positions.

TIED why NO unit separates them - naming the units you checked

↪ -

and then, last, that input order breaks the tie.

TIED IS A CLAIM ABOUT ALL THE UNITS, not about the one you looked at first.

↪ Two

candidates are not tied because their age evidence matches; they are tied

↪ when

you have been through every unit and none of them differs materially. A query
 value read in THIS CROP for one candidate and only in the track for the other

↪ is

NEVER a tie, on any attribute - that is a difference, and it is the reason.

INPUT ORDER CANNOT JUSTIFY INCLUSION. It resolves a tie you have already established; it never explains why a candidate is in the rows you wrote at
→ all.

`equivalent to the rows around it, input order` on its own is not a reason -
→ it
is the absence of one.

TIED, written properly:

`Tied with the rows above and below: teenager is an isolated 2/10 track reading in all three, carrying is a near backpack match in all three, and hair, lower and colour are read the same way; input order breaks the tie.`

EVERY RELIABILITY CLAIM MUST CARRY THE FIGURE THAT SHOWS IT. This is what separates a reason from a decoration. "Separated by linkage" is decoration. "Linkage favours skirt 0.78 against 0.64, so the crop's shorts reading is the undercut one" is a reason. If you cannot cite a figure for a claim, do not make the claim.

A reason answers ONE question: why HERE and not one place higher or lower. The comparison is with its NEIGHBOURS, and the reason should name what they have or lack. "This candidate has reliable age, hair, shorts and red" is a summary and places nothing.

Good:

"Compared with row 9: tied across all units (teenager track 1/10, backpack
→ near

match, red 10/10, shorts 8/10, female 9/10, short hair 6/10), with backpack linkage at 0.61 +/- 0.09 against row 9's 0.66 +/- 0.16, inside the spread so figures do not separate them, input order breaks the tie."

Names the row, checks every unit, quotes both figures, applies the spread, and reaches input order only after the tie is established.

"Compared with row 1: shares teenager read in this crop (2/10 track frames)
→ and

uncontested red, shorts and female, but sits here because hair style reads straight in this crop with linkage 0.91 against short 0.64."

Names what is shared, then the one unit that differs, with its figures.

Bad:

"Exact red top, shorts, female gender and short hair, separated by strong identity linkage."

A roll-call plus a mechanism with no figure. Neither half places anything, and every attribute it lists is shared by most of the pool.

STOP

never open with a roll-call of attributes that match, and never name more
→ than

TWO attributes in one reason

never name an attribute that reads the same way for most of this pool, or
→ for

the row you compared with - it cannot place either of them

never write a reason that contains no figure from the notes

never copy a note's opening sentence as the reason - the opening states

→ what

was read, the placing evidence is what follows it

never reinterpret a raw metric yourself, and never call a reading better,

stronger or more reliable because a number is higher or lower

never write: correct, incorrect, true value, winner, most likely, trust X

never write a row without having compared EVERY unit's location against the

row you compared with, and without naming that row's rank

never cite a difference - figure or location - that favours this candidate

over the row it sits below

never present something both rows share as the difference that separates

→ them

never call two candidates tied without having checked every unit, and never

when a query value is read in this crop for one and only in the track for

the other, on ANY attribute

never use input order to explain why a candidate is in the rows you wrote

never let any wording - rises, drops, stronger, further, better, favours -

claim a direction for a gap that sits inside its spread

never compare two figures that measure different things

never assert that a named row shares a reading without checking that row

never write fewer than {N_KEEP} rows, and never keep writing once the

evidence stops separating them

never pick the rows by reading only the first candidates in the list - what
you write is the top of a full comparison
every candidate exactly once - a candidate ranked twice loses another one

OUTPUT

Return ONLY this table:

Rank	Candidate	Reason
---	---	---

AT LEAST {N_KEEP} rows, ranked 1..N with no omissions and no repeats. A

↪ candidate

ranked twice costs another candidate its place.

THREE COLUMNS ONLY.

Reason: 1-3 sentences.

1	candidate 22	Teenager is established across frames that cohere at ↪ 0.76 +/- 0.12 over 21 pairs, where the rows below it reach teenager only ↪ as an isolated frame.
2	candidate 17	Its clothing reads cleanly but that is shared with most ↪ of this pool; it sits here because teenager is a single isolated frame ↪ and carrying reaches no closer than shoulder bag at 13%.
3	candidate 65	Equivalent to the rows around it on the evidence, input ↪ order.

User message.

SOURCE DATA

Query

{QUERY}

What each query value is worth in the index

{WORTH}

Candidates

{VERDICTS}

A.6 The coverage blocks

{COVERAGE} in the ranker's system prompt is one of these two.

Coarse round (75 → 25).

WHAT MUST NOT BE CUT

This is the COARSE round. Everything you place below the cut is gone and no later stage can bring it back, while a candidate you merely rank low can

→ still

be recovered by the round after this one. The two mistakes are not symmetric.

PRESENT means read ANYWHERE, exactly OR as a [near] value standing in for it: in one frame, as a minority reading, overshadowed by a competing value, or in the track but not in this crop. NEAR MATCH ONLY is present. A candidate

→ reading

a value either of those ways outranks a candidate whose notes say that value

→ is

read NOWHERE, when the table above calls it decisive. Present but weak beats absent.

ABSENT is the EVIDENCE GAP standing, in either of the two forms the notes

→ use:

`read nowhere in the track` - silence - and `nothing closer than [far] read anywhere` - contradiction. Neither is weakness, and a [far] reading is NOT a weak match for this floor to protect: it is a mismatch, and WHAT MAKES A

→ READING

WEIGH still sinks it.

This is NOT counting how many attributes match, and it is not a score. It

→ bites

in one place only: an ABSENCE, of a value the table calls decisive. An

→ absence

of a value the table calls almost nothing is not a reason for anything, and
→ two
candidates that both reach a value - exactly or near - are ordered by the
evidence as usual, never by which of them reads it in more frames.

Final round (25 → 10).

NOTHING IS BELOW THE CUT NOW

This is the FINAL round. What you rank is what is returned, so there is

→ nothing

below your cut left to protect and no later stage to recover anything for.

Rank on the evidence as it stands. A query value read nowhere - or read only

→ as

[far] - is still a reliable mismatch and should still sink a candidate; it
simply no longer earns a candidate a place its evidence has not earned. A

→ single

overshadowed frame, or a [near] value standing in for the query value, is

→ what

it is: weak positive evidence, weighed against everything else, and not a
floor.